CHAPTER 1

Getting Up and Blinking with the Arduino

Parts You'll Need for This Chapter:

Arduino Uno

USB cable

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, videos, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch1`.

In addition, all code can be found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 01 download and individually named according to the names throughout the chapter.

Now that you have some perspective on the Arduino platform and its capabilities, it's time to explore your options in the world of Arduino. In this chapter, you examine the available hardware, learn about the programming environment and language, and get your first program up and running. Once you have a grip on the functionality that the Arduino can provide, you'll write your first program and get the Arduino to blink!

NOTE To follow along with a video that introduces the Arduino platform, visit `www.jeremyblum.com/2011/01/02/arduino-tutorial-series-it-begins/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Exploring the Arduino Ecosystem

In your adventures with the Arduino, you'll depend on three main components for your projects:

- The Arduino board itself
- External hardware (including both shields and hand-made circuits, which you'll explore throughout this book)
- The Arduino integrated development environment, or Arduino IDE

All these system components work in tandem to enable you do just about anything with your Arduino.

You have a lot of options when it comes to Arduino development boards, but this book focuses on using official Arduino boards. Because the boards are all designed to be programmable via the same IDE, you can generally use any of the modern Arduino boards to complete the projects in this book with zero or minor changes. However, when necessary, you'll see caveats about using different boards for various projects. The majority of the projects use the Arduino Uno.

You start by exploring the basic functionality baked in to every Arduino board. Then you examine the differences between each modern board so that you can make an informed decision when choosing a board to use for your next project.

Arduino Functionality

All Arduino boards have a few key capabilities and functions. Take a moment to examine the Arduino Uno (see Figure 1-1); it will be your base configuration. These are some key components that you'll be concerning yourself with:

- Atmel microcontroller
- USB programming/communication interface(s)
- Voltage regulator and power connections
- Breakout I/O pins
- Debug, Power, and RX/TX LEDs
- Reset button
- In-circuit serial programmer (ICSP) connector(s)

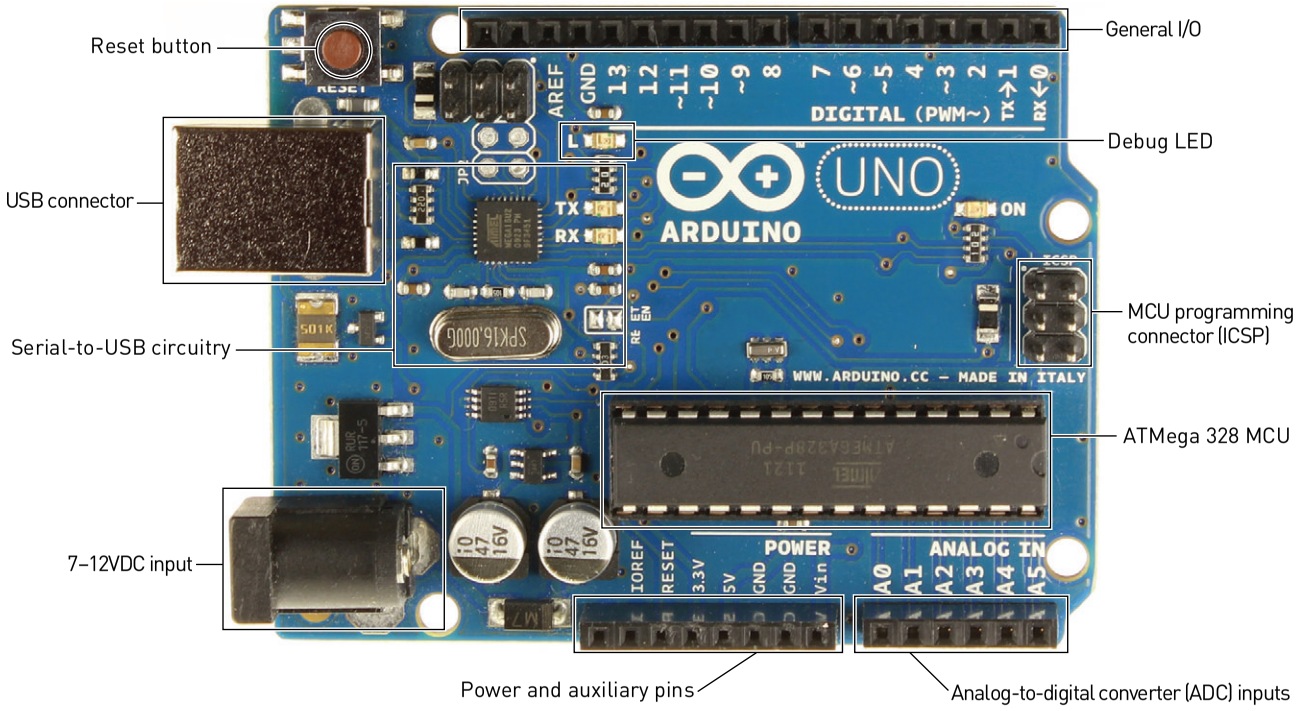


Figure 1-1: Arduino Uno components

Credit: Arduino, `www.arduino.cc`

Atmel Microcontroller

At the heart of every Arduino is an Atmel microcontroller unit (MCU). Most Arduino boards, including the Arduino Uno, use an AVR ATMega microcontroller. The Arduino Uno in Figure 1-1 uses an ATMega 328p. The Due is an exception; it uses an ARM Cortex microcontroller. This microcontroller is responsible for holding all of your compiled code and executing the commands you specify. The Arduino programming language gives you access to microcontroller peripherals, including analog-to-digital converters (ADCs), general-purpose input/output (I/O) pins, communication buses (including I^2C and SPI), and serial interfaces. All of this useful functionality is broken out from the tiny pins on the microcontroller to accessible female headers on the Arduino that you can plug wires or shields into. A 16 MHz ceramic resonator is wired to the ATMega's clock pins, which serves as the reference by which all program commands execute. You can use the Reset button to restart the execution of your program. Most Arduino boards come with a debug LED already connected to pin 13, which enables you to run your first program (blinking an LED) without connecting any additional circuitry.

Programming Interfaces

Ordinarily, ATMega microcontroller programs are written in C or Assembly and programmed via the ICSP interface using a dedicated programmer (see Figure 1-2). Perhaps the most important characteristic of an Arduino is that you can program it easily via USB, without using a separate programmer. This functionality is made possible by the Arduino bootloader. The bootloader is loaded onto the ATMega at the factory (using the ICSP header), which allows a serial USART (Universal Synchronous/Asynchronous Receiver/Transmitter) to load your program on the Arduino without using a separate programmer. (You can learn more about how the bootloader functions in "The Arduino Bootloader and Firmware Setup" sidebar.)

In the case of the Arduino Uno and Mega 2560, a secondary microcontroller (an ATMega 16U2 or 8U2 depending on your revision) serves as an interface between a USB cable and the serial USART pins on the main microcontroller. The Arduino Leonardo, which uses an ATMega 32U4 as the main microcontroller, has USB baked right in, so a secondary microcontroller is not needed. In older Arduino boards, an FTDI brand USB-to-serial chip was used as the interface between the ATMega's serial USART port and a USB connection.



Credit: © 2013 Atmel Corporation.
All rights reserved.

Figure 1-2: AVR ISP MKII programmer

General I/O and ADCs

The part of the Arduino that you'll care the most about during your projects is the general-purpose I/O and ADC pins. All of these pins can be individually addressed via the programs you'll write. All of them can serve as digital inputs and outputs. The ADC pins can also act as analog inputs that can measure voltages between 0 and 5V (usually from resistive sensors). Many of these pins are also multiplexed to serve additional functions, which you will explore during your projects. These special functions include various communication interfaces, serial interfaces, pulse-width-modulated outputs, and external interrupts.

Power Supplies

For the majority of your projects, you will simply use the 5V power that is provided over your USB cable. However, when you're ready to untether your project from a computer, you have other power options. The Arduino can accept between 6V and 20V (7-12V recommend) via the direct current (DC) barrel jack connector, or into the V_{in} pin. The Arduino has built-in 5V and 3.3V regulators:

- 5V is used for all the logic on the board. In other words, when you toggle a digital I/O pin, you are toggling it between 5V and 0V.
- 3.3V is broken out to a pin to accommodate 3.3V shields and external circuitry.

THE ARDUINO BOOTLOADER AND FIRMWARE SETUP

A *bootloader* is a chunk of code that lives in a reserved space in the program memory of the Arduino's main MCU. In general, AVR microcontrollers are programmed with an ICSP, which talks to the microcontroller via a serial peripheral interface (SPI). Programming via this method is fairly straightforward, but necessitates the user having a hardware programmer such as an STK500 or an AVR ISP MKII programmer (see Figure 1-2).

When you first boot the Arduino board, it enters the bootloader, which runs for a few seconds. If it receives a programming command from the IDE over the MCU's UART (serial interface) in that time period, it loads the program that you are sending it into the rest of the MCU's program memory. If it does not receive a programming command, it starts running your most recently uploaded sketch, which resides in the rest of the program memory.

When you send an "upload" command from the Arduino IDE, it instructs the USB-to-serial chip (an ATMega 16U2 or 8U2 in the case of the Arduino Uno) to reset the main MCU, hence forcing it into the bootloader. Then, your computer immediately begins to send the program contents, which the MCU is ready to receive over its UART connection (facilitated by the USB-to-serial converter).

Bootloaders are great because they enable simple programming via USB with no external hardware. However, they do have two downsides:

- First, they take up valuable program space. If you have written a complicated sketch, the approximately 2KB of space taken up by the bootloader might be really valuable.
- Second, using a bootloader means that your program will always be delayed by a few seconds at boot as the bootloader checks for a programming request.

If you have a programmer (or another Arduino that can be programmed to act as a programmer), you can remove the bootloader from your ATMega and program it directly by connecting your programmer to the ICSP header and using the File ➪ Upload Using Programmer command from within the IDE.

Arduino Boards

This book cannot possibly cover all the available Arduino boards; there are many, and manufacturers are constantly releasing new ones with various features. The following section highlights some of the features in the official Arduino boards.

The Uno (see Figure 1-3) is the flagship Arduino and will be used heavily in this book. It uses a 16U2 USB-to-serial converter chip and an ATMega 328p as the main MCU. It is available in both DIP and SMD versions (which defines whether the MCU is removable).

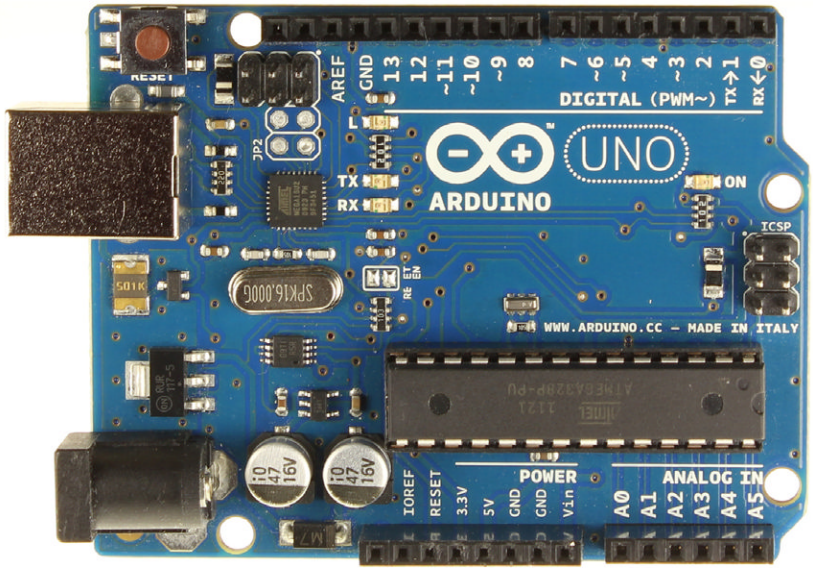


Figure 1-3: The Arduino Uno

The Leonardo (see Figure 1-4) uses the 32U4 as the main microcontroller, which has a USB interface built in. Therefore, it doesn't need a secondary MCU to perform the serial-to-USB conversion. This cuts down on the cost and enables you to do unique things like emulate a joystick or a keyboard instead of a simple serial device. You will learn how to use these features in Chapter 6, "USB and Serial Communication."

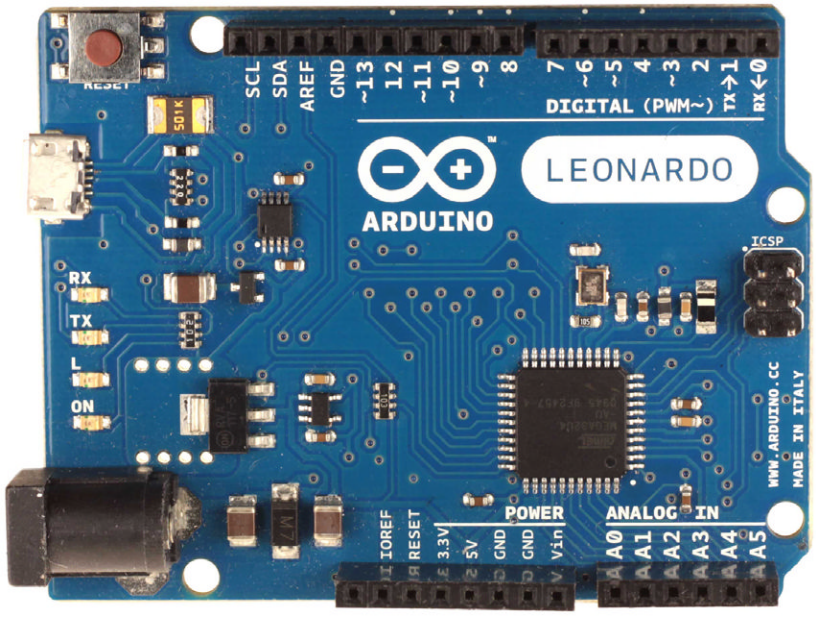
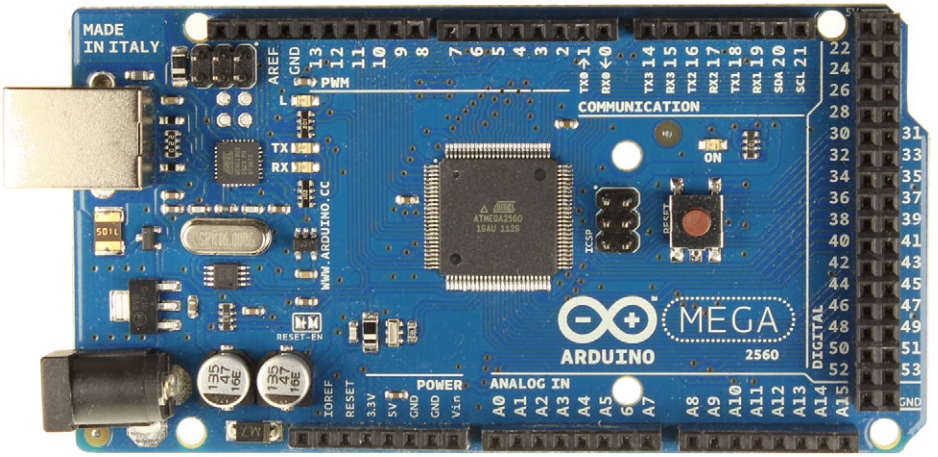


Figure 1-4: The Arduino Leonardo

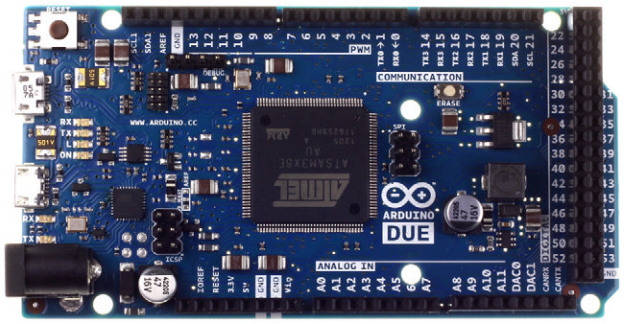
The Mega 2560 (see Figure 1-5) employs an ATMega 2560 as the main MCU, which has 54 general I/Os to enable you to interface with many more devices. The Mega also has more ADC channels, and has four hardware serial interfaces (unlike the one serial interface found on the Uno).



Credit: Arduino, www.arduino.cc

Figure 1-5: The Arduino Mega 2560

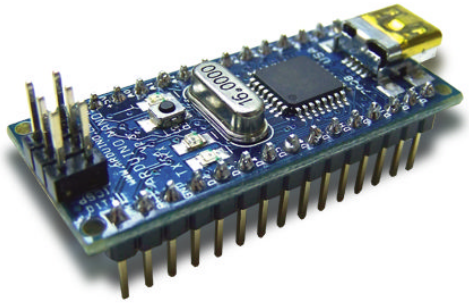
Unlike all the other Arduino variants, which use 8-bit AVR MCUs, the Due (see Figure 1-6) uses a 32-bit ARM Cortex M3 SAM3X MCU. The Due offers higher-precision ADCs, selectable resolution pulse-width modulation (PWM), Digital-to-Analog Converters (DACs), a USB host connector, and an 84 MHz clock speed.



Credit: Arduino, www.arduino.cc

Figure 1-6: The Arduino Due

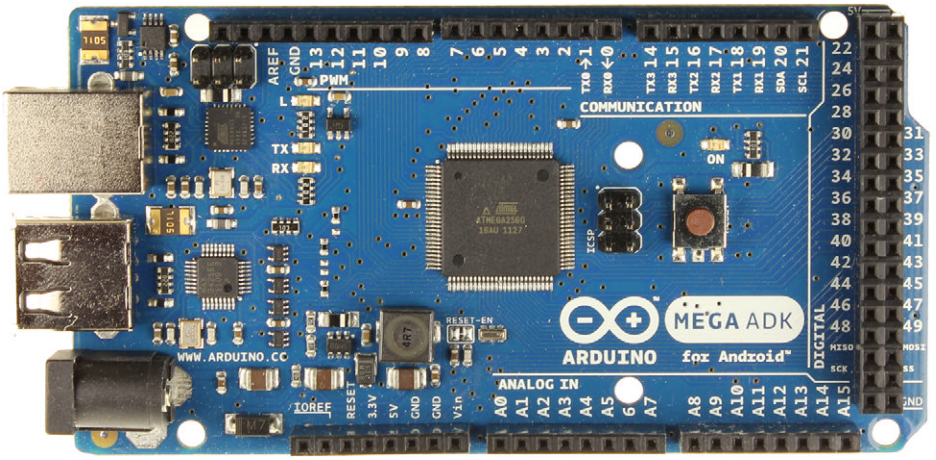
The Nano (see Figure 1-7) is designed to be mounted right into a breadboard socket. Its small form factor makes it perfect for use in more finished projects.



Credit: Cooking Hacks, www.cookinghacks.com

Figure 1-7: The Arduino Nano

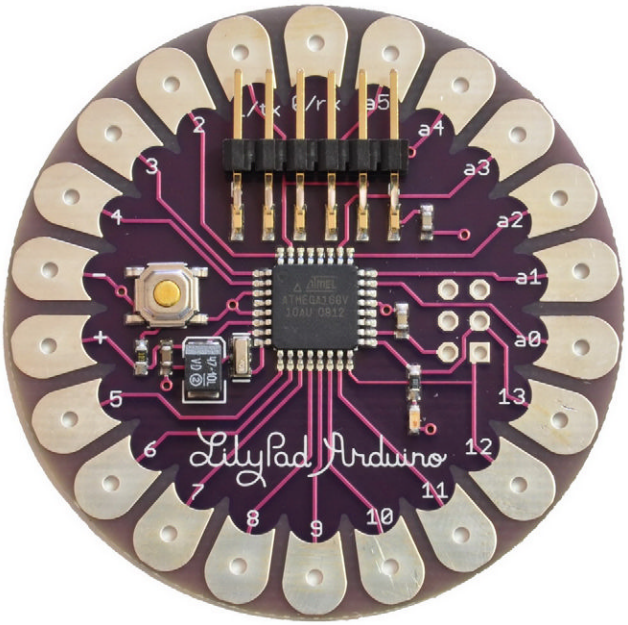
The Mega ADK (see Figure 1-8) is very similar to the Mega 2560, except that it has USB host functionality, allowing it to connect to an Android phone so that it can communicate with apps that you write.



Credit: Arduino, www.arduino.cc

Figure 1-8: The Arduino Mega ADK

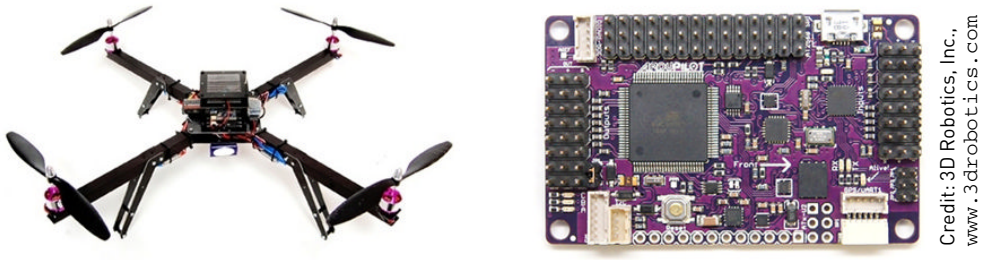
The LilyPad (see Figure 1-9) is unique because it is designed to be sewn into clothing. Using conductive thread, you can wire it up to sewable sensors, LEDs, and more. To keep size down, you need to program it using an FTDI cable.



Credit: Arduino, www.arduino.cc

Figure 1-9: The LilyPad Arduino

As explained in this book's introduction, the Arduino is open source hardware. As a result, you can find dozens and dozens of "Arduino compatible" devices available for sale that will work just fine with the Arduino IDE and all the projects you'll do in this book. Some of the popular third-party boards include the Seeduino, the adafruit 32U4 breakout board, and the SparkFun Pro Mini Arduino boards. Many third-party boards are designed for very particular applications, with additional functionality already built in to the board. For example, the ArduPilot is an autopilot board for use in autonomous DIY quadcopters (see Figure 1-10). You can even find Arduino-compatible circuitry baked in to consumer devices like the MakerBot Replicator and Replicator 2 3D printers.



Credit: 3D Robotics, Inc., www.3drobotics.com

Figure 1-10: Quadcopter and ArduPilot Mega controller

Creating Your First Program

Now that you understand the hardware that you'll be using throughout this book, you can install the software and run your first program. Start by downloading the Arduino software to your computer.

Downloading and Installing the Arduino IDE

Access the Arduino website at `www.arduino.cc` and download the newest version of the IDE from the Download page (see Figure 1-11).

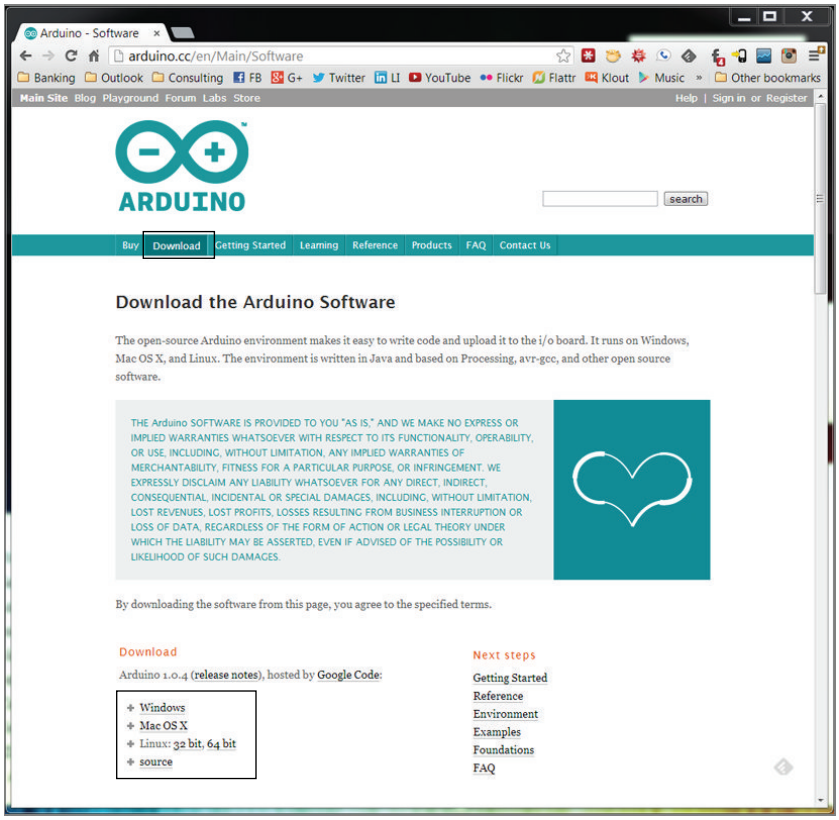


Figure 1-11: The Arduino.cc Download page

After completing the download, unzip it. Inside, you'll find the Arduino IDE. New versions of the Windows IDE are available as an installer that you can download and run, instead of downloading a ZIP file.

Running the IDE and Connecting to the Arduino

Now that you have the IDE downloaded and ready to run, you can connect the Arduino to your computer via USB, as shown in Figure 1-12. Mac and Linux machines install the drivers (mostly) automatically.

If you are using OS X, the first time you plug in an Uno or a Mega 2560, you will get a notification that a new network device has been added. Click the Network Preferences button. In the new window, click Apply. Even though the board will appear as "Not Configured" in the network device list, it will be ready to use. Now, quit System Preferences.

If you are using a modern Arduino on a Windows computer, you will probably need to install drivers. You can skip the following directions if you are not using a Windows computer that needs to have drivers installed. If you installed the IDE using the Windows installer, then these steps have been completed for you. If you downloaded the ZIP on your Windows machine, then you will need to follow the directions shown next.

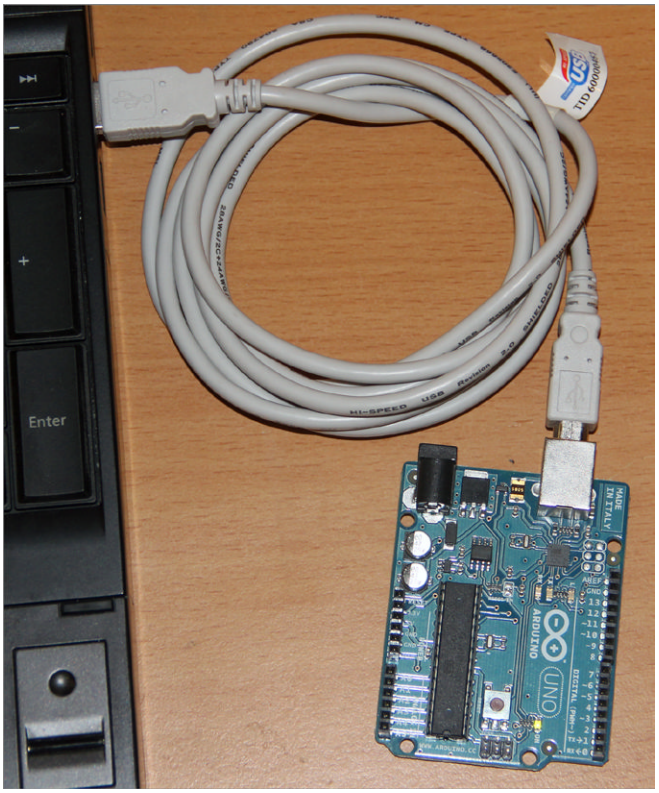


Figure 1-12: Arduino Uno connected to a computer via USB

On your Windows computer, follow these steps to install the drivers (instructions adapted from the Arduino.cc website):

1. Wait for the automatic install process to fail.
2. Open the Start menu, right-click My Computer, and select Properties.
3. Choose Device Manager.
4. Look under Ports (COM and LPT) for the Arduino that you connected.
5. Right-click it and choose Update Driver Software.
6. Choose to browse your computer for software.
7. Select the appropriate driver from the drivers directory of the Arduino IDE that you just downloaded (not the FTDI drivers directory).
8. Windows will now finish the driver installation.

Now, launch the Arduino IDE. You're ready to load your first program onto your Arduino. To ensure that everything is working as expected, you'll load the Blink example program, which will blink the onboard LED. Most Arduinos have an LED connected to pin 13. Navigate to File ➪ Examples ➪ Basic, and click the Blink program. This opens a new IDE window with the Blink program already written for you. First, you'll program the Arduino with this example sketch, and then you'll analyze the program to understand the important components so that you can start to write your own programs in the next chapter.

Before you load the program, you need to tell the IDE what kind of Arduino you have connected and what port it is connected to. Go to Tools ➪ Board and ensure that the right board is selected. This example uses the Uno, but if you are using a different board, select that one (assuming that it also has an LED connected to pin 13).

The last step before programming is to tell the IDE what port your board is connected to. Navigate to Tools ➪ Serial Port and select the appropriate port. On Windows machines, this will be `COM*`, where * is some number representing the serial port number.

TIP If you have multiple serial devices attached to your computer, try unplugging your board to see which COM port disappears from the menu; then plug it back in and select that COM port.

On Linux and Mac computers, the serial port looks something like `/dev/tty.usbmodem*` or `/dev/tty.usbserial*`, where `*` is a string of alphanumeric characters.

You're finally ready to load your first program. Click the Upload button ( ) on the top left of the IDE. The status bar at the bottom of the IDE shows a progress bar as it compiles and uploads your program. When the upload completes, the yellow LED on your Arduino should be blinking once per second. Congratulations! You've just uploaded your first Arduino program.

Breaking Down Your First Program

Take a moment to deconstruct the Blink program so that you understand the basic structure of programs written for the Arduino. Consider Figure 1-13. The numbered callouts shown in the figure correspond to the following list.

Here's how the code works, piece by piece:

1. This is a multiline comment. Comments are important for documenting your code. Everything you write between these symbols will not be compiled or even seen by your Arduino. Multiline comments start with `/*` and end with `*/`. Multiline comments are generally used when you have to say a lot (like the description of this program).
2. This is a single-line comment. When you put `//` on any line, the compiler ignores all text after that symbol on the same line. This is great for annotating specific lines of code or for "commenting out" a particular line of code that you believe might be causing problems.
3. This code is a variable declaration. A variable is a place in the Arduino's memory that holds information. Variables have different types. In this case, it's of type `int`, which means it will hold an integer. In this case, an integer variable called `led` is being set to the value of `13`, the pin that the LED is connected to on the Arduino Uno. Throughout the rest of the program, we can simply use `led` whenever we want to control pin 13. Setting variables is useful because you can just change this one line if you hook up your LED to a different I/O pin later on; the rest of the code will still work as expected.
4. `void setup()` is one of two functions that must be included in every Arduino program. A *function* is a piece of code that does a specific task. Code within the curly braces of the `setup()` function is executed once at the start of the program. This is useful for one-time settings, such as setting the direction of pins, initializing communication interfaces, and so on.

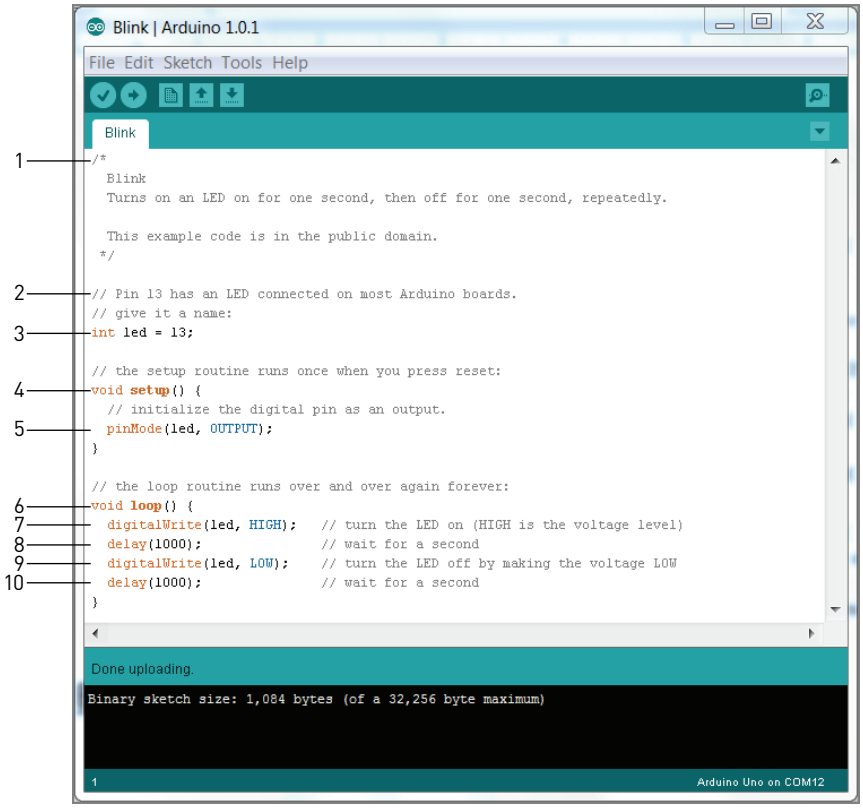


Figure 1-13: The components of the Blink program

5. The Arduino's digital pins can function as input or outputs. To configure their direction, use the command `pinMode()`. This command takes two arguments. An *argument* gives commands information on how they should operate. Arguments are placed inside the parentheses following a command. The first argument to `pinMode` determines which pin is having its direction set. Because you defined the `led` variable earlier in the program, you are telling the command that you want to set the direction of pin 13. The second argument sets the direction of the pin: `INPUT` or `OUTPUT`. Pins are inputs by default, so you need to explicitly set them to outputs if you want them to function as outputs. Because you want to light an LED, you have set the led pin to an output (current is flowing out of the I/O pin). Note that you have to do this only one time. It will then function as an output for the rest of the program, or until you change it to an input.

6. The second required function in all Arduino programs is `void loop()`. The contents of the loop function repeat forever as long as the Arduino is on. If you want your Arduino to do something once at boot only, you still need to include the loop function, but you can leave it empty.
7. `digitalWrite()` is used to set the state of an output pin. It can set the pin to either 5V or 0V. When an LED and resistor is connected to a pin, setting it to 5V will enable you to light up the LED. (You learn more about this in the next chapter.) The first argument to `digitalWrite()` is the pin you want to control. The second argument is the value you want to set it to, either `HIGH` (5V) or `LOW` (0V). The pin remains in this state until it is changed in the code.
8. The `delay()` function accepts one argument: a delay time in milliseconds. When calling `delay()`, the Arduino stops doing anything for the amount of time specified. In this case, you are delaying the program for 1000ms, or 1 second. This results in the LED staying on for 1 second before you execute the next command.
9. Here, `digitalWrite()` is used to turn the LED off, by setting the pin state to `LOW`.
10. Again, we delay for 1 second to keep the LED in the off state before the loop repeats and switches to the on state again.

That's all there is to it. Don't be intimidated if you don't fully understand all the code yet. As you put together more examples in the following chapters, you'll become more and more proficient at understanding program flow, and writing your own code.

Summary

In this chapter you learned about the following:

- All the components that comprise an Arduino board
- How the Arduino bootloader allows you to program Arduino firmware over a USB connection
- The differences between the various available Arduino boards
- How to connect and install the Arduino with your system
- How to load and run your first program

CHAPTER

2

Digital Inputs, Outputs, and Pulse-Width Modulation

Parts You'll Need for This Chapter:

Arduino Uno

Small breadboard

Jumper wires

1 10kΩ resistor

3 220Ω resistors

USB cable

Pushbutton

5mm single-color LED

5mm common-cathode RGB LED

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, videos, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch2`.

In addition, all code can be found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 02 download and individually named according to the names throughout the chapter.

Blinking an LED is great, as you learned in the preceding chapter, but what makes the Arduino microcontroller platform so useful is that the system is equipped with both inputs and outputs. By combining both, your opportunities are nearly limitless. For example, you can use a magnetic reed switch to play music when your door opens, create an electronic lockbox, or build a light-up musical instrument!

In this chapter, you start to learn the skills you need to build projects like these. You explore the Arduino's digital input capabilities, learn about pullup and pulldown resistors, and learn how to control digital outputs. Most Arduinos do not have analog outputs, but it is possible to use digital pulse-width modulation to emulate it in many scenarios. You learn about generating pulse-width modulated signals in this chapter. You will also learn how to debounce digital switches, a key skill when reading human input. By the end of the chapter, you will be able to build and program a controllable RGB (Red, Green, Blue) LED nightlight.

NOTE You can follow along with a video as I teach you about digital inputs and outputs, debouncing, and pulse-width modulation (PWM): `www.jeremyblum.com/2011/01/10/arduino-tutorial-2-now-with-more-blinky-things/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

If you want to learn more about some of the basics of electrical engineering touched on in this chapter, watch this video: `www.jeremyblum.com/2011/01/17/electrical-engineering-basics-in-arduino-tutorial-3/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Digital Outputs

In Chapter 1, "Getting Up and Blinking with the Arduino," you learned how to blink an LED. In this chapter, you will further explore Arduino digital output capabilities, including the following topics:

- Setting pins as outputs
- Wiring up external components
- New programming concepts, including `for` loops and constants
- Digital versus analog outputs and pulse-width modulation (PWM)

Wiring Up an LED and Using Breadboards

In Chapter 1, you learned how to blink the onboard LED, but what fun is that? Now it is time to whip out the breadboard and wire up an external LED to pin 9 of your Arduino. Adding this external LED will be a stepping-stone towards helping you to understand how to wire up more complex external circuits in

the coming chapters. What's more, pin 9 is PWM-enabled, which will enable you to pursue the analog output examples later in this chapter.

Working with Breadboards

It is important to understand how breadboards work so that you can use them effectively for the projects in this book. A *breadboard* is a simple prototyping tool that easily allows you to wire up simple circuits without having to solder together parts to a custom printed circuit board. First, consider the blue and red lines that run the length of the board. The pins adjacent to these color-coded lines are designed to be used as power and ground buses. All the red pins are electrically connected, and are generally used for providing power. In the case of most Arduinos and the projects in this book, this will generally be at 5V. All the blue pins are electrically connected and are used for the ground bus. All the vertically aligned pins are also connected in rows, with a division in the middle to make it easy to mount integrated circuits on the breadboard. Figure 2-1 highlights how the pins are electrically connected, with all the thick lines representing connected holes.

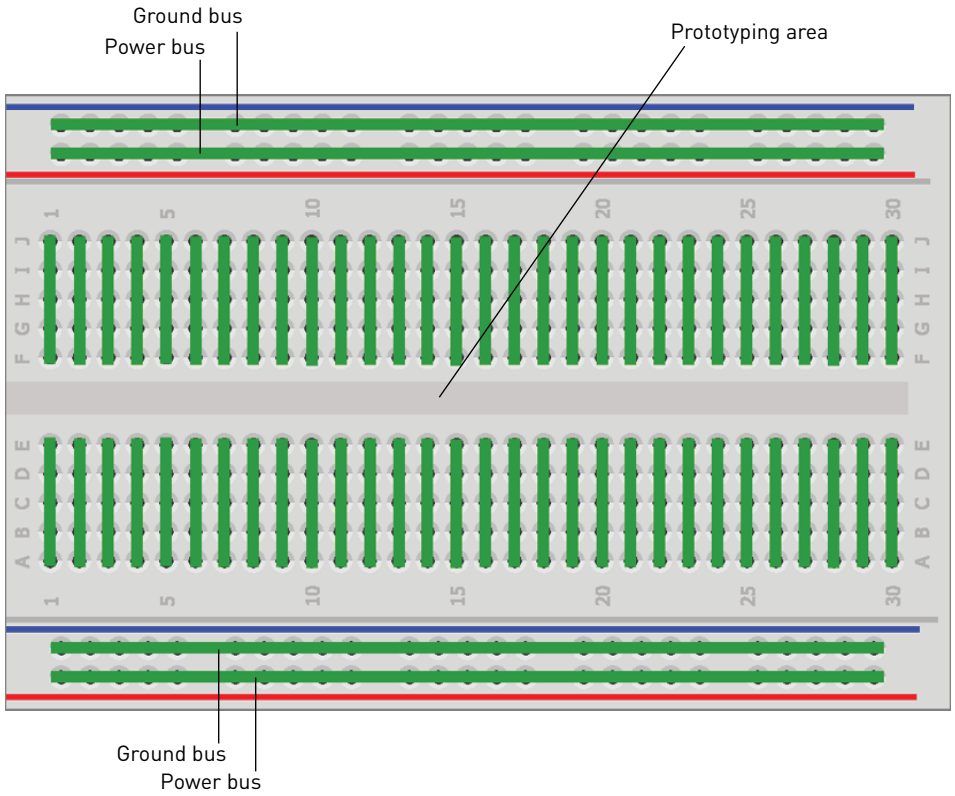


Figure 2-1: Breadboard electrical connections

Wiring LEDs

LEDs will almost certainly be one of the most-used parts in your projects throughout this book. LEDs are polarized; in other words, it matters in what direction you hook them up. The positive lead is called the *anode*, and the negative lead is called the *cathode*. If you look at the clear top of the LED, there will usually be a flat side on the lip of the casing. That side is the cathode. Another way to determine which side is the anode and which is the cathode is by examining the leads. The shorter lead is the cathode.

As you probably already know, LED stands for light-emitting diode. Like all diodes, LEDs allow current to flow in only one direction—from their anode to their cathode. Because current flows from positive to negative, the anode of the LED should be connected to the current source (a 5V digital signal in this case), and the cathode should be connected to ground. The resistor can be inserted in series on either side of the LED. Resistors are not polarized, and so you do not have to worry about their orientation.

You'll wire the LED into pin 9 in series with a resistor. LEDs must always be wired in series with a resistor to serve as a current limiter. The larger the resistor value, the more it restricts the flow of current and the dimmer the LED glows. In this scenario, you use a 220Ω resistor. Wire it up as shown in Figure 2-2.

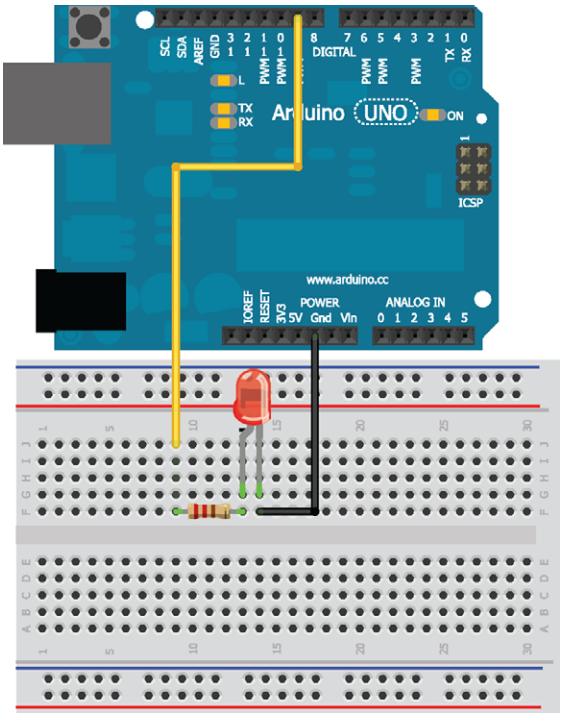


Image created with Fritzing.

Figure 2-2: Arduino Uno wired to an LED

OHM'S LAW AND THE POWER EQUATION

The most important equation for any electrical engineer to know is Ohm's law. Ohm's law dictates the relationship between voltage (measured in volts), current (measured in amps), and resistance (measured in ohms or Ω) in a circuit. A circuit is a closed loop with a source of electrical energy (like a 9V battery) and a load (something to use up the energy, like an LED). Before delving into the law, it is important to understand what each term means, at least at a basic level:

- *Voltage* represents the potential electrical difference between two points.
- *Current* flows from a point of higher potential energy to lower potential energy. You can think of current as a flow of water, and voltage as elevation. Water (or current) always flows from high elevation (higher voltage) to lower elevation (ground, or a lower voltage). Current, like water in a river, will always follow the path of least resistance in a circuit.
- *Resistance*, in this analogy, is representative of how easy it is for current to flow. When the water (the current) is flowing through a narrow pipe, less can pass through in the same amount of time as through a larger pipe. The narrow pipe is equivalent to a high resistance value because the water will have a harder time flowing through. The wider pipe is equivalent to a low resistance value (like a wire) because current can flow freely through it.

Ohm's law is defined as follows:

V = IR

Where V is Voltage difference in volts, I is Current in amps, and R is the Resistance in ohms.

In a circuit, all voltage gets used up, and each component offers up some resistance that lowers the voltage. Knowing this, the above equation comes in handy for things like figuring out what resistor value to match up with an LED. LEDs have a predefined voltage drop across them and are designed to operate at a particular current value. The larger the current through the LED, the brighter the LED glows, up to a limit. For the most common LEDs, the maximum current designed to go through an LED is 20milliamps (a milliamp is 1/1000 of an amp and is typically abbreviated as mA). The voltage drop across an LED is defined in its datasheet. A common value is around 2V. Consider the LED circuit shown in Figure 2-3.

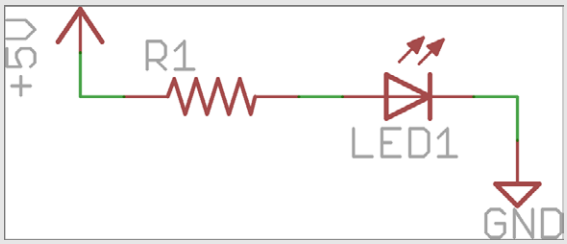


Image created with Eagle.

Figure 2-3: Simple LED circuit

Continues

continued

You can use Ohm's law to decide on a resistor value for this circuit. Assume that this is a standard LED with 20mA forward current and a 2V drop across it. Because the source voltage is 5V and it ends at ground, a total of 5V must drop across this circuit. Since the LED has a 2V drop, the other 3V must drop across the resistor. Knowing that you want approximately 20mA to flow through these components, you can find the resistor value by solving for R:

R = V/I

Where V = 3V and I = 20mA.

Solving for R, R = 3V / 0.02A = 150Ω. So, with a resistor value of 150Ω, 20mA flows through both the resistor and LED. As you increase the resistance value, less current is allowed to flow through. 220Ω is a bit more than 150Ω, but still allows the LED to glow sufficiently bright, and is a very commonly available resistor value.

Another useful equation to keep in mind is the power equation. The power equation tells you how much power, in watts, is dissipated across a given resistive component. Because increased power is associated with increased heat dissipation, components generally have a maximum power rating. You want to ensure that you do not exceed the maximum power rating for resistors because otherwise they might overheat. A common power rating for resistors is 1/8 of a watt (abbreviated as W, milliwatts as mW). The power equation is as follows:

P = IV

Where P is power in watts, and I and V are still defined as the current and voltage.

For the resistor defined earlier with a voltage drop of 3V and a current of 20mA, P = 3V × 0.02A = 60mW, well under the resistor's rating of 1/8W, or 125mW. So, you do not have to worry about the resistor overheating; it is well within its operating limits.

Programming Digital Outputs

By default, all Arduino pins are set to inputs. If you want to make a pin an output, you need to first tell the Arduino how the pin should be configured. In the Arduino programming language, the program requires two parts: the `setup()` and the `loop()`.

As you learned in Chapter 1, the `setup()` function runs one time at the start of the program, and the `loop()` function runs over and over again. Because you'll generally dedicate each pin to serve as either an input or an output, it is

common practice to define all your pins as inputs or outputs in the setup. You start by writing a simple program that sets pin 9 as an output and turns it on when the program starts.

To write this program, use the `pinMode()` command to set the direction of pin 9, and use `digitalWrite()` to make the output high (5V). See Listing 2-1.

Listing 2-1: Turning on an LED—led.ino

```
const int LED=9;                //define LED for pin 9
void setup()
{
  pinMode (LED, OUTPUT);     //Set the LED pin as an output
  digitalWrite(LED, HIGH);   //Set the LED pin high
}

void loop()
{
  //we are not doing anything in the loop!
}
```

Load this program onto your Arduino, wired as shown in Figure 2-2. In this program, also notice that I used the `const` operator before defining the pin integer variable. Ordinarily, you'll use variables to hold values that may change during program execution. By putting `const` before your variable declaration, you are telling the compiler that the variable is "read only" and will not change during program execution. All instances of `LED` in your program will be "replaced" with `9` when they are called. When you are defining values that will not change, using the `const` qualifier is recommended. In some of the examples later in this chapter, you will define non-constant variables that may change during program execution.

You must specify the type for any variable that you declare. In the preceding case, it is an integer (pins will always be integers), so you should set it as such. You can now easily modify this sketch to match the one you made in Chapter 1 by moving the `digitalWrite()` command to the loop and adding some delays. Experiment with the delay values and create different blink rates.

Using For Loops

It's frequently necessary to use loops with changing variable values to adjust parameters of a program. In the case of the program you just wrote, you can implement a `for` loop to see how different blink rates impact your system's operation. You can visualize different blink rates by using a `for` loop to cycle through various rates. The code in Listing 2-2 accomplishes that.

Listing 2-2: LED with Changing Blink Rate—blink.ino

```
const int LED=9;              //define LED for Pin 9
void setup()
{
  pinMode (LED, OUTPUT); //Set the LED pin as an output
}

void loop()
{
  for (int i=100; i<=1000; i=i+100)
  {
    digitalWrite(LED, HIGH);
    delay(i);
    digitalWrite(LED, LOW);
    delay(i);
  }
}
```

Compile the preceding code and load it onto your Arduino. What happens? Take a moment to break down the `for` loop to understand how it works. The `for` loop declaration always contains three semicolon-separated entries:

- The first entry sets the index variable for the loop. In this case, the index variable is `i` and is set to start at a value of `100`.
- The second entry specifies when the loop should stop. The contents of the loop will execute over and over again while that condition is true. `<=` indicates less than or equal to. So, for this loop, the contents will continue to execute as long as the variable `i` is still less than or equal to `1000`.
- The final entry specifies what should happen to the index variable at the end of each loop execution. In this case, `i` will be set to its current value plus `100`.

To better understand these concepts, consider what happens in two passes through the `for` loop:

1. `i` equals `100`.
2. The LED is set high, and stays high for 100ms, the current value of `i`.
3. The LED is set low, and stays low for 100ms, the current value of `i`.
4. At the end of the loop, `i` is incremented by `100`, so it is now `200`.
5. `200` is less than or equal to `1000`, so the loop repeats again.
6. The LED is set high, and stays high for 200ms, the current value of `i`.
7. The LED is set low, and stays low for 200ms, the current value of `i`.

8. At the end of the loop, `i` is incremented by `100`, so it is now `300`.
9. This process repeats until `i` surpasses `1000` and the outer loop function repeats, setting the `i` value back to `100` and starting the process again.

Now that you've generated digital outputs from your Arduino, you'll learn about using PWM to create analog outputs from the I/O pins on your Arduino.

Pulse-Width Modulation with analogWrite()

So, you have mastered digital control of your pins. This is great for blinking LEDs, controlling relays, and spinning motors at a constant speed. But what if you want to output a voltage other than 0V or 5V? Well, you can't—unless you are using the digital-to-analog converter (DAC) pins on the Due or are using an external DAC chip.

However, you can get pretty close to generating analog output values by using a trick called *pulse-width modulation* (PWM). Select pins on each Arduino can use the `analogWrite()` command to generate PWM signals that can emulate a pure analog signal when used with certain peripherals. These pins are marked with a ~ on the board. On the Arduino Uno, Pins 3, 5, 6, 9, 10, and 11 are PWM pins. If you're using an Uno, you can continue to use the circuit from Figure 2-1 to test out the `analogWrite()` command with your LED. Presumably, if you can decrease the voltage being dropped across the resistor, the LED should glow more dimly because less current will flow. That is what you will try to accomplish using PWM via the `analogWrite()` command. The `analogWrite()` command accepts two arguments: the pin to control and the value to write to it.

The PWM output is an 8-bit value. In other words, you can write values from 0 to 2^8-1, or 0 to 255. Try using a similar `for` loop structure to the one you used previously to cycle through varying brightness values (see Listing 2-3).

Listing 2-3: LED Fade Sketch—fade.ino

```
const int LED=9;   //define LED for Pin 9
void setup()
{
  pinMode (LED, OUTPUT);    //Set the LED pin as an output
}

void loop()
{
  for (int i=0; i<256; i++)
  {
    analogWrite(LED, i);
    delay(10);
```

```
  }
  for (int i=255; i>=0; i--)
  {
    analogWrite(LED, i);
    delay(10);
  }
}
```

What does the LED do when you run this code? You should observe the LED fading from off to on, then from on to off. Of course, because this is all in the main loop, this pattern repeats ad infinitum. Be sure to note a few differences in this `for` loop. In the first loop, `i++` is just shorthand code to represent `i=i+1`. Similarly, `i--` is functionally equivalent to `i=i-1`. The first `for` loop fades the LED up, and the second loop fades it down.

PWM control can be used in lots of circumstances to emulate pure analog control, but it cannot always be used when you actually need an analog signal. For instance, PWM is great for driving direct current (DC) motors at variable speeds (you experiment with this in later chapters), but it does not work well for driving speakers unless you supplement it with some external circuitry. Take a moment to examine how PWM actually works. Consider the graphs shown in Figure 2-4.

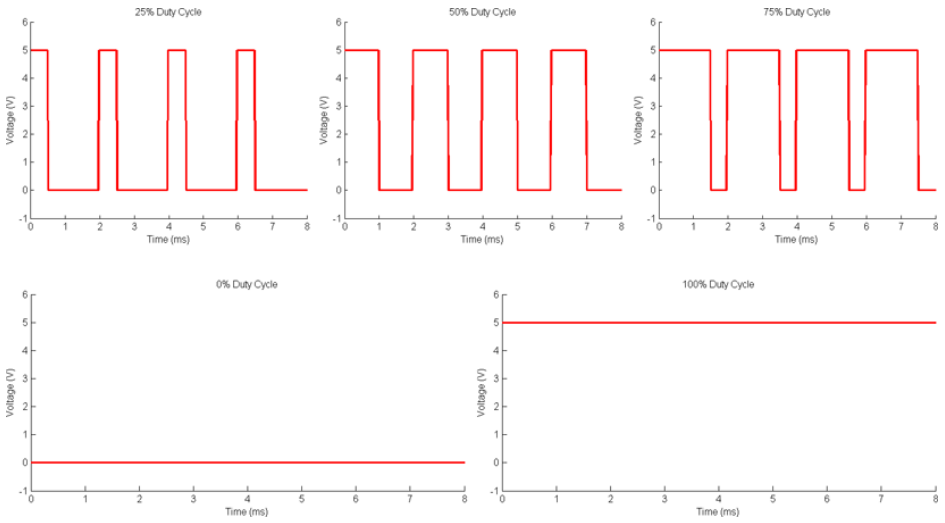


Image created with MATLAB.

Figure 2-4: PWM signals with varying duty cycles

PWM works by modulating the duty cycle of a square wave (a signal that switches on and off). *Duty cycle* refers to the percentage of time that a square wave is high versus low. You are probably most familiar with square waves that have a duty cycle of 50%—they are high half of the time, and low half of the time.

The `analogWrite()` command sets the duty cycle of a square wave depending on the value you pass to it:

- Writing a value of 0 with `analogWrite()` indicates a square wave with a duty cycle of 0 percent (always low).
- Writing a 255 indicates a square wave with a duty cycle of 100 percent (always high).
- Writing a 127 indicates a square wave with a duty cycle of 50 percent (high half of the time, low half of the time).

The graphs in Figure 2-4 show that for a signal with a duty cycle of 25 percent, it is high 25 percent of the time, and low 75 percent of the time. The frequency of this square wave, in the case of the Arduino, is about 490Hz. In other words, the signal varies between high (5V) and low (0V) about 490 times every second.

So, if you are not actually changing the voltage being delivered to an LED, why do you see it get dimmer as you lower the duty cycle? It is really a result of your eyes playing a trick on you! If the LED is switching on and off every 1ms (which is the case with a duty cycle of 50 percent), it appears to be operating at approximately half brightness because it is blinking faster than your eyes can perceive. Therefore, your brain actually averages out the signal and tricks you into believing that the LED is operating at half brightness.

Reading Digital Inputs

Now it is time for the other side of the equation. You've managed to successfully *generate* both digital and analog(ish) outputs. The next step is to *read* digital inputs, such as switches and buttons, so that you can interact with your project in real time. In this section, you learn to read inputs, implement pullup and pulldown resistors, and debounce a button in software.

Reading Digital Inputs with Pulldown Resistors

You should start by modifying the circuit that you first built from Figure 2-1. Following Figure 2-5, you'll add a pushbutton and a pulldown resistor connected to a digital input pin.

TIP Be sure to also connect the power and ground buses of the breadboard to the Arduino. Now that you're using multiple devices on the breadboard, that will come in handy.

Before you write the code to read from the pushbutton, it is important to understand the significance of the pulldown resistor used with this circuit. Nearly

all digital inputs use a pullup or pulldown resistor to set the "default state" of the input pin. Imagine the circuit in Figure 2-5 without the 10kΩ resistor. In this scenario, the pin would obviously read a high value when the button is pressed.

But, what happens when the button is not being pressed? In that scenario, the input pin you would be reading is essentially connected to nothing—the input pin is said to be "floating." And because the pin is not physically connected to 0V or 5V, reading it could cause unexpected results as electrical noise on nearby pins causes its value to fluctuate between high and low. To remedy this, the pulldown resistor is installed as shown in Figure 2-5.

Now, consider what happens when the button is not pressed with the pulldown resistor in the circuit: The input pin will be connected through a 10kΩ resistor to ground. While the resistor will restrict the flow of current, there is still enough current flow to ensure that the input pin will read a low logic value. 10kΩ is a fairly common pulldown resistor value. Larger values are said to be *weak pulldowns* because it easier to overcome them, and smaller resistor values are said to be *strong pulldowns* because it is easier for more current to flow through them to ground. When the button is pressed, the input pin is directly connected to 5V through the button.

Now, the current has two options:

- It can flow through a nearly zero resistance path to the 5V rail.
- It can flow through a high resistance path to the ground rail.

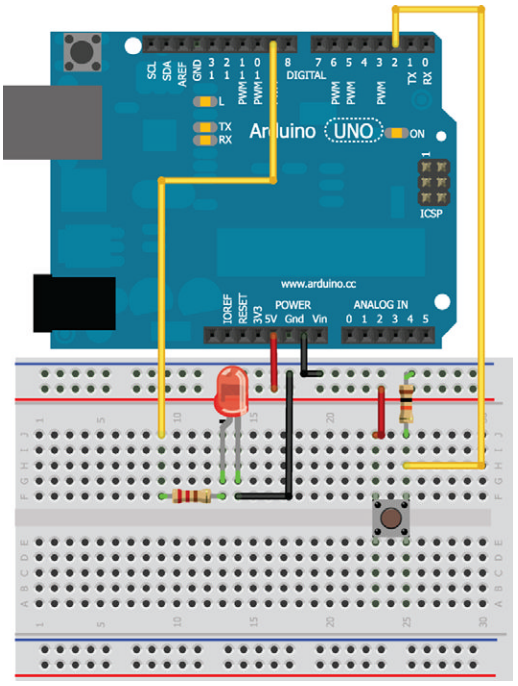


Image created with Fritzing.

Figure 2-5: Wiring an Arduino to a button and an LED

Recall from the previous sidebar on Ohm's law and the power equation that current will always follow the path of the least resistance in a circuit. In this scenario, the majority of the current flows through the button, and a high logic level is generated on the input pin, because that is the path of least resistance.

NOTE This example uses a pulldown resistor, but you could also use a pullup resistor by connecting the resistor to 5V instead of ground and by connecting the other side of the button to ground. In this setup, the input pin reads a high-logic value when the button is unpressed and a low-logic value when the button is being pressed.

Pulldown and pullup resistors are important because they ensure that the button does not create a short circuit between 5V and ground when pressed and that the input pin is never left in a floating state.

Now it is time to write the program for this circuit! In this first example, you just have the LED stay on while the button is held down, and you have it stay off while the button is unpressed (see Listing 2-4).

Listing 2-4: Simple LED Control with a Button—led_button.ino

```
const int LED=9;        //The LED is connected to pin 9
const int BUTTON=2;       //The Button is connected to pin 2

void setup()
{
  pinMode (LED, OUTPUT);      //Set the LED pin as an output
  pinMode (BUTTON, INPUT);    //Set button as input (not required)
}

void loop()
{
  if (digitalRead(BUTTON) == LOW)
  {
    digitalWrite(LED, LOW);
  }
  else
  {
    digitalWrite(LED, HIGH);
  }
}
```

Notice here that the code implements some new concepts, including `digitalRead` and `if/else` statements. A new `const int` statement has been added for the button pin. Further, this code defines the button pin as an input in the `setup` function. This is not explicitly necessary, though, because pins are inputs by default; it is shown for completeness. `digitalRead()` reads the

value of an input. In this case, it is reading the value of the `BUTTON` pin. If the button is being pressed, `digitalRead()` returns a value of `HIGH`, or `1`. If it is not being pressed, it returns `LOW`, or `0`. When placed in the `if()` statement, you're checking the state of the pin and evaluating if it matches the condition you've declared. In this `if()` statement, you're checking to see if the value returned by `digitalRead()` is `LOW`. The `==` is a comparison operator that tests whether the first item (`digitalRead()`) is equal to the second (`LOW`). If this is true (that is, the button is not being pressed), the code inside the brackets executes, and the LED set to `LOW`. If this is not true (the button is being pressed), the `else` statement is executed, and the LED is turned `HIGH`.

That's it! Program your circuit with this code and confirm that it works as expected.

Working with "Bouncy" Buttons

When was the last time you had to hold a button down to keep a light on? Probably never. It makes more sense to be able to click the button once to turn it on and to click the button again to turn it off. This way, you do not have to hold the button down to keep the light on. Unfortunately, this is not quite as easy as you might first guess. You cannot just look for the value of the switch to change from low to high; you need to worry about a phenomenon called *switch bouncing*.

Buttons are mechanical devices that operate as a spring-damper system. In other words, when you push a button down, the signal you read does not just go from low to high, it bounces up and down between those two states for a few milliseconds before it settles. Figure 2-6 illustrates the expected behavior next to the actual behavior you might see when probing the button using an oscilloscope (though this figure was generated using a MATLAB script):

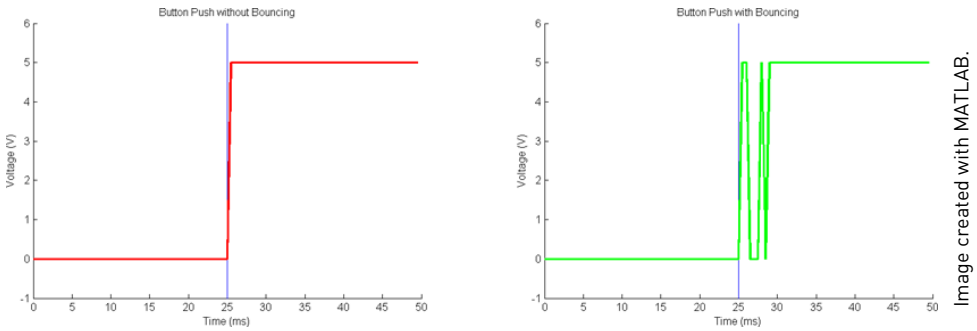


Figure 2-6: Bouncing button effects.

The button is physically pressed at the 25ms mark. You would expect the button state to be immediately read as a high logic level as the graph on the left

shows. However, the button actually bounces up and down before settling, as the graph on the right shows.

If you know that the switch is going to do this, it is relatively straightforward to deal with it in software. Next, you write switch-debouncing software that looks for a button state change, waits for the bouncing to finish, and then reads the switch state again. This program logic can be expressed as follows:

1. Store a previous button state and a current button state (initialized to `LOW`).
2. Read the current button state.
3. If the current button state differs from the previous button state, wait 5ms because the button must have changed state.
4. After 5ms, reread the button state and use that as the current button state.
5. If the previous button state was low, and the current button state is high, toggle the LED state.
6. Set the previous button state to the current button state.
7. Return to step 2.

This is a perfect opportunity to explore using *functions* for the first time. *Functions* are blocks of code that can accept input arguments, execute code based on those arguments, and optionally return a result. Without realizing it, you've already been using predefined functions throughout your programs. For example, `digitalWrite()` is a function that accepts a pin and a state, and writes that state to the given pin. To simplify your program, you can define your own functions to encapsulate actions that you do over and over again.

Within the program flow (listed in the preceding steps) is a series of repeating steps that need to be applied to changing variable values. Because you'll want to repeatedly debounce the switch value, it's useful to define the steps for debouncing as a function that can be called each time. This function will accept the previous button state as an input and outputs the current debounced button state. The following program accomplishes the preceding steps and switches the LED state every time the button is pressed. You'll use the same circuit as the previous example for this. Try loading it onto your Arduino and see how it works (see Listing 2-5).

Listing 2-5: Debounced Button Toggling—debounce.ino

```
const int LED=9;                    //The LED is connected to pin 9
const int BUTTON=2;                 //The Button is connected to pin 2
boolean lastButton = LOW;           //Variable containing the previous
                                    //button state
boolean currentButton = LOW; //Variable containing the current
                                    //button state
```

```
boolean ledOn = false;        //The present state of the LED (on/off)

void setup()
{
  pinMode (LED, OUTPUT);      //Set the LED pin as an output
  pinMode (BUTTON, INPUT);    //Set button as input (not required)
}

/*
* Debouncing Function
* Pass it the previous button state,
* and get back the current debounced button state.
*/
boolean debounce(boolean last)
{
  boolean current = digitalRead(BUTTON);      //Read the button state
  if (last != current)                        //if it's different…
  {
    delay(5);                                 //wait 5ms
    current = digitalRead(BUTTON);            //read it again

  return current;                             //return the current value
}

void loop()
{
  currentButton = debounce(lastButton);           //read deboucned state
  if (lastButton == LOW && currentButton == HIGH) //if it was pressed...
  {
    ledOn = !ledOn;                               //toggle the LED value
  }
  lastButton = currentButton;                     //reset button value

  digitalWrite(LED, ledOn);                       //change the LED state

}
```

Now, break down the code in Listing 2-5. First, constant values are defined for the pins connected to the button and LED. Next, three Boolean *variables* are declared. When the `const` qualifier is not placed before a variable declaration, you are indicating that this variable can change within the program. By defining these values at the top of the program, you are declaring them as *global* variables that can be used and changed by any function within this sketch. The three Boolean variables declared at the top of this sketch are *initialized* as well, meaning that they have been set to an initial value (`LOW`, `LOW`, and `false` respectively). Later in the program, the values of these variables can be changed with an assignment operator (a single equals sign: `=`).

Consider the function definition in the preceding code: `boolean debounce(boolean last)`. This function accepts a Boolean (a data type that has only two states: true/false, high/low, on/off, 1/0) input variable called `last` and returns a Boolean value representing the current debounced pin value. This function compares the current button state with the previous (`last`) button state that was passed to it as an argument. The `!=` represents inequality and is used to compare the present and previous button values in the `if` statement. If they differ, then the button must have been pressed and the `if` statement will execute its contents. The `if` statement waits 5ms before checking the button state again. This 5ms gives sufficient time for the button to stop bouncing. The button is then checked again to ascertain its stable value. As you learned earlier, functions can optionally return values. In the case of this function, the `return current` statement returns the value of the `current` Boolean variable when the function is called. `current` is a *local* variable—it is declared and used only within the debounce function. When the debounce function is called from the main loop, the returned value is written to the *global* `currentButton` variable that was defined at the top of the sketch. Because the function was defined as `debounce`, you can call the function by writing `currentButton = debounce(lastButton)` from within the `setup` or `loop` functions. `currentButton` will be set equal to the value that is returned by the `debounce` function.

After you've called the function and populated the `currentButton` variable, you can easily compare it to the previous button state by using the `if` statement in the code. The `&&` is a logical operator that means "AND". By joining two or more equality statements with an `&&` in an `if` statement, you are indicating that the contents of the `if` statement block should execute only if both of the equalities evaluate to `true`. If the button was previously `LOW`, and is now `HIGH`, you can assume that the button has been pressed, and you can invert the value of the `ledOn` variable. By putting an `!` in front of the `ledOn` variable, you reset the variable to the opposite of whatever it currently is. The loop is finished off by updating the previous button variable and writing the updated LED state.

This code should change the LED state each time the button is pressed. If you try to accomplish the same thing without debouncing the button, you will find the results unpredictable, with the LED sometimes working as expected and sometimes not.

Building a Controllable RGB LED Nightlight

In this chapter, you have learned how to control digital outputs, how to read debounced buttons, and how to use PWM to change LED brightness. Using those skills, you can now hook up an RGB LED and a debounced button to cycle

through some colors for a controllable RGB LED nightlight. It's possible to mix colors with an RGB LED by changing the brightness of each color.

In this scenario, you use a common cathode LED. That means that the LED has four leads. One of them is a cathode pin that is shared among all three diodes, while the other three pins connect to the anodes of each diode color. Wire that LED up to three PWM pins through current-limiting resistors on the Arduino as shown in the wiring diagram in Figure 2-7.

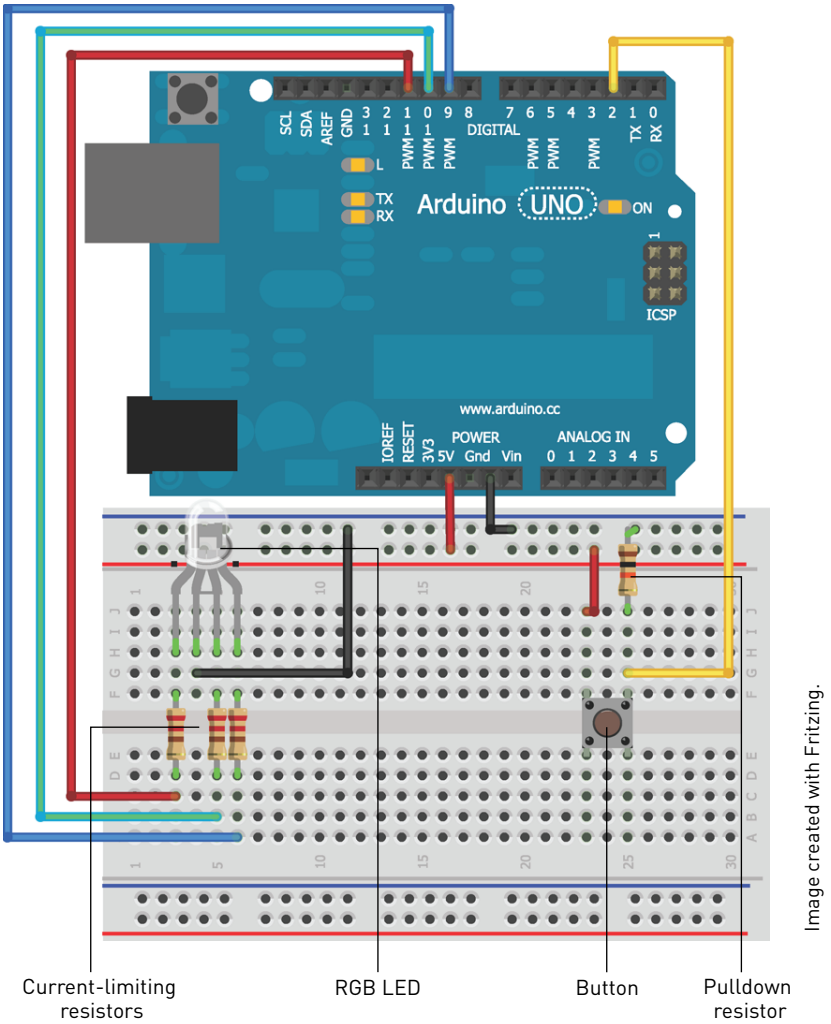


Figure 2-7: Nightlight wiring diagram

You can configure a debounced button to cycle through a selection of colors each time you press it. To do this, it is useful to add an additional function to

set the RGB LED to the next state in the color cycle. In the following program (see Listing 2-6), I have defined seven total color states, plus one off state for the LED. Using the `analogWrite()` function, you can choose your own color-mixing combinations. The only change to the `loop()` from the previous example is that instead of flipping a single LED state, an LED state counter is incremented each time the button is pressed, and it is reset back to zero when you cycle through all the options. Upload this to your Arduino connected to the circuit you just built and enjoy your nightlight. Modify the color states by changing the values of `analogWrite()` to make your own color options.

Listing 2-6: Toggling LED Nightlight—rgb_nightlight.ino

```
const int BLED=9;     //Blue LED on Pin 9
const int GLED=10;    //Green LED on Pin 10
const int RLED=11;    //Red LED on Pin 11
const int BUTTON=2;     //The Button is connected to pin 2

boolean lastButton = LOW;     //Last Button State
boolean currentButton = LOW; //Current Button State
int ledMode = 0;              //Cycle between LED states

void setup()
{
  pinMode (BLED, OUTPUT);   //Set Blue LED as Output
  pinMode (GLED, OUTPUT);   //Set Green LED as Output
  pinMode (RLED, OUTPUT);   //Set Red LED as Output
  pinMode (BUTTON, INPUT); //Set button as input (not required)
}

/*
* Debouncing Function
* Pass it the previous button state,
* and get back the current debounced button state.
*/
boolean debounce(boolean last)
{
  boolean current = digitalRead(BUTTON);      //Read the button state
  if (last != current)                        //if it's different...
  {
    delay(5);                                 //wait 5ms
    current = digitalRead(BUTTON);            //read it again
  }
  return current;                             //return the current value
}

/*
* LED Mode Selection
* Pass a number for the LED state and set it accordingly.
*/
void setMode(int mode)
```

```
{
  //RED
  if (mode == 1)
  {
    digitalWrite(RLED, HIGH);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, LOW);
  }
  //GREEN
  else if (mode == 2)
  {
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, HIGH);
    digitalWrite(BLED, LOW);
  }
  //BLUE
  else if (mode == 3)
  {
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, HIGH);
  }
  //PURPLE (RED+BLUE)
  if (mode == 4)
  {
    analogWrite(RLED, 127);
    analogWrite(GLED, 0);
    analogWrite(BLED, 127);
  }
  //TEAL (BLUE+GREEN)
  else if (mode == 5)
  {
    analogWrite(RLED, 0);
    analogWrite(GLED, 127);
    analogWrite(BLED, 127);
  }
  //ORANGE (GREEN+RED)
  else if (mode == 6)
  {
    analogWrite(RLED, 127);
    analogWrite(GLED, 127);
    analogWrite(BLED, 0);
  }
  //WHITE (GREEN+RED+BLUE)
  else if (mode == 7)
  {
    analogWrite(RLED, 85);
    analogWrite(GLED, 85);
    analogWrite(BLED, 85);
  }
```

```
  //OFF (mode = 0)
  else
  {
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, LOW);
  }
}

void loop()
{
  currentButton = debounce(lastButton);          //read deboucned state
  if (lastButton == LOW && currentButton == HIGH) //if it was pressed...
  {
    ledMode++;                                   //increment the LED value
  }
  lastButton = currentButton;                    //reset button value
  //if you've cycled through the different options,
  //reset the counter to 0
  if (ledMode == 8) ledMode = 0;
  setMode(ledMode);                              //change the LED state
}
```

This might look like a lot of code, but it is nothing more than a conglomeration of code snippets that you have already written throughout this chapter.

How else could you modify this code? You could add additional buttons to independently control one of the three colors. You could also add blink modes, using code from Chapter 1 that blinked the LED. The possibilities are limitless.

Summary

In this chapter you learned about the following:

- How a breadboard works
- How to pick a resistor to current-limit an LED
- How to wire an external LED to your Arduino
- How to use PWM to write "analog" values to LEDs
- How to read a pushbutton
- How to debounce a pushbutton
- How to use `for` loops
- How to utilize pullup and pulldown resistors

CHAPTER 3

Reading Analog Sensors

Parts You'll Need for This Chapter

Arduino Uno
Small breadboard
Jumper wires
10kΩ potentiometer
10kΩ resistor (×2)
220Ω resistor (×3)
USB cable
Photoresistor
TMP36 temperature sensor (or any other 5V analog sensor)
5mm common-cathode RGB LED

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, video, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch3`.

In addition, all code can be found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 03 download and individually named according to the names throughout the chapter.

The world around you is analog. Even though you might hear that the world is "going digital," the majority of observable features in your environment will always be analog in nature. The world can assume an infinite number of potential states, whether you are considering the color of sunlight, the temperature of the ocean, or the concentration of contaminants in the air. This chapter focuses on developing techniques for discretizing these infinite possibilities into palatable digital values that can be analyzed with a microcontroller system like the Arduino.

In this chapter, you will learn about the differences between analog and digital signals and how to convert between the two, as well as a handful of the analog sensors that you can interface with your Arduino. Using skills that you acquired in the preceding chapter, you will add light sensors for automatically adjusting your nightlight. You will also learn how to send analog data from your Arduino to your computer via a USB-to-serial connection, which opens up enormous possibilities for developing more complex systems that can transmit environmental data to your computer.

NOTE You can follow along with a video as I teach you about reading from analog inputs: `www.jeremyblum.com/2011/01/24/arduino-tutorial-4-analog-inputs/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

If you want to learn more about the differences between analog and digital signals, check out this video that explains each in depth: `www.jeremyblum.com/2010/06/20/lets-get-digital-or-analog/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Understanding Analog and Digital Signals

If you want your devices to interface with the world, they will inevitably be interfacing with analog data. Consider the projects you completed in the preceding chapter. You used a switch to control an LED. A switch is a digital input—it has only two possible states: on or off, high or low, 1 or 0, and so on. Digital information (what your computer or the Arduino processes) is a series of binary (or digital) data. Each bit has only has one of two values.

The world around you, however, rarely expresses information in only two ways. Take a look out the window. What do you see? If it's daytime, you probably see sunlight, trees moving in the breeze, and maybe cars passing or people walking around. All these things that you perceive cannot readily be classified as binary data. Sunlight is not on or off; its brightness varies over the course of a day. Similarly, wind does not just have two states; it gusts at different speeds all the time.

Comparing Analog and Digital Signals

The graphs in Figure 3-1 show how analog and digital signals compare to each other. On the left is a square wave that varies between only two values: 0 and 5 volts. Just like with the button that you used in the preceding chapter, this signal is only a "logic high" or "logic low" value. On the right is part of a cosine wave. Although its bounds are still 0 and 5 volts, the signal takes on an infinite number of values between those two voltages.

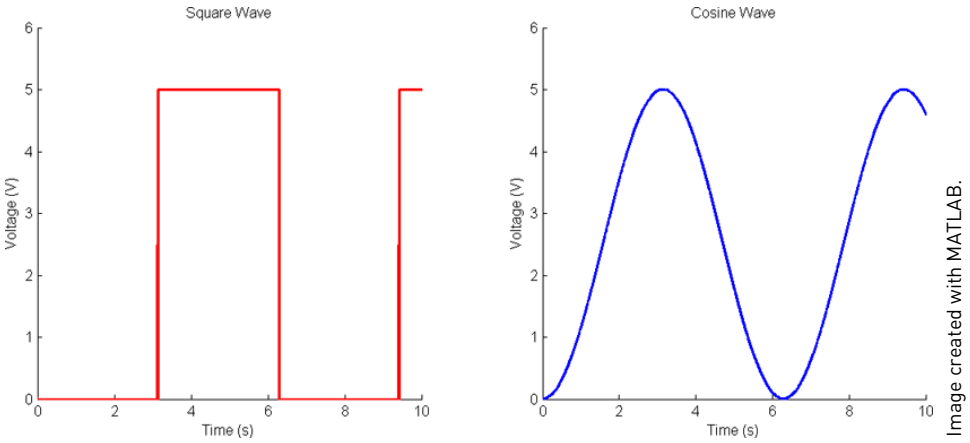


Figure 3-1: Analog and digital signals

Analog signals are those that cannot be discretely classified; they vary within a range, theoretically taking on an infinite number of possible values within that range. Think about sunlight as an example of an analog input you may want to measure. Naturally, there is a reasonable range over which you might measure sunlight. Often measured in lux, or luminous flux per unit area, you can reasonably expect to measure values between 0 lux (for pitch black) and 130,000 lux in direct sunlight. If your measuring device were infinitely accurate, you could measure an infinite number of values between those two. An indoor setting might be 400 lux. If it were slightly brighter, it could be 401 lux, then 401.1 lux, then 401.11 lux, and so on. A computer system could never feasibly measure an infinite number of decimal places for an analog value because memory and computer power must be finite values. If that's the case, how can you interface your Arduino with the "real world?" The answer is analog-to-digital converters (ADC), which can convert analog values into digital representations with a finite amount of precision and speed.

Converting an Analog Signal to a Digital One

Suppose that you want to measure the brightness of your room. Presumably, a good light sensor could produce a varying output voltage that changes with the brightness of the room. When it is pitch black, the device would output 0V, and when it's completely saturated by light, it would output 5V, with values in between corresponding to the varying amount of light. That's all well and good, but how do you go about reading those values with an Arduino to figure out how bright the room is? You can use the Arduino's analog-to-digital converter (ADC) pins to convert analog voltage values into number representations that you can work with.

The accuracy of an ADC is determined by the resolution. In the case of the Arduino Uno, there is a 10-bit ADC for doing your analog conversions. "10-bit" means that the ADC can subdivide (or quantize) an analog signal into 2^{10} different values. If you do the math, you'll find that $2^{10} = 1024$; hence, the Arduino can assign a value from 0 to 1023 for any analog value that you give it. Although it is possible to change the reference voltage, you'll be using the default 5V reference for the analog work that you do in this book. The reference voltage determines the max voltage that you are expecting, and, therefore, the value that will be mapped to 1023. So, with a 5V reference voltage, putting 0V on an ADC pin returns a value of 0, 2.5V returns a value of 512 (half of 1023), and 5V returns a value of 1023. To better understand what's happening here, consider what a 3-bit ADC would do, as shown in Figure 3-2.

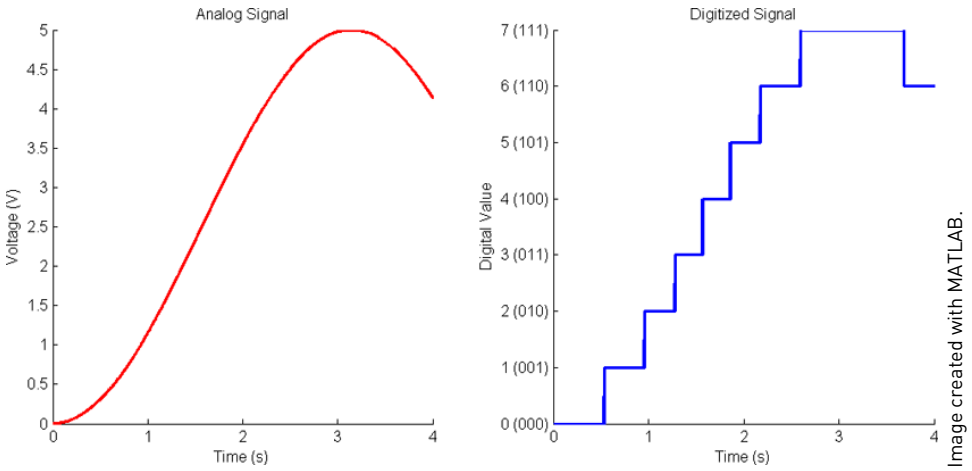


Figure 3-2: 3-bit analog quantization

NOTE If you want to learn more about using your own reference voltage or using a different internal voltage reference, check out the *analogReference()* page on the Arduino website: `www.arduino.cc/en/Reference/AnalogReference`.

A 3-bit ADC has 3 bits of resolution. Because $2^3=8$, there are 8 total logic levels, from 0 to 7. Therefore, any analog value that is passed to a 3-bit ADC will have to be assigned a value from 0 to 7. Looking at Figure 3-2, you can see that voltage levels are converted to discrete digital values that can be used by the microcontroller. The higher the resolution, the more steps that are available for representing each value. In the case of the Arduino Uno, there are 1024 steps rather than the 8 shown here.

Reading Analog Sensors with the Arduino: analogRead()

Now that you understand how to convert analog signals to digital values, you can integrate that knowledge into your programs and circuits. Different Arduinos have different numbers of analog input pins, but you read them all the same way, using the `analogRead()` command. First, you'll experiment with a potentiometer and a packaged analog sensor. Then, you'll learn how voltage dividers work, and how you can use them to make your own analog sensors from devices that vary their resistance in response to some kind of input.

Reading a Potentiometer

The easiest analog sensor to read is a simple potentiometer (a pot, for short). Odds are that you have tons of these around your home in your stereos, speakers, thermostats, cars, and elsewhere. Potentiometers are variable voltage dividers (discussed later in this chapter) that look like knobs. They come in lots of sizes and shapes, but they all have three pins. You connect one of the outer pins to ground, and the other to the 5V. Potentiometers are symmetrical, so it doesn't matter which side you connect the 5V and ground to. You connect the middle pin to analog input 0 on your Arduino. Figure 3-3 shows how to properly hook up your potentiometer to an Arduino.

As you turn the potentiometer, you vary the voltage that you are feeding into analog input 0 between 0V and 5V. If you want, you can confirm this with a multimeter in voltage measurement mode by hooking it up as shown Figure 3-4 and reading the display as you turn the knob. The red (positive) probe should be connected to the middle pin, and the black (negative) probe should be connected to whichever side is connected to ground. Note that your potentiometer and multimeter might look different than shown here.

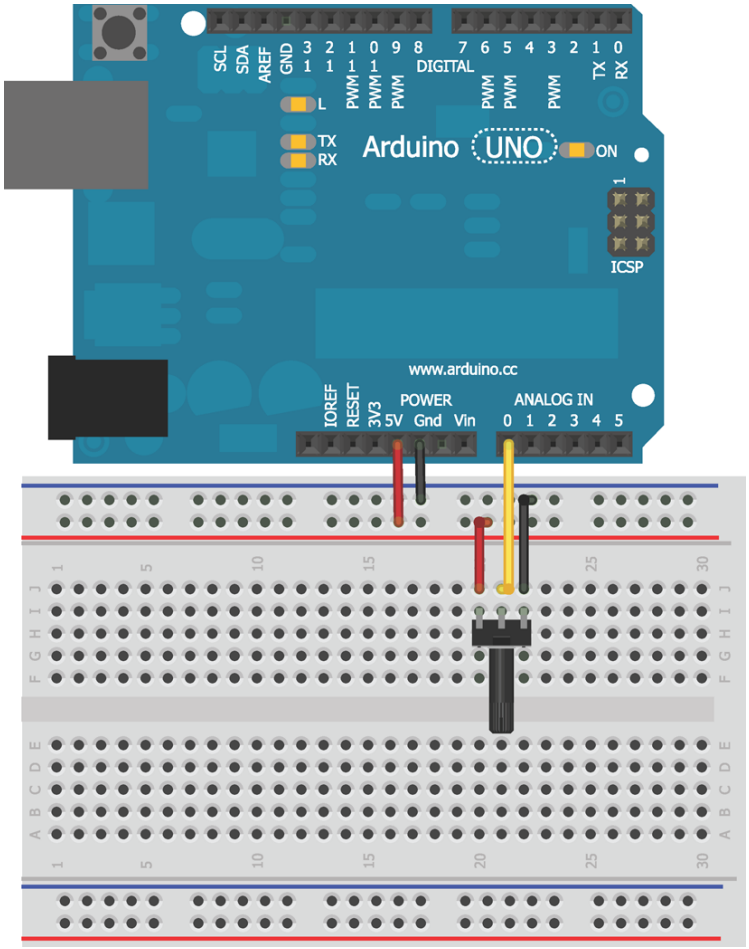


Figure 3-3: Potentiometer circuit

Before you use the potentiometer to control another piece of hardware, use the Arduino's serial communication functionality to print out the potentiometer's ADC value on your computer as it changes. Use the `analogRead()` function to read the value of the analog pin connected to the Arduino and the `Serial.println()` function to print it to the Arduino IDE serial monitor. Start by writing and uploading the program in Listing 3-1 to your Arduino.



Figure 3-4: Multimeter measurement

Listing 3-1: Potentiometer Reading Sketch—pot.ino

```
//Potentiometer Reading Program

const int POT=0;  //Pot on analog pin 0
int val = 0;      //variable to hold the analog reading from the POT

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  val = analogRead(POT);
  Serial.println(val);
  delay(500);
}
```

You'll investigate the functionality of the serial interface more in later chapters. For now, just be aware that the serial interface to the computer must be started in the `setup`. `Serial.begin()` takes one argument that specifies the communication speed, or baud rate. The *baud rate* specifies the number of bits being transferred per second. Faster baud rates enable you to transmit more data in less time, but can also introduce transmission errors in some communication systems. 9600 baud is a common value, and it's what you use throughout this book.

In each iteration through the loop, the `val` variable is set to the present value that the ADC reports from analog pin 0. The `analogRead()` command requires the number of the ADC pin to be passed to it. In this case, it's 0 because that's what you hooked the potentiometer up to. You can also pass `A0`, though the `analogRead()` function knows you must be passing it an analog pin number, so you can pass `0` as shorthand. After the value has been read (a number between `0` and `1023`), `Serial.println()` prints that value over serial to the computer's serial terminal, followed by a "newline" that advances the cursor to the next line. The loop then delays for half a second (so that the numbers don't scroll by faster than you can read them), and the process repeats.

After loading this onto your Arduino, you'll notice that the TX LED on your Arduino is blinking every 500ms (at least it should be). This LED indicates that your Arduino is transmitting data via the USB connection to the serial terminal on your computer. You can use a variety of terminal programs to see what your Arduino is sending, but the Arduino IDE conveniently has one built right in! Click the circled button shown in Figure 3-5 to launch the serial monitor.

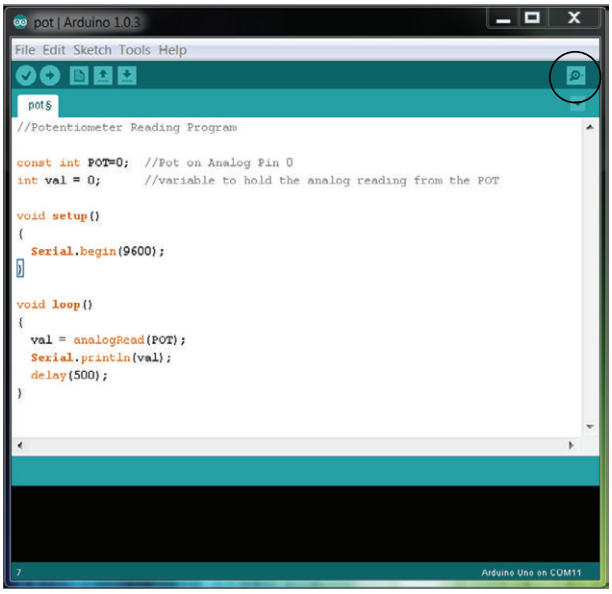


Figure 3-5: Serial monitor button

After launching the serial monitor, you should see a window with numbers streaming by. Turn the dial and you'll see the numbers go up and down to correspond with the position of the potentiometer. If you turn it all the way in one direction, the numbers should approach `0`, and if you turn it all the way in the other direction, the numbers should approach `1023`. It will look like the example shown in Figure 3-6.

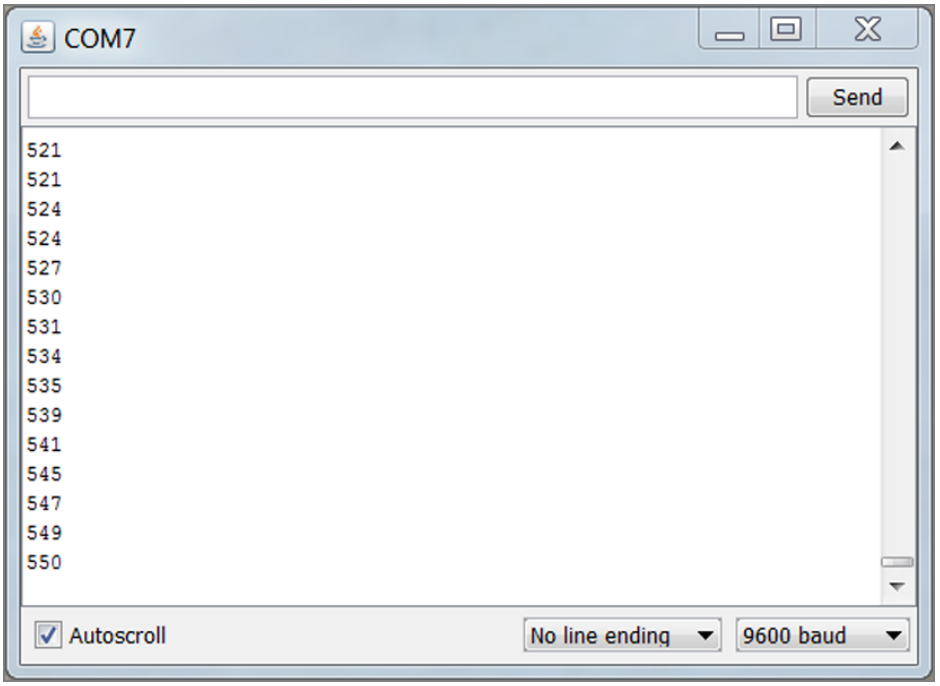


Figure 3-6: Incoming serial data

NOTE If you're getting funky characters, make sure that you have the baud rate set correctly. Because you set it to 9600 in the code, you need to set it to 9600 in this window as well.

You've now managed to successfully turn a dial and make some numbers change; pretty exciting, right? No? Well, this is the just the first step. Next, you learn about other types of analog sensors and how you can use the data from analog sensors to control other pieces of hardware. For now, you use the familiar LED, but in later chapters you use motors and other output devices to visualize your analog inputs.

Using Analog Sensors

Although potentiometers generate an analog voltage value on a pin, they aren't really sensors in the traditional sense. They "sense" your turning of the dial, but that gets boring pretty quickly. The good news is that all kinds of sensors generate analog output values corresponding to "real-world" action. Examples of such include the following:

- Accelerometers that detect tilting (many smartphones and tablets now have these)
- Magnetometers that detect magnetic fields (for making digital compasses)
- Infrared sensors that detect distance to an object
- Temperature sensors that can tell you about the operating environment of your project

Many of these sensors are designed to operate in a manner similar to the potentiometer you just experimented with: You provide them with a power (VCC) and ground (GND) connection, and they output an analog voltage between VCC and GND on the third pin that you hook up to your Arduino's ADC.

For this next experiment, you get to choose what kind of analog sensor you want to use. They all output a value between 0V and 5V when connected to an Arduino, so they will all work the same for your purposes. Here are some examples of sensors that you can use:

- **Sharp Infrared Proximity Sensor**

  `www.exploringarduino.com/parts/IR-Distance-Sensor`

  Connector: `www.exploringarduino.com/parts/JST-Wire`

  The Sharp infrared distance sensors are popular for measuring the distance between your project and other objects. As you move farther from the object you are aiming at, the voltage output decreases. Figure 5 in the datasheet from the part webpage linked above shows the relationship between voltage and measured distance.

- **TMP36 Temperature Sensor**

  `www.exploringarduino.com/parts/TMP36`

  The TMP36 temperature sensor easily correlates temperature readings in Celsius with voltage output levels. Since every 10mV corresponds to 1°C, you can easily create a linear correlation to convert from the voltage you measure back to the absolute temperature of the ambient environment: °C = [(Vout in mV) – 500]/10. The offset of –500 is for dealing with temperatures below 0°C. The graph in Figure 3-7 (extracted from the datasheet) shows this conversion.

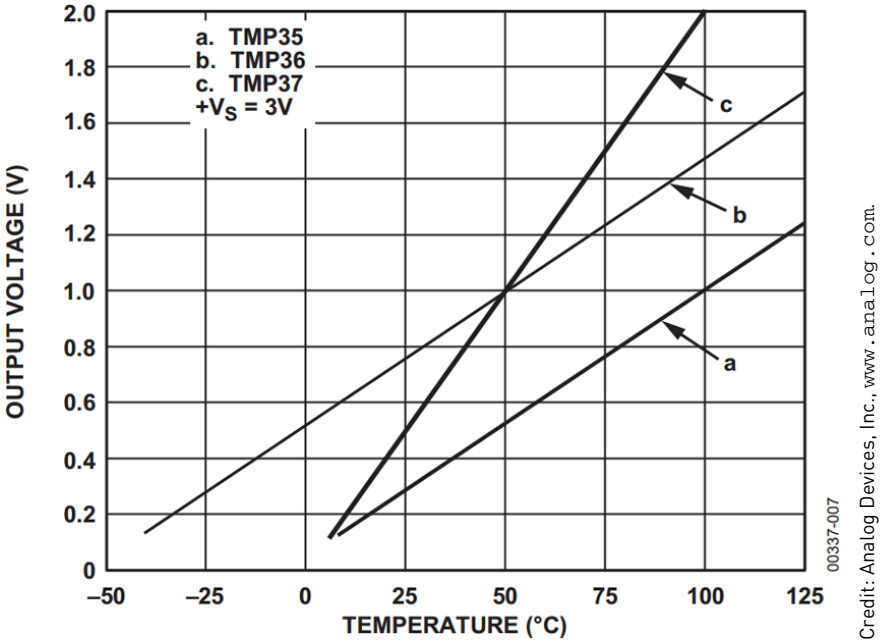


Figure 3-7: Voltage to Temperature Correlation

- **Triple Axis Analog Accelerometer**

  `www.exploringarduino.com/parts/TriAxis-Analog-Accelerometer`

  Triple axis accelerometers are great for detecting orientation. Analog accelerometers output an analog value corresponding to each axis of movement: X, Y, and Z (each on a different pin). Using some clever math (trigonometry and knowledge of gravity), you can use these voltage values to ascertain the position of your project in 3D space! Importantly, many of these sensors are 3.3V, so you will need to use the `analogReference()` command paired with the AREF pin to set a 3.3V voltage reference to enable you to get the full resolution out of the sensor.

- **Dual Axis Analog Gyroscope**

  `www.exploringarduino.com/parts/DualAxis-Analog-Gyroscope`

  Gyroscopes, unlike accelerometers, are not affected by gravity. Their analog output voltages fluctuate in accordance with angular acceleration around an axis. These prove particularly useful for detecting twisting motions. For an example of a gyroscope in action with an Arduino, check out my SudoGlove, a glove I designed that captures hand gestures to control hardware like music synthesizers and RC cars: `www.sudoglove.com`. Like accelerometers, be aware that many gyroscopes are 3.3V parts.

Now that you've chosen a sensor, it's time to put that sensor to use.

Working with Analog Sensors to Sense Temperature

This simple example uses the TMP36 temperature sensor mentioned in the previous section. However, feel free to use any analog sensor you can get your hands on. Experiment with one of the examples listed earlier, or find your own. (It should be 5V compliant if you are using the Arduino Uno.) The following steps are basically the same for any analog sensor you might want to use.

To begin, wire up your RGB LED as you did in the preceding chapter, and wire the temperature sensor up to analog input 0 as shown in the Figure 3-8.

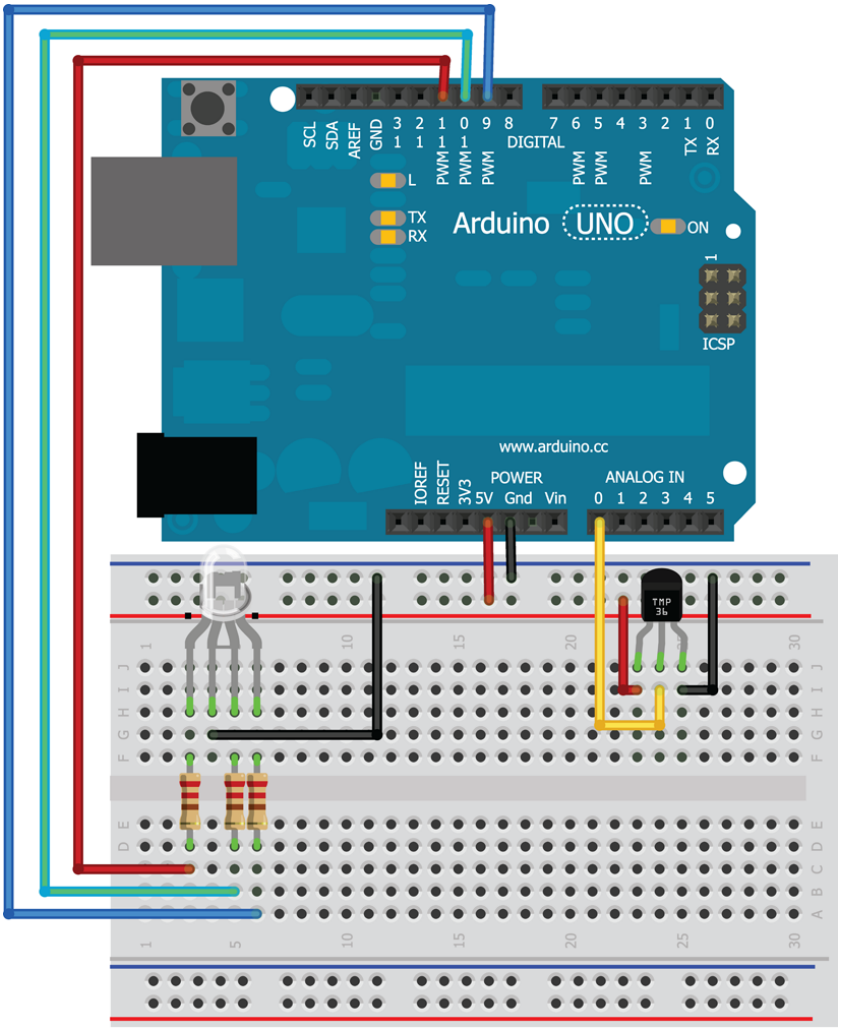


Image created with Fritzing.

Figure 3-8: Temperature sensor circuit

Using this circuit, you'll make a simple temperature alert system. The light will glow green when the temperature is within an acceptable range, will turn red when it gets too hot, and will turn blue when it gets too cold.

First things first, you need to ascertain what values you want to use as your cutoffs. Using the exact same sketch from Listing 3-1, use the serial monitor to figure out what analog values correspond to the temperature cutoffs you care about. My room is about 20°C, which corresponds to an analog reading of about `143`. These numbers might differ for you, so launch the sketch from before, open the serial terminal, and take a look at the readings you are getting. You can confirm the values mathematically using the graph from Figure 3-7. In my case, a value of 143/1023 corresponds to a voltage input of about 700mV. Deriving from the datasheet, the following equation can be used to convert between the temperature (°C) and the voltage (mV):

$$\text{Temperature}(°\text{C}) \times 10 = \text{voltage (mV)} - 500$$

Plugging in the value of 700mV, you can confirm that it equates to a temperature of 20°C. Using this same logic (or by simply observing the serial window and picking a value), you can determine that 22°C is a digital value of `147` and 18°C is a digital value of `139`. Those values will serve as the cutoffs that will change the color of the LED to indicate that it is too hot or too cold. Using the `if` statements, the `digitalWrite` function, and the `analogRead` function that you have now learned about, you can easily read the temperature, determine what range it falls in, and set the LED accordingly.

NOTE Before you copy the code in Listing 3-2, try to write this yourself and see whether you can make it work. After giving it a shot, compare it with the code here. How did you do?

Listing 3-2: Temperature Alert Sketch—tempalert.ino

```
//Temperature Alert!
const int BLED=9;          //Blue LED on pin 9
const int GLED=10;         //Green LED on pin 10
const int RLED=11;         //Red LED on pin 11
const int TEMP=0;          //Temp Sensor is on pin A0

const int LOWER_BOUND=139; //Lower Threshold
const int UPPER_BOUND=147; //Upper Threshold

int val = 0;               //Variable to hold analog reading


void setup()
{
  pinMode (BLED, OUTPUT); //Set Blue LED as Output
  pinMode (GLED, OUTPUT); //Set Green LED as Output
```

```
  pinMode (RLED, OUTPUT); //Set Red LED as Output
}

void loop()
{
  val = analogRead(TEMP);

  if (val < LOWER_BOUND)
  {
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, HIGH);
  }
  else if (val > UPPER_BOUND)
  {
    digitalWrite(RLED, HIGH);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, LOW);
  }
  else
  {
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, HIGH);
    digitalWrite(BLED, LOW);
  }
}
```

This code listing doesn't introduce any new concepts; rather, it combines what you have learned so far to make a system that uses both inputs and outputs to interact with the environment. To try it out, squeeze the temperature sensor with your fingers or exhale on it to heat it up. Blow on it to cool it down.

Using Variable Resistors to Make Your Own Analog Sensors

Thanks to physics, tons of devices change resistance as a result of physical action. For example, some conductive inks change resistance when squished or flexed (force sensors and flex sensors), some semiconductors change resistance when struck by light (photoresistors), and some polymers change resistance when heated or cooled (thermistors). These are just a few examples of components that you can take advantage of to build your own analog sensors. Because these sensors are changing resistance and not voltage, you need to create a voltage divider circuit so that you can measure their resistance change.

Using Resistive Voltage Dividers

A resistive voltage divider uses two resistors to output a voltage that is some fraction of the input voltage. The output voltage is a function directly related to the value of the two resistors. So, if one of the resistors is a variable resistor, you can monitor the change in voltage from the voltage divider that results from the varying resistance. The size of the other resistor can be used to set the sensitivity of the circuit, or you can use a potentiometer to make the sensitivity adjustable.

First, consider a fixed voltage divider and the equations associated with it, as shown in Figure 3-9. A0 in the Figure 3-9 refers to analog pin 0 on the Arduino.

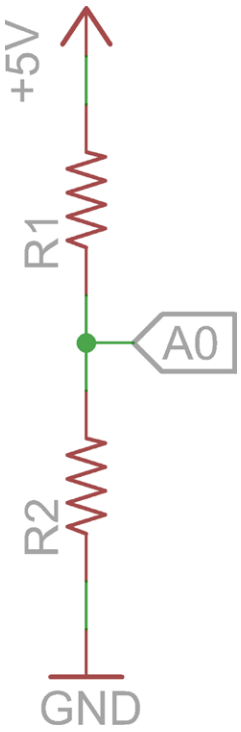


Figure 3-9: Simple voltage divider circuit

The equation for a voltage divider is as follows:

$$Vout = Vin(R2/(R1 + R2))$$

In this case, the voltage input is 5V, and the voltage output is what you'll be feeding into one of the analog pins of the Arduino. In the case where R1 and R2 are matched (both 10kΩ for example), the 5V is divided by 2 to make 2.5V at the analog input. Confirm this by plugging values into the equation:

$$Vout = 5V(10k/(10k + 10k)) = 5V \times .5 = 2.5V$$

Now, suppose one of those resistors is replaced with a variable resistor, such as a photoresistor. Photoresistors (see Figure 3-10) change resistance depending on the amount of light that hits them. In this case, I'll opt to use a 200kΩ photoresistor. When in complete darkness, its resistance is about 200kΩ; when saturated with light, the resistance drops nearly to zero. Whether you choose to replace R1 or R2 and what value you choose to make the fixed resistor will affect the scale and precision of the readings you receive. Try experimenting with different configurations and using the serial monitor to see how your values change. As an example, I will choose to replace R1 with the photoresistor, and I'll make R2 a 10kΩ resistor (see Figure 3-11). You can leave the RGB LED in place for now, though you'll only use one of the colors for this exercise.



Figure 3-10: Photoresistor

Load up your trusty serial printing sketch again (Listing 3-1) and try changing the lighting conditions over the photoresistor. Hold it up to a light and cup it with your hands. Odds are, you aren't going to be hitting the full range from 0 to 1023 because the variable resistor will never have a resistance of zero. Rather, you can probably figure out the maximum and minimum values that you are likely to receive. You can use the data from your photoresistor to make a more intelligent nightlight. The nightlight should get brighter as the room gets darker, and vice versa. Using your serial monitor sketch, pick the values that represent when your room is at full brightness or complete darkness. In my case, I found that a dark room has a value of around `200` and a completely bright room has a value around `900`. These values will vary for you based upon your lighting conditions, the resistor value you are using, and the value of your photoresistor.

Using Analog Inputs to Control Analog Outputs

Recall that you can use the `analogWrite()` command to set the brightness of an LED. However, it is an 8-bit value; that is, it accepts values between 0 and 255 only, whereas the ADC is returning values as high as 1023. Conveniently, the Arduino programming language has two functions that are useful for mapping between two sets of values: the `map()` and `constrain()` functions. The `map()` function looks like this:

```
output = map(value, fromLow, fromHigh, toLow, toHigh)
```

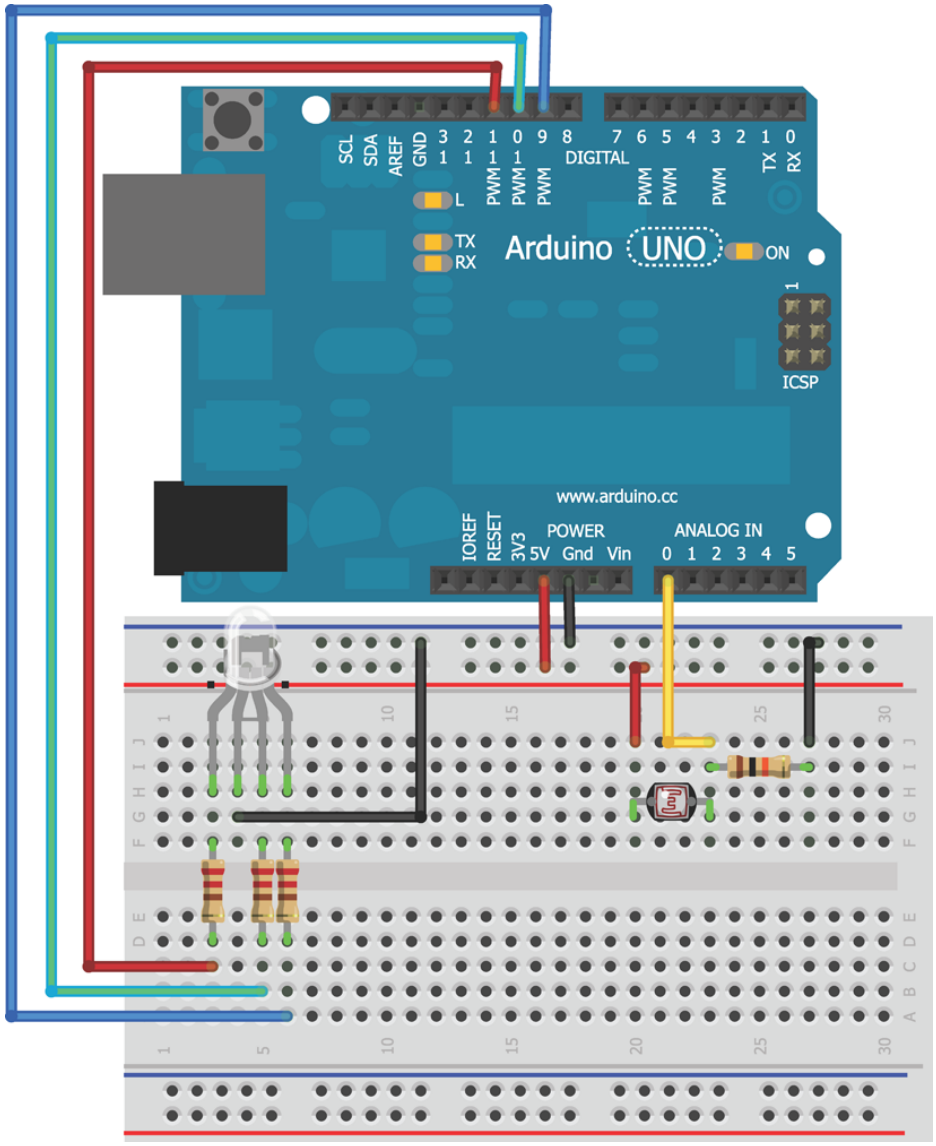


Figure 3-11: Photoresistor circuit

`value` is the information you are starting with. In your case, that's the most recent reading from the analog input. `fromLow` and `fromHigh` are the input boundaries. These are values you found to correspond to the minimum and maximum brightness in your room. In my case, they were `200` and `900`. `toLow` and `toHigh` are the values you want to map them to. Because `analogWrite()` expects value between `0` and `255`, you use those values. However, we want a darker room to map to a brighter LED. Therefore, when the input from the ADC is a low value, you want the output to the LED to be a high value, and vice versa.

Conveniently, the `map` function can handle this automatically; simply swap the high and low values so that the low value is `255` and the high value is 0. The `map()` function creates a linear mapping. For example, if your `fromLow` and `fromHigh` values are `200` and `900`, respectively, and your `toLow` and `toHigh` values are `255` and `0`, respectively, `550` maps to `127` because `550` is halfway between `200` and `900` and `127` is halfway between `255` and `0`. Importantly, however, the `map()` function does not constrain these values. So, if the photoresistor does measure a value below `200`, it is mapped to a value above `255` (because you are inverting the mapping). Obviously, you don't want that because you can't pass a value greater than `255` to the `analogWrite()` function. You can deal with this by using the `constrain()` function. The `constrain()` function looks like this:

```
output = constrain(value, min, max)
```

If you pass the output from the `map` function into the `constrain` function, you can set the `min` to `0` and the `max` to `255`, ensuring that any numbers above or below those values are constrained to either `0` or `255`. Finally, you can then use those values to command your LED! Now, take a look at what that final sketch will look like (see Listing 3-3).

Listing 3-3: Automatic Nightlight Sketch—nightlight.ino

```
//Automatic Nightlight

const int RLED=9;          //Red LED on pin 9 (PWM)
const int LIGHT=0;         //Lght Sensor on analog pin 0
const int MIN_LIGHT=200; //Minimum expected light value
const int MAX_LIGHT=900; //Maximum Expected Light value
int val = 0;               //variable to hold the analog reading

void setup()
{
  pinMode(RLED, OUTPUT); //Set LED pin as output
}

void loop()
{
  val = analogRead(LIGHT);                        //Read the light sensor
  val = map(val, MIN_LIGHT, MAX_LIGHT, 255, 0); //Map the light reading
  val = constrain(val, 0, 255);                   //Constrain light value
  analogWrite(RLED, val);                         //Control the LED
}
```

Note that this code reuses the `val` variable. You can alternatively use a different variable for each function call. In functions such as `map()` where `val` is both the input and the output, the previous value of `val` is used as the input, and its value is reset to the updated value when the function has completed.

Play around with your nightlight. Does it work as expected? Remember, you can adjust the sensitivity by changing the minimum and maximum bounds of the mapping function or changing the fixed resistor value. Use the serial monitor to observe the differences with different settings until you find one that works the best. Can you combine this sketch with the color-selection nightlight that you designed in the preceding chapter? Try adding a button to switch between colors, and use the photoresistor to adjust the brightness of each color.

Summary

In this chapter you learned about the following:

- The differences between analog and digital signals
- How to convert analog signals to digital signals
- How to read an analog signal from a potentiometer
- How to display data using the serial monitor
- How to interface with packaged analog sensors
- How to create your own analog sensors
- How to map and constrain analog readings to drive analog outputs

CHAPTER
4

Using Transistors and Driving Motors

Parts You'll Need for This Chapter:

Arduino Uno
USB cable
9V battery
9V battery clip
5V L4940V5 linear regulator
22uF electrolytic capacitor
.1uF electrolytic capacitor
1uF ceramic capacitor
Blue LEDs (×4)
1kΩ resistors (×4)
PN2222 NPN BJT transistor
Jumper wires
Sharp GP2Y0A41SK0F IR distance sensor with cable
Hot glue or tape
Standard servo motor

DC motor
Breadboard
Potentiometer
SN754410 H-Bridge IC

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, videos, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch4`.

The wiley.com code downloads for this chapter are found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 04 download and individually named according to the names throughout the chapter.

You're now a master of *observing* information from the world around you. But how can you *control* that world? Blinking LEDs and automatically adjusting nightlights are a good start, but you can do so much more. Using assorted types of motors and actuators, and with the help of transistors, you can use your Arduino to generate physical action in the real world. By pairing motors with your Arduino, you can drive robots, build mechanical arms, add an additional degree of freedom to distance sensors, and much more.

In this chapter, you learn how to control inductive loads like direct current (DC) motors, how to use transistors to switch high-current devices, and how to interface with precision actuators (namely, servo motors). At the end of this chapter, you build a sweeping distance sensor capable of identifying the location of nearby obstacles. This sensor is perfect for mounting on a self-driving robotic car, for example. Having completed this chapter, you'll have all the skills you need to build a machine that you can really interact with!

NOTE If you want to learn all about motors and transistors, check out this video: `www.jeremyblum.com/2011/01/31/arduino-tutorial-5-motors-and-transistors/`. You can also find this video on the Wiley website shown at the beginning of this chapter.

WARNING In this chapter, you use a 9V battery so that you can power motors that require more power than what the Arduino can provide. These voltages are still not high enough to pose a danger to you, but if hooked up improperly, these batteries can damage your electronics. As you make your way through the exercises in this chapter, follow the diagrams and instructions carefully. Avoid short circuits (connecting power directly to ground), and while you'll be sharing the ground line between power supplies, don't try to connect two separate voltage sources to each other. For example, don't try to hook both the 9V supply and the Arduino's 5V supply into the same supply row on the breadboard.

Driving DC Motors

DC motors, which you can find in numerous devices around your home, rotate continuously when a DC voltage is applied across them. Such motors are commonly found as the driving motors in radio control (RC) cars, and as the motors that make the discs spin in your DVD player. DC motors are great because they come in a huge array of sizes and are generally very cheap. By adjusting the voltage you apply to them, you can change their rotation speed. By reversing the direction of the voltage applied to them, you can change their direction of rotation as well. This is generally done using an H-bridge, which you learn about later in this chapter.

Brushed DC motors, such as the one you are using for this chapter, employ stationary magnets and a spinning coil. Electricity is transferred to the coil using "brushes," hence the reason they are called *brushed* DC motors. Unlike *brushless* DC motors (such as stepper motors), brushed DC motors are cheap and have easier speed control. However, brushed DC motors do not last as long because the brushes can wear out over time. These motors work through an inductive force. When current passes through the spinning coil, it generates a magnetic field that is either attracted to or repelled by the stationary magnets depending on the polarity. By using the brushes to swap the polarity each half-rotation, you can generate angular momentum. The exact same configuration can be used to create a generator if you manually turn the armature. This will generate a fluctuating magnetic field that will, in turn, generate current. This is how hydroelectric generators work—falling water turns the shaft, and a current is produced. This capability to create current in the opposite direction is why you will use a diode later in this chapter to ensure that the motor cannot send current back into your circuit when it is forcibly turned.

Handling High-Current Inductive Loads

DC motors generally require more current than the Arduino's built-in power supply can provide, and they can create harmful voltage spikes due to their inductive nature. To address this issue, you first learn how to effectively isolate a DC motor from your Arduino, and then how to power it using a secondary supply. A transistor will allow the Arduino to switch the motor on and off safely, as well as to control the speed using the pulse-width modulation (PWM) techniques that you learned about in Chapter 3, "Reading Analog Sensors." Reference the schematic shown in Figure 4-1 as you learn about the various components that go into connecting a DC motor to an Arduino with a secondary power supply. Make sure you understand all of these concepts before you actually start wiring.

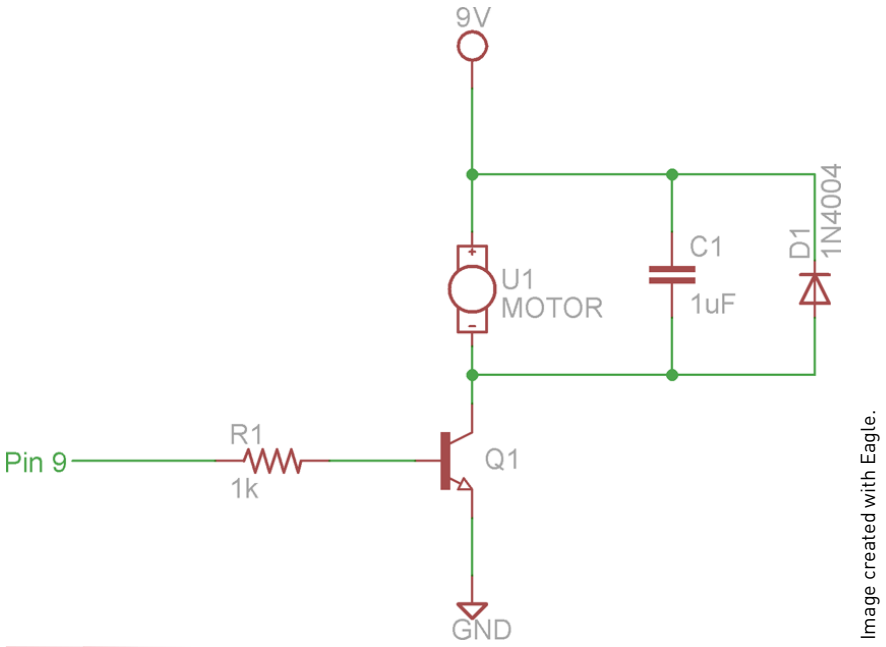


Figure 4-1: DC motor control schematic

Before you hook up your DC motor, it's important to understand what all these components are doing:

- Q1 is an NPN bipolar-junction transistor (BJT) used for switching the separate 9V supply to the motor. There are two types of BJTs, NPN and PNP, which refer to the different semiconductor "doping" techniques used to create the transistor. This book will focus on using NPN BJTs. You can simplistically think of an NPN transistor as a voltage-controlled switch that allows you to inhibit or allow current flow.
- A 1kΩ resistor is used to separate the transistor's base pin from the control pin of the Arduino.
- U1 is the DC motor.
- C1 is for filtering noise caused by the motor.
- D1 is a diode used to protect the power supply from reverse voltage caused by the motor acting like an inductor.

Using Transistors as Switches

Transistors can do an exceptional number of tasks, from making amplifiers to making up the CPU inside your computer and smartphone. You can use a single

transistor to make a simple electrically controlled switch. Every BJT has three pins (see Figure 4-2): the emitter (E), the collector (C), and the base (B).

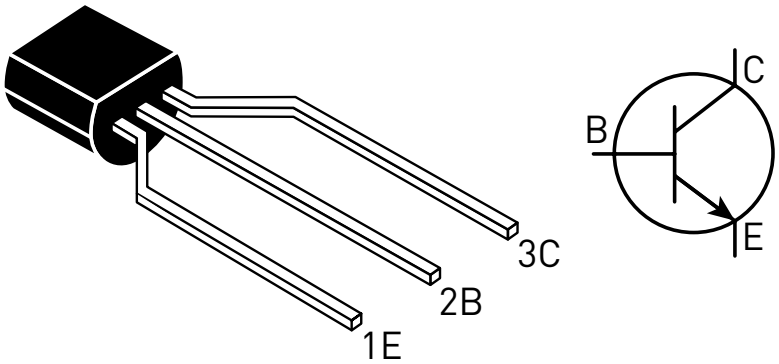


Figure 4-2: An NPN BJT

Current flows in through the collector and out of the emitter. By modulating the base pin, you can control whether current is permitted to flow. When a sufficiently high voltage is applied to the base, current is allowed to flow through the transistor, and the motor spins as a result. The 5V generated by the Arduino I/O pins more than suffices to turn on the transistor. By taking advantage of PWM, you can control the speed of the motor by rapidly turning the transistor on and off. Because the motor can maintain momentum, the duty of the cycle of the PWM signal determines the motor's speed. The transistor is essentially connecting and disconnecting one terminal of the motor from the ground and determining when a complete circuit can be made with the battery.

important

Using Protection Diodes

It is important to consider issues caused by DC motors acting like inductors. (*Inductors* are electrical devices that store energy in their magnetic fields and resist changes in current.) As the DC motor spins, energy is built up and stored in the inductance of the motor coils. If power is instantaneously removed from the motor, the energy is dissipated in the form of an inverted voltage spike, which could prove harmful to the power supply. That's where protection diodes come in. By putting the diode across the motor, you ensure that the current generated by the motor flows through the diode and that the reverse voltage cannot exceed the forward voltage of the diode (because diodes allow current to flow in one direction only). This will also absorb any current generated by you forcibly turning the motor.

two-terminal passive devices that allow current to flow in only one direction.

Using a Secondary Power Source

Note, as well, from the circuit diagram shown in Figure 4-1 that the power supply to the motor is 9V, instead of the usual 5V from the USB connection that you've been using. For the purposes of this experiment, a 9V battery suffices, but you could also use an AC-DC wall adapter. The reason for using a power source separate from the Arduino's built-in 5V supply is twofold:

1. By using a separate supply, you reduce the chances that improper wiring of a higher-power circuit could harm your Arduino.
2. You can take advantage of higher current limits and higher voltages.

Some DC motors can consume more current than the Arduino 5V supply can source. Further, many motors are rated at voltages higher than 5V. Although they might spin at 5V, you can reach their max speed at only 9V or 12V (depending on the motor specifications).

Note that you must connect the ground of both your secondary power supply and the Arduino ground. This connection ensures a common reference point between the voltage levels in the two parts of the circuit.

Wiring the Motor

Now that you understand the intricacies of controlling a brushed DC motor, it's time to get it wired up on your breadboard. Try to wire it by only referencing the previous schematic (shown in Figure 4-1). After you've tried to assemble the circuit using only the schematic, reference the graphical version shown in Figure 4-3 to confirm that you wired it correctly.

It's important to get good at reading electrical schematics without having to look at a graphical layout. Did you wire it correctly? Remember to check for the following as you wire up the circuit:

1. Make sure that you've connected the ground from your 9V battery to the ground from your Arduino. You might want to use the horizontal bus on the breadboard to accomplish this, as shown in Figure 4-3.
2. Make sure that the 9V supply is not connected to the 5V supply. In fact, you don't even need to wire the 5V supply to the breadboard.
3. Make sure that the orientation of your transistor is correct. If you aren't using the same NPN BJT listed in the parts list for this chapter, reference the datasheet to ensure that the emitter, base, and collector are connected to the same pins. If they are not, adjust your wiring.

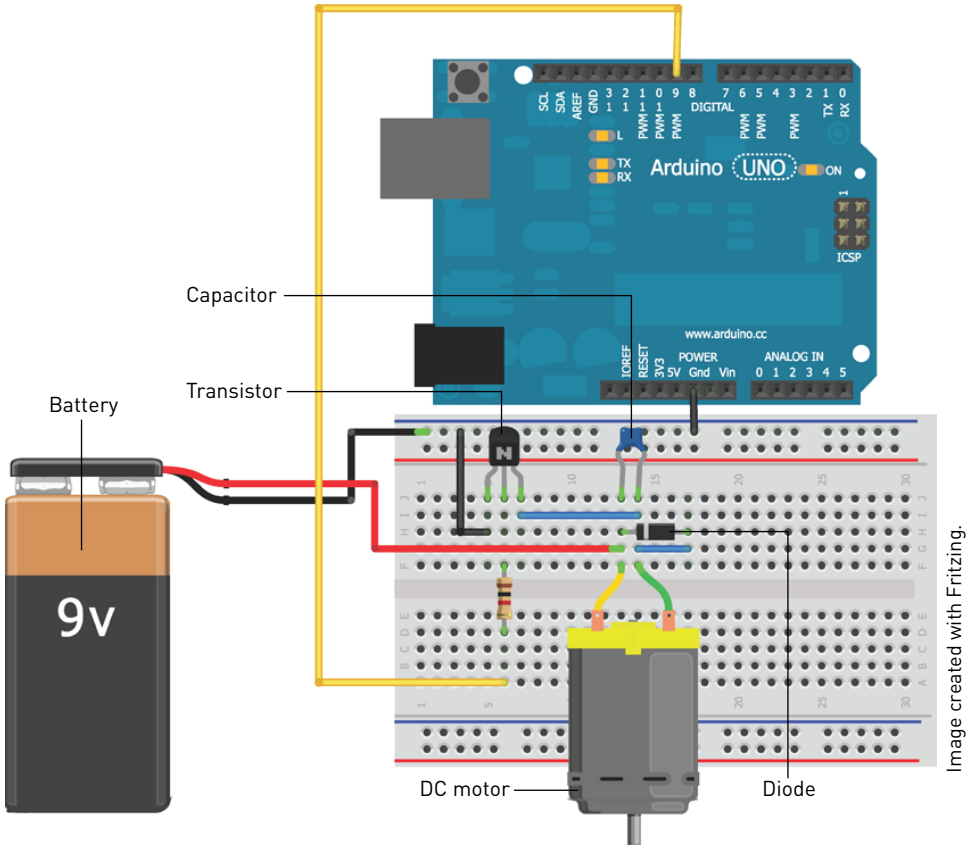


Figure 4-3: DC Motor wiring

4. Make sure that the orientation of the diode is correct. Current flows from the side with no stripe to the side with the stripe. The stripe on the physical device matches the line in the schematic symbol. You use a ceramic capacitor for this exercise, so the polarity doesn't matter.

Next up, it's time to get this motor spinning. You might want to attach a piece of tape or a wheel to the end of the motor so that you can more easily see the speed at which it is spinning. Before you write the program, you can confirm that the circuit is working correctly by providing power to the Arduino over the USB connection, plugging in the 9V battery, and connecting the transistor's base pin (after the resistor) directly to 5V from the Arduino. This simulates a logic high command and should make the motor spin. Connecting that same wire to ground will ensure that it does not spin. If this doesn't work, check your wiring before moving on to the next step: programming.

Controlling Motor Speed with PWM

First up, you can use a program very similar to the one you used to adjust LED brightness of your nightlight in Chapter 3 to adjust the speed of your motor. By sending varying duty-cycle signals to the transistor, the current flow through the motor rapidly starts and stops resulting in a change in velocity. Try out the program in Listing 4-1 to repeatedly ramp the motor speed up and down.

Listing 4-1: Automatic Speed Control—motor.ino

```
//Simple Motor Speed Control Program

const int MOTOR=9;     //Motor on Digital Pin 9

void setup()
{
    pinMode (MOTOR, OUTPUT);
}

void loop()
{
    for (int i=0; i<256; i++)
    {
        analogWrite(MOTOR, i);
        delay(10);
    }
    delay(2000);
    for (int i=255; i>=0; i--)
    {
        analogWrite(MOTOR, i);
        delay(10);
    }
    delay(2000);
}
```

If everything is hooked up correctly, this code should slowly ramp the motor speed up, then back down again in a loop. Using these techniques, you could easily make a simple roving robot.

Next up, you can combine your new knowledge of DC motors with your knowledge of analog sensors. Using a potentiometer, you can manually adjust the motor speed. To begin, add a potentiometer to analog pin 0, as shown in Figure 4-4. Note that you must connect the 5V pin from the Arduino to the power rail on the breadboard if you want to connect the potentiometer to that row on the board.

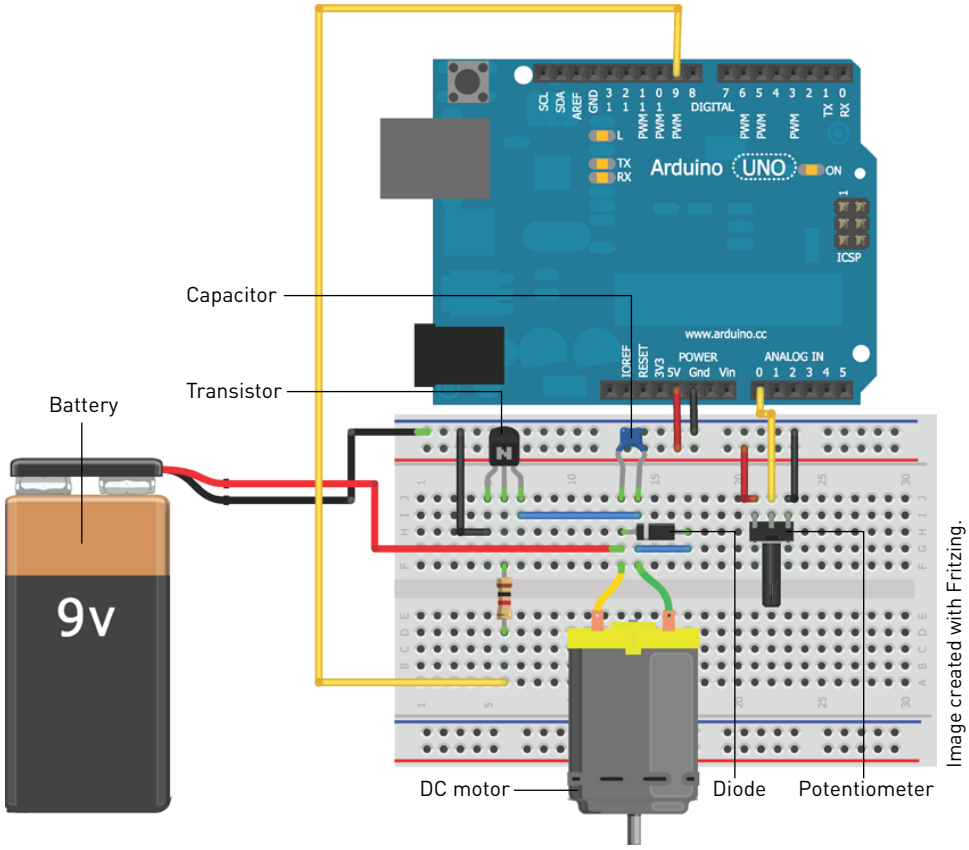


Figure 4-4: Adding a potentiometer

You can now modify the program to control the motor speed based on the present setting of the potentiometer. With the potentiometer at zero, the motor stops; with the potentiometer rotated fully, the motor runs at full speed. Recall that the Arduino is running quite fast; it's actually running through the loop several thousand times every second! Therefore, you can simply check the potentiometer speed each time through the loop and adjust the motor speed after each check. It checks often enough that motor speed adjusts in real time with the potentiometer. The code in Listing 4-2 does the trick. Create a new sketch (or update your previous sketch to match this code) and upload it to your Arduino from the integrated development environment (IDE).

Listing 4-2: Adjustable Speed Control—motor_pot.ino

```
//Motor Speed Control with a Pot

const int MOTOR=9;  //Motor on Digital Pin 9
```

```
const int POT=0;  //POT on Analog Pin 0

int val = 0;

void setup()
{
    pinMode (MOTOR, OUTPUT);
}

void loop()
{
    val = analogRead(POT);
    val = map(val, 0, 1023, 0, 255);
    analogWrite(MOTOR, val);
}
```

A lot of this code should look familiar from your previous experience dealing with analog sensors. Note that the `constrain` function is not required when using a potentiometer, because you can use the entire input range, and the value will never go below `0` or above `1023`. After uploading the code to your Arduino, adjust the pot and observe the speed of the motor changing accordingly.

Using an H-Bridge to Control DC Motor Direction

So, you can change DC motor speed. This is great for making wheels on an Arduino-controlled robot… as long as you only want it to drive forward. Any useful DC motor needs to be able to spin in two directions. To accomplish this, you can use a handy device called an *H-bridge*. The operation of an H-bridge can best be explained with a diagram (see Figure 4-5).

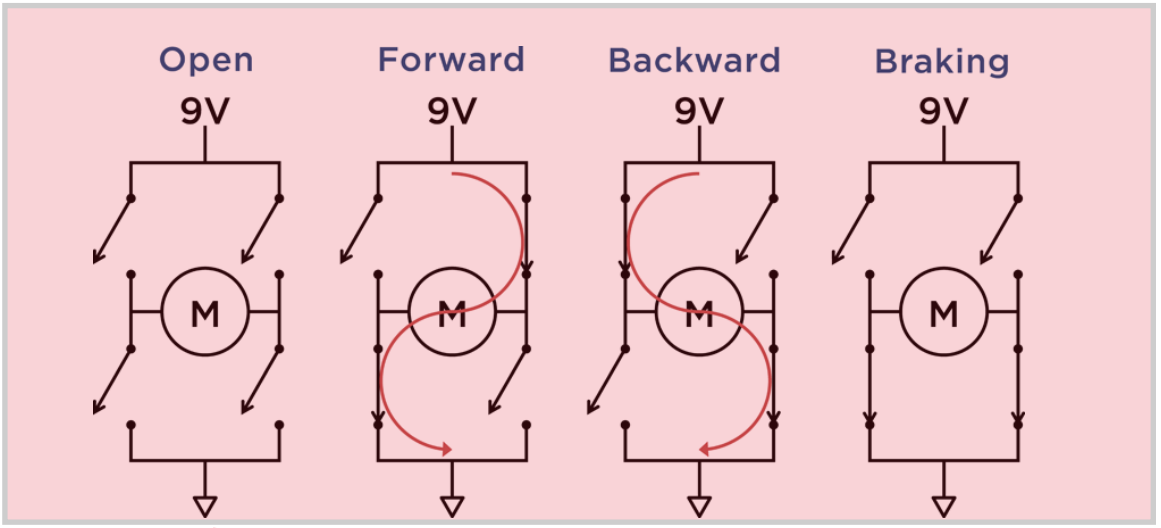


Figure 4-5: H-bridge operation

Can you figure out why it's called an H-bridge? Notice that the motor in combination with the four switches forms an uppercase *H*. Although the diagram shows them as switches, the switching components are actually transistors, similar to the ones you used in the previous exercise. Some additional circuitry, including protection diodes, is also built in to the H-bridge integrated circuit.

The H-bridge has four main states of operation: open, braking, forward, and backward. In the open state, all the switches are open and the motor won't spin. In the forward state, two diagonally opposing switches are engaged, causing current to flow from 9V, through the motor, and down to ground. When the opposing switches are flipped, current then runs through the motor in the opposite direction, causing it to spin in the opposite direction. If the H-bridge is put in the braking state, all residual motion caused by momentum is ceased, and the motor stops.

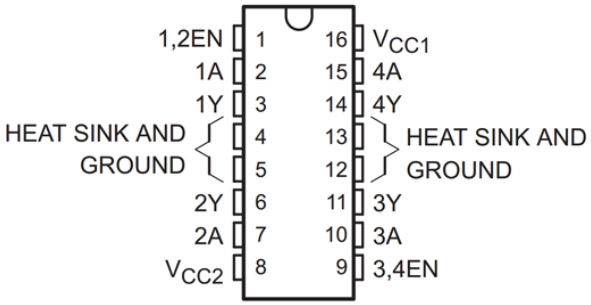
CREATING SHORT CIRCUITS WITH H-BRIDGES

Be aware of one extremely important consideration when using H-bridges. What would happen if both switches on the left or both switches on the right were opened? It would cause a direct short between 9V and ground. If you've ever shorted a 9V battery before, you know that this is not something you want to do. A shorted battery heats up very quickly, and, in rare circumstances, could burst or leak. Furthermore, a short could destroy the H-bridge or other parts of the circuit. An H-bridge is a rare scenario where you could potentially destroy a piece of hardware by programming something wrong. For this experiment, you use SN754410 Quadruple Half-H Driver. This chip has a built-in thermal shutdown that should kick in before a short circuit destroys anything, but it's still a good idea to be cautious.

To ensure that you don't blow anything up, *always* disable the chip before flipping the states of any of the switches. This ensures that a short cannot be created even when you quickly switch between motor directions. You'll use three control pins: one for controlling the top two gates, one for controlling the bottom two gates, and one for enabling the circuit.

Building an H-bridge Circuit

With the preceding considerations in mind, it's time to build the circuit. The H-bridge chip you use is the SN754410 Quadruple Half-H driver. Two Half-H drivers are combined into one Full-H driver, such as the one shown in Figure 4-5. For this exercise, you just use two of the four Half-H drivers to drive one DC motor. If you want to make an RC car, for example, you could use this chip to control two DC motors (one for the left wheels and one for the right wheels). Before you actually get it wired up, take a look at the pin-out and logic table from the part's datasheet (see Figure 4-6).



FUNCTION TABLE
(each driver)

INPUTS†		OUTPUT
A	EN	Y
H	H	H
L	H	L
X	L	Z

H = high-level, L = low-level
X = irrelevant
Z = high-impedance (off)
† In the thermal shutdown mode, the output is in a high-impedance state regardless of the input levels.

Image used with permission courtesy of Texas Instruments.

Figure 4-6: H-bridge pin-out and logic table

Pin numbering on integrated circuits (ICs) always starts at the top-left pin and goes around the part counter-clockwise. Chips will always have some kind of indicator to show which pin is Pin 1, so that you don't plug the IC in upside-down. On through-hole parts (which is what you will use exclusively in this chapter), a half circle on one end of the chip indicates the top of the chip (where Pin 1 is located). Some chips may have a small circle marked next to pin one on the plastic casing in addition to, or instead of the half-circle.

Let's run through the pins and how you'll be using them:

- **GND (Pins 4, 5, 12, & 13):** The four pins in the middle connect to a shared ground between your 9V and 5V supplies.
- **V_{CC2} (Pin 8):** V_{CC2} supplies the motor current, so you connect it to 9V.
- **V_{CC1} (Pin 16):** V_{CC1} powers the chip's logic, so you connect it to 5V.
- **1Y and 2Y (Pins 3 and 6):** These are the outputs from the left driver. The motor wires connect to these pins.
- **1A and 2A (Pins 2 and 7):** The states of the switches on the left are controlled by these pins, so they are connected to I/O pins on the Arduino for toggling.
- **1,2EN (Pin 1):** This pin is used to enable or disable the left driver. It is connected to a PWM pin on the Arduino, so that speed can be controlled dynamically.
- **3Y and 4Y (Pins 11 and 14):** These are the outputs from the right driver. Because you are using the left driver only, you can leave these disconnected.
- **3A and 4A (Pins 10 and 15):** The states of the switches on the right are controlled by these pins, but you are using only the left driver in this example, so you can leave them disconnected.

- **3,4EN (Pin 9):** This pin is used to enable or disable the right driver. Because you will not be using the right driver, you can disable it by connecting this pin directly to GND.

For reference, confirm your wiring with Figure 4-7. Keep the potentiometer wired as it was before.

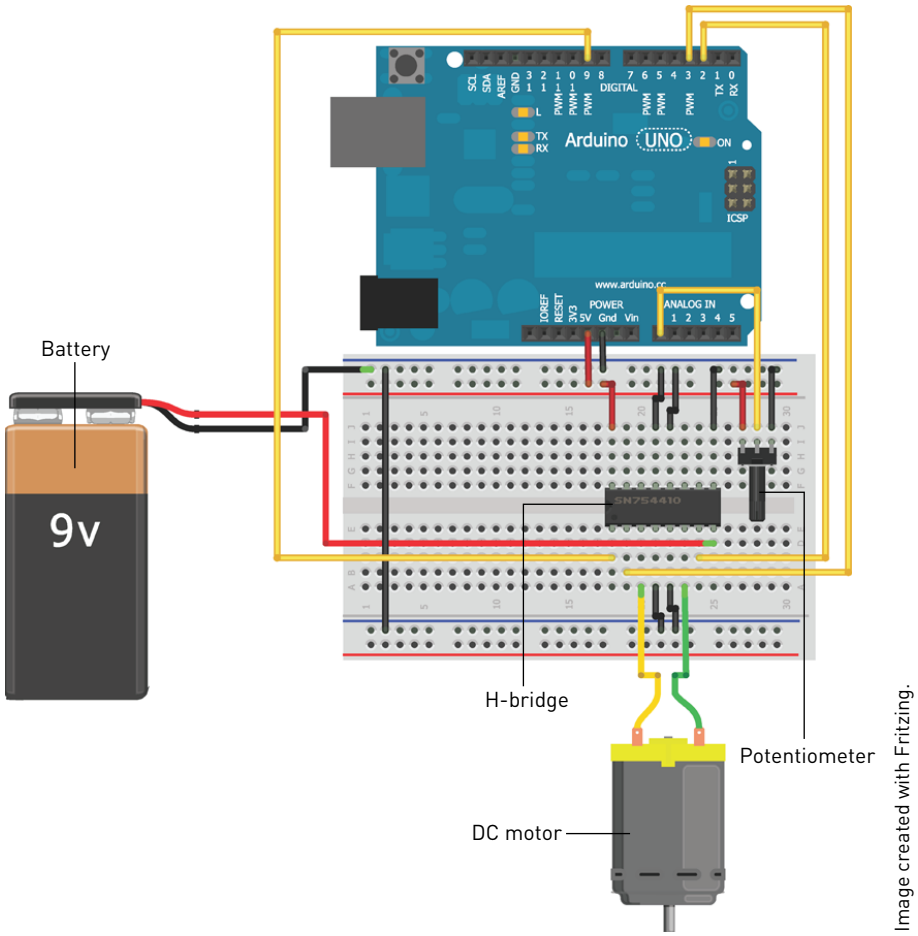


Figure 4-7: H-bridge wiring diagram

You can confirm that the circuit is working before you program it by hooking up the enable pin to 5V, hooking up one of the *A* pins to ground, and the other *A* pin to 5V. You can reverse direction by swapping what the *A* pins are connected to.

WARNING You should disconnect the 9V battery while swapping the *A* pins to ensure that you can't possibly cause an accidental short circuit within the H-bridge.

Operating an H-bridge Circuit

Next up, you write a program to control the motor's direction and speed using the potentiometer and the H-bridge. Setting the potentiometer in a middle range stops the motor, setting the potentiometer in a range above the middle increases the speed forward, and setting the potentiometer in a range below the middle increases the speed backward. This is another perfect opportunity to employ functions in your Arduino program. You can write a function to stop the motor, one to cause it spin forward at a set speed, and one to cause it to spin backward at a set speed. Ensure that you correctly disable the H-bridge at the beginning of the function before changing the motor mode; doing so reduces the probability that you will make a mistake and accidentally short out the H-bridge.

Following the logic diagram from Figure 4-6, you can quickly figure out how you need to control the pins to achieve the desired results:

- To stop current flow through the device, set the enable pin low.
- To set the switches for rotation in one direction, set one high, the other low.
- To set switches for rotation in the opposite direction, swap which is high and which is low.
- To cause the motor to stop immediately, set both switches low.

NOTE Always disable the current flow before changing the state of the switches to ensure that a momentary short cannot be created as the switches flip.

First, you should devise the functions that safely execute the previously described motions. Create a new Arduino sketch and start by writing your new functions:

```
//Motor goes forward at given rate (from 0-255)
void forward (int rate)
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, HIGH);
    digitalWrite(MC2, LOW);
    analogWrite(EN, rate);
}

//Motor goes backward at given rate (from 0-255)
void reverse (int rate)
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, LOW);
    digitalWrite(MC2, HIGH);
    analogWrite(EN, rate);
```

```
}

//Stops motor
void brake ()
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, LOW);
    digitalWrite(MC2, LOW);
    digitalWrite(EN, HIGH);
}
```

Note that at the beginning of each function the *EN* pin is always set low, and then the *MC1* and *MC2* pins (Motor Control pins) are adjusted. When that is done, the current flow can be reenabled. To vary the speed, just use the same technique you did before. By using PWM, you can change the duty with which the *EN* pin is toggled, thus controlling the speed. The *rate* variable must be between 0 and 255. The main loop takes care of making the right *rate* from the input potentiometer data.

Next, consider the main program loop:

```
void loop()
{
    val = analogRead(POT);

    //go forward
    if (val > 562)
    {
        velocity = map(val, 563, 1023, 0, 255);
        forward(velocity);
    }

    //go backward
    else if (val < 462)
    {
        velocity = map(val, 461, 0, 0, 255);
        reverse(velocity);
    }

    //brake
    else
    {
        brake();
    }
}
```

In the main loop, the potentiometer value is read, and the appropriate function can be called based on the potentiometer value. Recall that analog inputs

are converted to digital values between 0 and 1023. Refer to Figure 4-8 to better understand the control scheme and compare that with the preceding loop code.



Figure 4-8: Motor control plan.

When the potentiometer is within the 100 units surrounding the midpoint, the `brake` function is called. As the potentiometer value increases from `562` to `1023`, the speed forward increases. Similarly, the speed increases in the reverse direction between potentiometer values of `462` and `0`. The `map` function should look familiar to you from the previous chapter. Here, when determining the reverse speed, note the order of the variables: `461` is mapped to `0`, and `0` is mapped to `255`; the `map` function can invert the mapping when the variables are passed in descending order. Putting the loop together with the functions, and the `setup`, you get a completed program that looks like the one shown in Listing 4-3. Ensure that your program matches the one here and load it onto your Arduino.

Listing 4-3: H-Bridge Potentiometer Motor Control—hbridge.ino

```
//Hbridge Motor Control
const int EN=9;   //Half Bridge 1 Enable
const int MC1=3;  //Motor Control 1
const int MC2=2;  //Motor Control 2
const int POT=0;  //POT on Analog Pin 0

int val = 0;      //for storing the reading from the POT
int velocity = 0; //For storing the desired velocity (from 0-255)

void setup()
{
    pinMode(EN, OUTPUT);
    pinMode(MC1, OUTPUT);
    pinMode(MC2, OUTPUT);
    brake(); //Initialize with motor stopped
}

void loop()
{
    val = analogRead(POT);

    //go forward
    if (val > 562)
    {
```

```
        velocity = map(val, 563, 1023, 0, 255);
        forward(velocity);
    }

    //go backward
    else if (val < 462)
    {
        velocity = map(val, 461, 0, 0, 255);
        reverse(velocity);
    }

    //brake
    else
    {
        brake();
    }
}

//Motor goes forward at given rate (from 0-255)
void forward (int rate)
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, HIGH);
    digitalWrite(MC2, LOW);
    analogWrite(EN, rate);
}

//Motor goes backward at given rate (from 0-255)
void reverse (int rate)
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, LOW);
    digitalWrite(MC2, HIGH);
    analogWrite(EN, rate);
}

//Stops motor
void brake ()
{
    digitalWrite(EN, LOW);
    digitalWrite(MC1, LOW);
    digitalWrite(MC2, LOW);
    digitalWrite(EN, HIGH);
}
```

Does everything work as expected? If not, make sure that you wired up your circuit correctly. As an additional challenge, grab a second DC motor and hook it up to the other half of the H-bridge chip. You should be able to drive two motors simultaneously with minimal effort.

Driving Servo Motors

DC motors serve as excellent drive motors, but they are not as ideal for precision work because no feedback occurs. In other words, without using an external encoder of some kind, you will never know the absolute position of a DC motor. Servo motors, or servos, in contrast, are unique in that you command them to rotate to a particular angular position and they stay there until you tell them to move to a new position. This is important for when you need to move your system to a known position. Examples include actuating door locks, moving armatures to specific rotations, and precisely controlling the opening of an aperture. In this section, you learn about servo motors and how to control them from your Arduino.

Understanding the Difference Between Continuous Rotation and Standard Servos

You can buy both standard and continuous rotation servos. Unmodified servos always have a fixed range (usually from 0 to 180 degrees) because there is a potentiometer in line with the drive shaft, which is used for reporting the present position. Servo control is achieved by sending a pulse of a particular length. The length of the pulse, in the case of a standard rotation servo, determines the absolute position that the servo will rotate to. If you remove the potentiometer, however, the servo is free to rotate continuously, and the pulse length sets the speed of the motor instead.

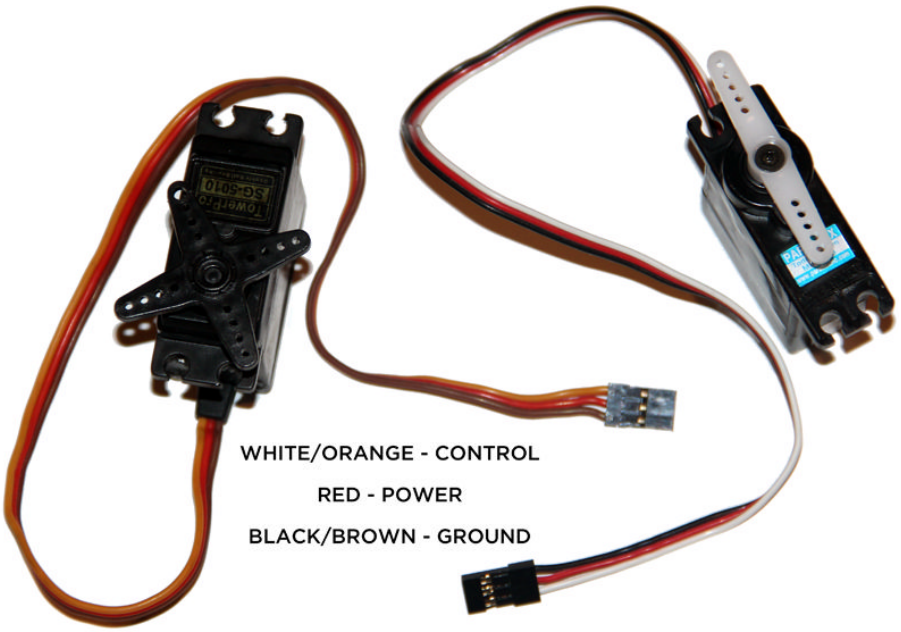
In this book, you use standard servos that rotate to an absolute position. You can experiment with continuous rotation servos either by opening a standard servo and carefully removing the potentiometer, or by buying premodified servos configured for continuous rotation.

Understanding Servo Control

Unlike their DC motor counterparts, servo motors have three pins: power (usually red), ground (usually brown or black), and signal (usually white or orange). These wires are color-coded, typically in the same order, and generally look like the ones shown in Figure 4-9. Some manufactures may use non-standard ordering, so always be sure to check the datasheet to ensure you are wiring the servo correctly.

The coloring might vary slightly between servos, but the color schemes listed previously are the most common. (Check the servo's documentation if you're unsure.) Like DC motors, servos can draw quite a bit of a current (usually

more than the Arduino can supply). Although you can sometimes run one or two servos directly from the Arduino's 5V supply, you learn here how to use a separate power supply for the servos so that you have the option to add more if you need to.



Used with permission from Parallax Inc. Copyright © 2013 Parallax Inc. All rights reserved.

Figure 4-9: Servo motors

Servos have a dedicated control pin, unlike DC motors, that instructs them what position to turn to. The power and ground lines of a servo should always be connected to a steady power source.

Servos are controlled using adjustable pulse widths on the signal line. For a standard servo, sending a 1ms 5V pulse turns the motor to 0 degrees, and sending a 2ms 5V pulse turns the motor to 180 degrees, with pulse lengths in the middle scaling linearly. A 1.5ms pulse, for example, turns the motor to 90 degrees. Once a pulse has been sent, the servo turns to that position and stays there until another pulse instruction is received. However, if you want a servo to "hold" its position (resist being pushed on and try to maintain the exact position), you just resend the command once every 20ms. The Arduino servo commands that you will later employ take care of this for you. To better understand how servo control works, study the timing diagram shown in Figure 4-10.

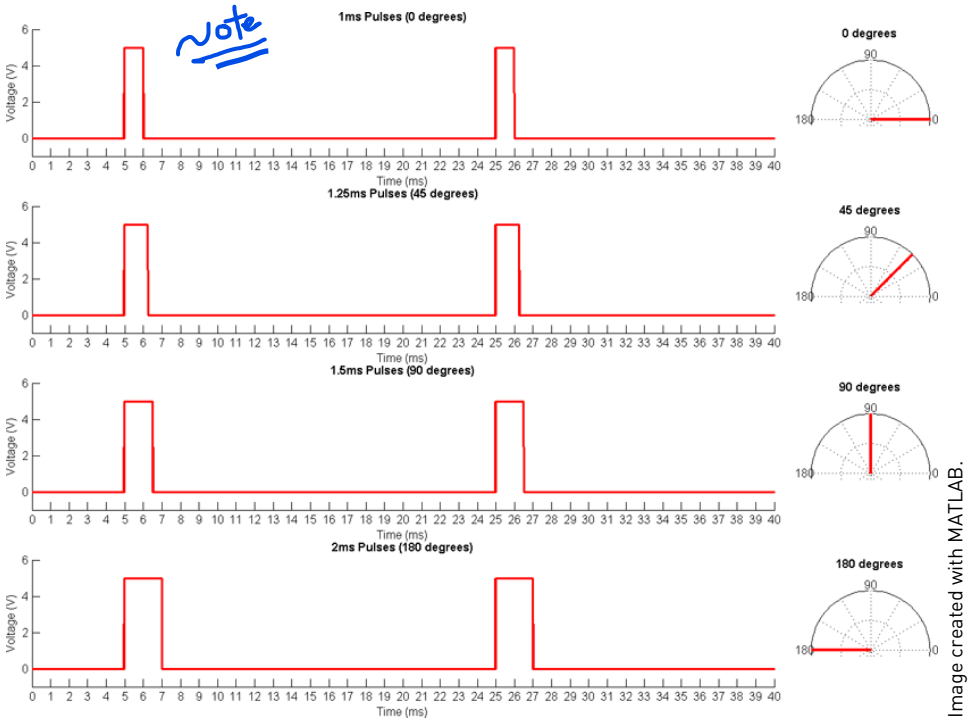


Figure 4-10: Servo motor timing diagram

Note that in each of the examples in Figure 4-10 the pulse is sent every 20ms. As the pulse length increases from 1ms to 2ms, the angle of rotation of the motor (shown to the right of the pulse graph) increases from 0 to 180 degrees.

As mentioned before, servos can draw more current than your Arduino may be able to provide. However, most servos are designed to run at 5V, not 9V or 12V like a DC motor. Even though the voltage is the same as that of an Arduino, you want to use a separate power source that can supply more current.

To do this, you learn here how to use a 9V battery and a linear regulator to generate a 5V supply from your 9V battery. A linear regulator is an extremely simple device that generally has three pins: input voltage, output voltage, and ground. The ground pin is connected to both the ground of the input supply and to the ground of the output. In the case of linear-voltage regulators, the input voltage always must be higher than the output voltage, and the output voltage is set at a fixed value depending on the regulator you use.

The voltage drop between the input and the output is burned off as heat, and the regulator takes care of ensuring that the output always remains the same, even as the voltage of the input drops (in the case of a battery discharging over time). For these experiments, you use an L4940V5 5V voltage regulator. It's capable of supplying up to 1.5 amps at 5V. Figure 4-11 shows a schematic of how to hook up the regulator.

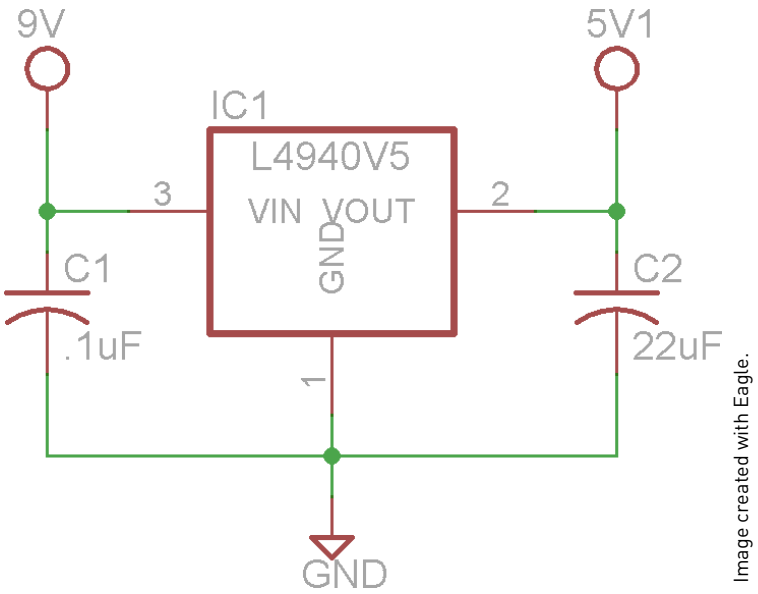


Figure 4-11: 5V Linear regulator schematic

Note the capacitors on each side of the regulator. These are called *decoupling capacitors;* they are used to smooth out the voltage signal from each supply voltage by charging and discharging to oppose ripples in the voltage. Most linear regulator datasheets include a suggested circuit that includes ideal values and types for these capacitors based on your use case scenario. Also keep in mind that the 5V rail created by this regulator should be kept separate from the 5V power rail of the Arduino. Their grounds, however, should be tied together.

Using all this information, it's time to wire up a servo. Referencing Figure 4-12, wire the servo, the 5V regulator, and the potentiometer. Leave the potentiometer connected to analog pin 0, connect the servo control pin to pin 9, and ensure that the 5V regulator supplies the servo's power.

While wiring, keep in mind a few important things. First, ensure that you have the orientation of the regulator correct. With the metal tab on the side farthest from you, connect the battery to the leftmost pin, the ground to the center pin, and the servo's power line to the right pin. Second, if using polarized electrolytic capacitors (as in Figure 4-12), make sure to put them in the correct direction. The stripe indicates the negative terminal and should be connected to the common ground. Make sure that the pins don't touch; otherwise, it could cause a short. After you're all wired up, move on to the next section to learn how to program the servo controller.

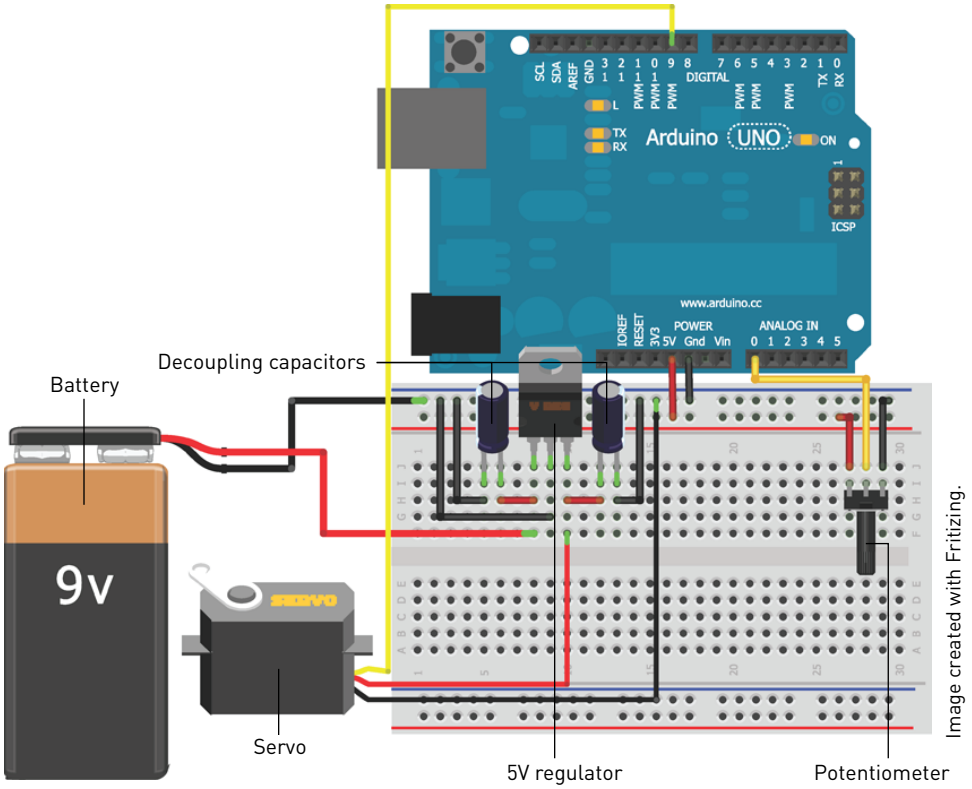


Figure 4-12: Servo experiment wiring diagram

UNDERSTANDING LINEAR REGULATORS AND THE LIMITS OF ARDUINO POWER SUPPLIES

Why is it necessary to use an external power supply when certain items require more current? There are few reasons. The I/O pins cannot supply more than 40 milliamps (mA) each. Because a DC or servo motor can consume hundreds of milliamps, the I/O pins are not capable of driving them directly. Even if they were, you wouldn't want to because of the damage that can be caused by inductive voltage spikes.

It makes sense that you need to use an external supply with a DC motor because you need the higher voltage, but why does a servo need an external supply if it is at the same voltage as the Arduino? The Arduino generates the 5V used for the logic either directly from the USB or by using a built-in linear regulator with the DC barrel jack as the supply voltage. When you use USB, a maximum of 500mA is available to the Arduino and all its peripherals, because that is what the USB specification allows. When you use an external supply of sufficient current, the built-in regulator can supply up to 1 amp to the components on the 5V rail.

Servos have a tendency to consume current in bursts as they turn. They generally consume little current while stationary, and they consume several hundred milliamps for a few milliseconds when they are actuated. These current spikes can ripple on the 5V line, and can even be seen in other components, like LEDs. By keeping the supply for the servo on a separate rail, you ensure that this does not happen.

Insufficient current for a servo might also cause it to behave erratically. When you finish the final project for this chapter, try hooking the servo supply pin up to the built-in 5V rail. (Don't worry; this won't damage anything.) When the servo is powered over USB, you may see the servo doing all kinds of unexpected motions due to an insufficient current supply. Naturally, the degree of this behavior depends on the specification of your particular servo.

Controlling a Servo

The Arduino IDE includes a built-in library that makes controlling servos a breeze. A software library is a collection of code that is useful, but not always needed in sketches. The Arduino IDE contains a number of libraries for common tasks. The servo library abstracts the timing routines you would need to write out on your own for pulsing the servo pin. All you have to do is attach a servo "object" to a particular pin and give it an angle to rotate to. The library takes care of the rest, even setting the pin as an output. The simplest way to test out the functionality of your servo is to map the potentiometer directly to servo positions. Turning the potentiometer to `0` moves the servo to 0 degrees, and moving it to `1023` moves the servo to 180 degrees. Create a new sketch with the code from Listing 4-4 and load it onto your Arduino to see this functionality in action.

Listing 4-4: Servo Potentiometer Control—servo.ino

```
//Servo Potentiometer Control

#include <Servo.h>

const int SERVO=9; //Servo on Pin 9
const int POT=0;   //POT on Analog Pin 0

Servo myServo;
int val = 0;      //for storing the reading from the POT

void setup()
{
    myServo.attach(SERVO);
```

```
}

void loop()
{
    val = analogRead(POT);            //Read Pot
    val = map(val, 0, 1023, 0, 179); //scale it to servo range
    myServo.write(val);               //sets the servo
    delay(15);                        //waits for the servo
}
```

The `include` statement at the top of the program adds the functionality of the servo library to your sketch. `Servo myServo` makes a servo object called `myServo`. In your code, whenever you want to tell the servo what to do, you'll refer to `myServo`. In `setup()`, attaching the servo initializes everything necessary to control the servo. You can add multiple servos by calling the objects different things and attaching a different pin to each one. In `loop()`, the pot is read, scaled to an appropriate value for the servo control, and is then "written" to the servo by pulsing the appropriate pin. The 15ms delay ensures that the servo reaches its destination before you try to send it another command.

Building a Sweeping Distance Sensor

To wrap up this chapter, you apply your knowledge from the past few chapters to build a light-up sweeping distance sensor. The system consists of an infrared (IR) distance sensor mounted on a servo motor and four LEDs. As the servo motor cycles, it pans the distance sensor around the room, allowing you to roughly determine where objects are close and where they are far. The four LEDs correspond to four quadrants of the sweep and change brightness depending on how close an object is in that quadrant.

Because IR light is a part of the electromagnetic spectrum that humans cannot see, a system like this can be implemented to create "night vision." The IR distance sensor works by shining an IR LED and using some fairly complex circuitry to calculate the angle at which that IR light returns to a photo sensor mounted next to the IR LED. Using analog voltages created by the IR photo sensor readings, the distance is calculated and converted to an analog voltage signal that you can read into the microcontroller. Even if the room is dark and you cannot see how close an object is, this sensor can because it is using a wavelength of light that the human eye cannot detect.

Different models of IR rangefinders may have different interfaces. If you're using a rangefinder that is different than the one used in this example, check the datasheet to make sure it sends out a variable voltage as an output.

NOTE You can watch a demo video of the sweeping distance sensor online: `www.exploringarduino.com/content/ch4`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Start by hot-gluing your distance sensor to the top of a servo motor, as shown in Figure 4-13. I like to use hot glue because it holds well and is fairly easy to remove if you need to. However, you could also use super glue, putty, or tape to get the job done.

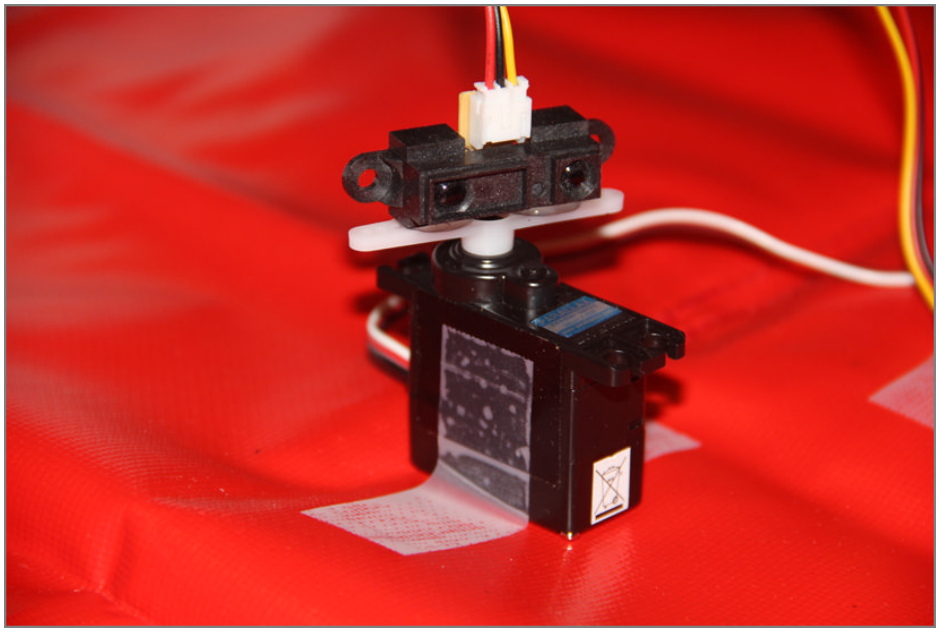


Figure 4-13: IR distance sensor mounted to the servo

Next, hook your servo up to your Arduino, using the 5V regulator to power it, just as you did before. The IR distance sensor replaces the potentiometer and plugs into analog pin 0. Four LEDs plug into pins 3, 5, 6, and 11 through 1kΩ resistors. The Arduino Uno has six total PWM pins, but pins 9 and 10 cannot create PWM signals (using `analogWrite`) when you are using the servo library. This is because the servo library uses the same hardware timer as the one used to control PWM on those two pins. Hence, the other four PWM pins were chosen. (If you want to do this project with more LEDs, either use the Arduino Mega or implement a software PWM solution, something this book does not cover.) Follow the wiring diagram in Figure 4-14 to confirm that you have everything wired up correctly. I chose to use blue LEDs, but you can use any color you want. After you have it all wired up, consider taping it down, as shown in Figure 4-13.

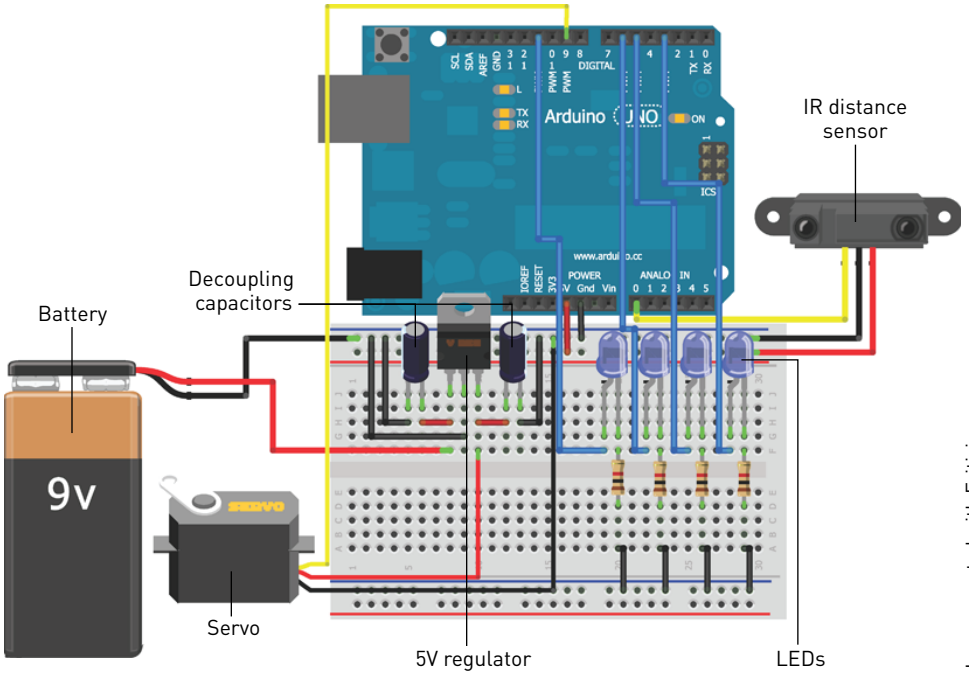


Figure 4-14: Sweeping distance sensor wiring diagram

The last step is to program the sensor. The system works in the following manner: Rotate to a given position, measure the distance, convert it to a value that can be used for the LED, change that LED's brightness, move to the next position, and so on, and so forth. Listing 4-5 shows the code to accomplish this. Copy it into a new sketch and upload it to your Arduino.

Listing 4-5: Sweeping Distance Sensor—sweep.ino

```
//Sweeping Distance Sensor
#include <Servo.h>

 const int SERVO  =9;   //Servo on Pin 9
 const int IR     =0;   //IR Distance Sensor on Analog Pin 0
 const int LED1   =3;   //LED Output 1
 const int LED2   =5;   //LED Output 2
 const int LED3   =6;   //LED Output 3
 const int LED4   =11;  //LED Output 4

Servo myServo;      //Servo Object
int dist1 = 0;      //Quadrant 1 Distance
int dist2 = 0;      //Quadrant 2 Distance
int dist3 = 0;      //Quadrant 3 Distance
```

```
int dist4 = 0;      //Quadrant 4 Distance

void setup()
{
    myServo.attach(SERVO); //Attach the Servo
    pinMode(LED1, OUTPUT); //Set LED to Output
    pinMode(LED2, OUTPUT); //Set LED to Output
    pinMode(LED3, OUTPUT); //Set LED to Output
    pinMode(LED4, OUTPUT); //Set LED to Output
}

void loop()
{
    //Sweep the Servo into 4 regions and change the LEDs
    dist1 = readDistance(15);      //Measure IR Distance at 15 degrees
    analogWrite(LED1, dist1);      //Adjust LED Brightness
    delay(300);                    //delay before next measurement

    dist2 = readDistance(65);      //Measure IR Distance at 65 degrees
    analogWrite(LED2, dist2);      //Adjust LED Brightness
    delay(300);                    //delay before next measurement

    dist3 = readDistance(115);     //Measure IR Distance at 115 degrees
    analogWrite(LED3, dist3);      //Adjust LED Brightness
    delay(300);                    //delay before next measurement

    dist4 = readDistance(165);     //Measure IR Distance at 165 degrees
    analogWrite(LED4, dist4);      //Adjust LED Brightness
    delay(300);                    //delay before next measurement
}

int readDistance(int pos)
{
    myServo.write(pos);                 //Move to given position
    delay(600);                         //Wait for Servo to move
    int dist = analogRead(IR);          //Read IR Sensor
    dist = map(dist, 50, 500, 0, 255); //scale it to LED range
    dist = constrain(dist, 0, 255);     //Constrain it
    return dist;                        //Return scaled distance
}
```

The program employs a simple function that rotates the servo to the requested degree, takes the distance measurement, scales it, and then returns it to the `loop()`. Which map you choose for the LED range depends on the setup of your system. I found that the closest object I wanted to detect registered around `500`, and the farthest object was around `50`, so the `map()` was set accordingly. `loop()` executes this function for each of the four LEDs, then repeats. When complete, your system should function similarly to the one shown in the demo video listed at the beginning of this section.

Summary

In this chapter you learned about the following:

- DC motors use electromagnetic induction to create mechanical action from changes in current.
- Motors are inductive loads that must utilize proper protection and power circuitry to interface safely with your Arduino.
- DC motor speed and direction can be controlled with PWM and an H-bridge.
- Servo motors enable precise positioning and can be controlled using the Arduino Servo library.
- A linear regulator can be used to create a secondary 5V supply from a 9V battery.
- IR distance sensors return analog values representing distances detected by bouncing infrared light off objects.
- Code commenting is critical for easing debugging and sharing.

CHAPTER

5

Making Sounds

Parts You'll Need for This Chapter

Arduino Uno

USB cable

Pushbuttons (×5)

10kΩ resistors (×5)

150Ω resistor

Jumper wires

Breadboard

10KΩ potentiometer

8Ω loudspeaker

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, video, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch5`.

The wiley.com code downloads for this chapter are found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 05 download and individually named according to the names throughout the chapter.

Humans have five senses. As you might have guessed, you won't be interfacing your sense of taste with too many electronics; licking your Arduino is a poor idea. Similarly, smell won't generally come into play. In fact, if you can smell your electronics, something is probably burning (and you should stop what you're doing). That just leaves the senses of touch, sight, and sound. You've already interfaced with potentiometers and buttons that take advantage of your sense of touch, and you've hooked up LEDs that interface with you sense of sight. Now, what about your auditory senses? This chapter focuses on using the Arduino to make sounds so that you can more easily gather feedback from your projects.

You can generate sound with an Arduino in a number of ways. The simplest method is to use the `tone()` function, which this chapter focuses on most heavily. However, you can also use various shields that add more complex, music-playing capabilities to Arduino with the help of some external processing. (Shields are add-on boards that attach to the top of your Arduino to add specific functionality. You won't use any in this chapter, but you'll be using assorted shields in some of the later chapters.) If you own the Arduino Due, you can use its true digital-to-analog converter (DAC) to produce sounds.

Understanding How Speakers Work

Before you can make sounds with your Arduino, you need to understand what sounds are and how humans perceive them. In this first section, you learn about how sound waves are generated, their properties, and how manipulation of those properties can produce music, voices, and so on.

The Properties of Sound

Sound is transmitted through the air as a pressure wave. As an object such as a speaker, a drum, or a bell vibrates, that object also vibrates the air around it. As the air particles vibrate, they transfer energy to the particles around them, vibrating these particles as well. In this fashion, a pressure wave is transferred from the source to your eardrum, by creating a chain reaction of vibrating particles. So, why do you need to know this to understand how to make sounds with your Arduino?

You can control two properties of these vibrating particles with your Arduino: frequency and amplitude. The *frequency* represents how quickly the air particles vibrate back and forth, and the *amplitude* represents the magnitude of their vibrations. In the physical sense, higher amplitude sounds are louder, and lower amplitude sounds are quieter. High-frequency sounds are a higher pitch (like a soprano), and low-frequency sounds are a lower pitch (like bass). Consider the diagram in Figure 5-1, which shows sinusoidal representations of sound waves of various amplitudes and frequencies.

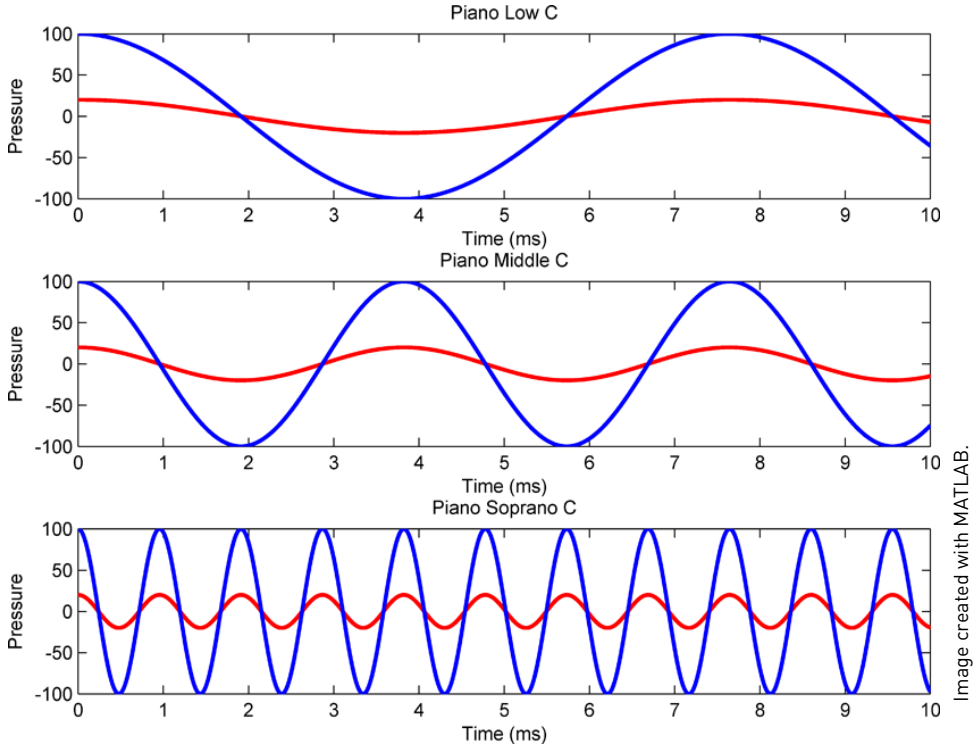


Figure 5-1: Sound waves of varying frequencies and amplitudes

Figure 5-1 shows three piano notes: low, middle, and soprano C. Each one shows the given frequencies at both low and high amplitudes. As an example, to understand frequency and amplitude, focus on middle C. Middle C has a frequency of 261.63 Hertz (Hz). In other words, a speaker, a guitar string, or a piano string would complete 261.63 oscillations per second. By taking the reciprocal of that value, you can find the period of the wave, which is easy to see in Figure 5-1. 1/261.63 equals 3.822 milliseconds, which is the width of one complete oscillation in the graph. Using the Arduino, you can set that period for a square wave and thus adjust the tone of the note.

Importantly, the Arduino (excluding the Due's true DAC) cannot actually make a sinusoidal wave that you might observe in the real world. A square wave is a digital periodic wave—it also oscillates between a high and a low value, but it switches instantaneously, instead of slowly like a sine wave. This still creates a pressure wave that results in sound, but it isn't quite as "pretty" sounding as a sinusoidal wave.

As for the amplitude, you can control that by changing the amount of the current permitted to flow through the speaker. Using a potentiometer in-line with the speaker, you can dynamically adjust the volume level of the speaker.

How a Speaker Produces Sound

Speakers, much like the motors that you learned about in the preceding chapter, take advantage of electromagnetic forces to turn electricity into motion. Try holding a piece of metal up to the rear of your speaker. Did you notice anything interesting? The metal probably sticks to the rear of your speaker, because all speakers have a sizeable permanent magnet mounted to the back. Figure 5-2 shows a cross section of a common speaker.

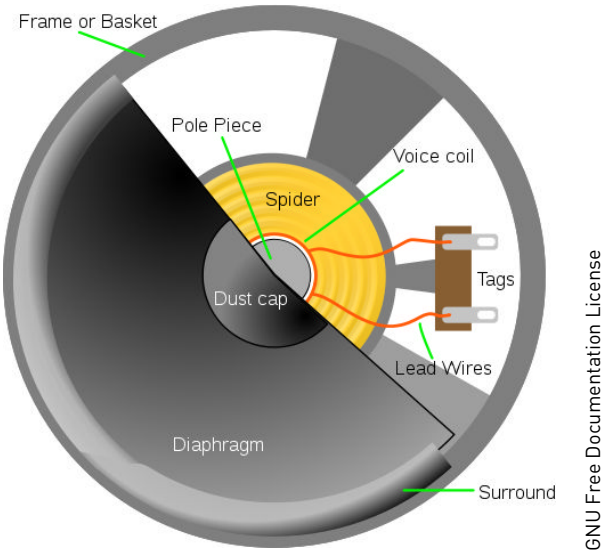


Figure 5-2: Speaker cross section

The permanent magnet is mounted behind the voice coil and pole piece shown in the image. As you send a sinusoidal voltage signal (or a square wave in the case of the Arduino) into the leads of the coil, the changing current induces a magnetic field that causes the pole piece and diaphragm to vibrate up and down as the permanent magnet is attracted to and then repulsed by the magnetic field that you have generated. This back-and-forth vibration, in turn, vibrates the air in front of the speaker, effectively creating a sound wave that can travel to your eardrum.

Using tone() to Make Sounds

The Arduino IDE includes a built-in function for easily making sounds of arbitrary frequencies. The `tone()` function generates a square wave of the selected frequency on the output pin of your choice. The `tone()` function accepts three arguments, though the last one is optional:

- The first argument sets the pin to generate the tone on.
- The second argument sets the frequency of the tone.
- The third (optional) argument sets the duration of the tone. If the third argument is not set, the tone continues playing until you call `noTone()`.

Because `tone()` uses one of the ATMega's hardware timers, you can start a tone and do other things with your Arduino while it continues to play sound in the background.

In the following sections, you learn how to play arbitrary sound sequences. Once you've gotten that working, you can use `tone()` as a response to various inputs (buttons, distance sensors, accelerometers, etc.). At the end of the chapter, you build a simple five-button piano that you can play.

Including a Definition File

When it comes to playing music, a definition file that maps frequencies to note names proves useful. This makes it more intuitive to play simple musical clips. For those familiar with reading sheet music, you know that notes are denoted with letters representing their pitch. The Arduino IDE includes a header file that correlates each of these notes with its respective frequency. Instead of digging through the Arduino install directory to find it, just visit the Exploring Arduino Chapter 5 webpage, and download the pitch file to your desktop (`www.exploringarduino.com/content/ch5`). You'll place it in your sketch directory after you've created it.

Next, open your Arduino IDE and save the blank sketch that is automatically created when you open the IDE. As you've probably already noticed, when you save a sketch, it actually saves a folder with that name and places an .ino file inside of that folder. By adding other files to that folder, you can include them in your program, all while keeping your code better organized. Copy the pitches.h file you saved to the desktop into the folder created by the IDE; then close the Arduino IDE. Open your .ino file in the Arduino IDE and notice the two tabs that now appear (see Figure 5-3).

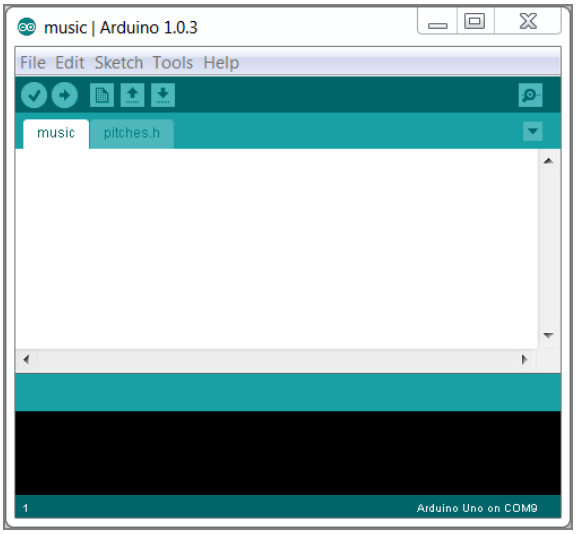


Figure 5-3: Arduino IDE with a secondary header file

Click the pitches.h tab to see the contents of the file. Notice that it's just a list of definition statements, which map human-readable names to given frequency values. Simply having the header file in the IDE does not suffice, though. To ensure that the compiler actually uses those definitions when compiling your program for the Arduino, you need to tell the compiler to look for that file. Doing so is easy. Just add this line of code to the top of your .ino file:

```
#include "pitches.h" //Header file with pitch definitions
```

To the compiler, this is essentially the same thing as copying and pasting the contents of the header file into the top of your main file. However, this keeps the file neater and easier for you to read. In the next sections, you write the code for the rest of this file so that you can actually use the pitch definitions that you have just imported.

Wiring the Speaker

Now that you have your pitches header file included, you're ready to build a test circuit and to write a simple program that can play some music. The electrical setup is fairly simple and just involves hooking up a speaker to an output pin of your Arduino. However, remember what you've learned in previous chapters about current-limiting resistors.

Just as with LEDs, you want to put a current-limiting resistor in series with the speaker to ensure that you don't try to draw too much current from one of the Arduino's I/O pins. As you learned previously, each I/O pin can supply only a max of 40mA, so pick a resistor that prevents you from exceeding that. The

speaker that comes with this book's kit has an internal resistance of 8Ω (as do most loudspeakers that you can buy); this resistance comes from the windings of wire that make up the electromagnet. Recall that Ohm's law states that V = IR. In this scenario, the I/O pin is outputting 5V, and you don't want to exceed 40mA. Solving for R, you find that the minimum resistance must be: R = 5V / 40mA = 125Ω. 8Ω is already accounted for by the speaker, so your in-line resistor must be at least 125Ω – 8Ω = 117Ω. The nearest common resistor is 150Ω, so you can use that. By adjusting that resistor value, you can change the volume of the speaker. To make this as easy as possible, you can use a potentiometer in-line with the 150Ω resistor, as shown in Figure 5-4. In the schematic, R1 is the 150Ω resistor, and R2 is the potentiometer.

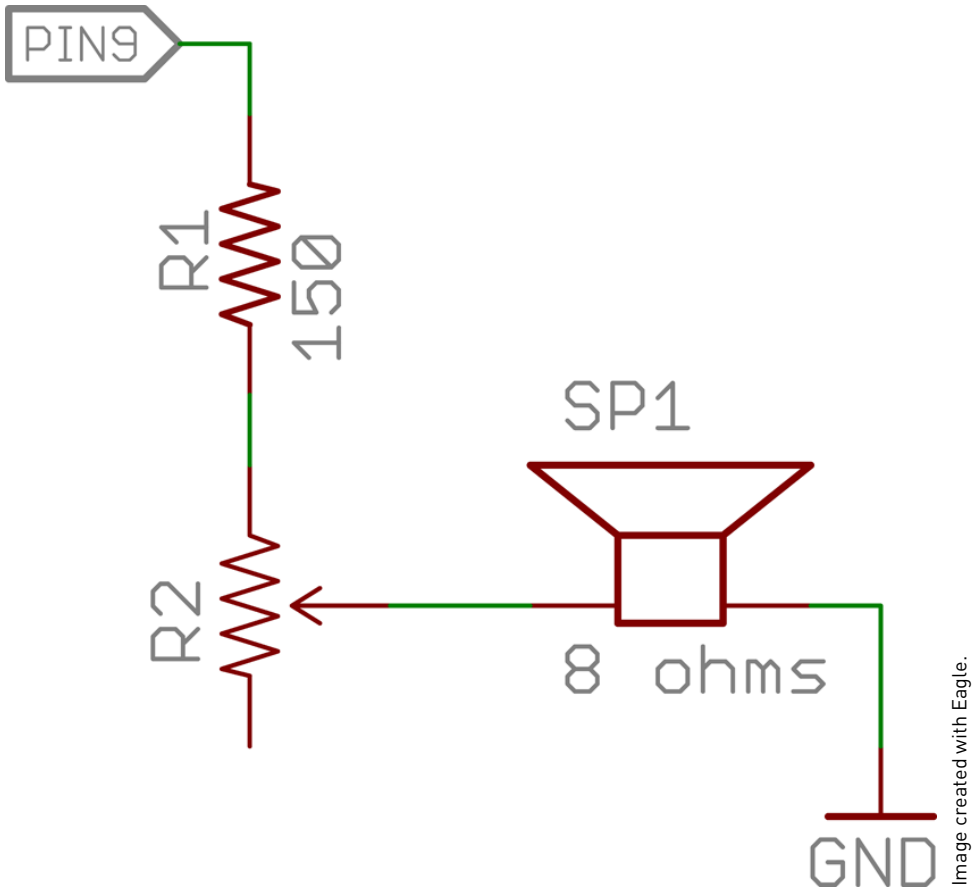


Figure 5-4: Speaker wiring with volume adjustment knob

Note that unlike in your previous usages of potentiometers this configuration uses only two pins: the middle (or wiper) pin goes to the speaker, and either one

of the end pins connects to the 150Ω resistor. When the knob is turned all the way toward the unconnected terminal, the entire resistance of the potentiometer is added to the series resistance of the 150Ω resistor, and the volume lowers. When the knob is turned all the way toward the connected end terminal, it adds no resistance to the series, and the speaker is at max volume. Referencing the schematic in Figure 5-4, wire your speaker to the Arduino. Then, confirm your wiring using the diagram in Figure 5-5.

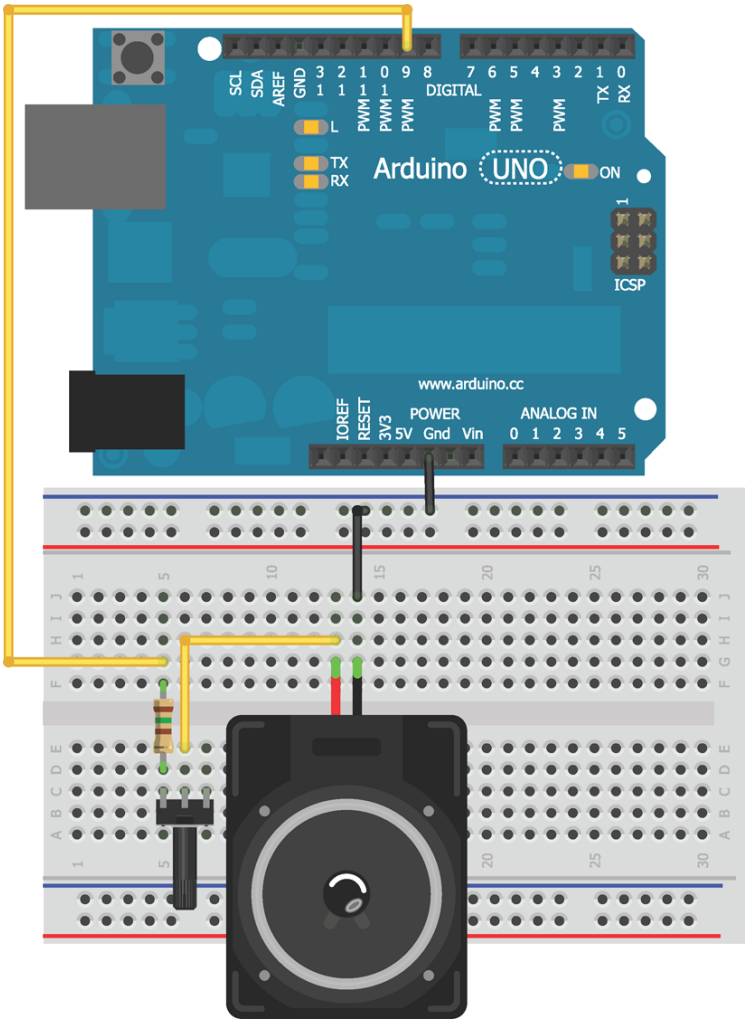


Image created with Fritzing.

Figure 5-5: Speaker wiring diagram

Speakers do not have a polarity; you can connect them in either direction. After wiring your speaker successfully, you're ready to make music!

Making Sound Sequences

To play back some songs, you first learn about using arrays to store multiple values easily. You then implement a simple loop to iterate through the arrays of notes and play them back on the speaker.

Using Arrays

An array is a sequence of values that are related in some way. By grouping them together, it is an ideal format to iterate through. You can think of an array as a numbered list. Each position has an index that indicates its location in the list, and each index has a value that you want to store. You use an array here to store the list of notes that you want to play, in the order that you want to play them.

To ensure that the Arduino's memory is properly managed, you need to declare arrays with a known length. You can do this either by explicitly specifying the number of items or by simply populating the array with all the values you are interested in. For example, if you want to make an array that contains four integer values, you could create it like this:

```
int numbers[4];
```

You can optionally initialize the values when you declare it. If you initialize the values, specifying the length in the brackets is optional. If unspecified, the length is assumed to equal the number elements that you initialized:

```
//Both of these are acceptable
int numbers[4] = {-7, 0, 6, 234};
int numbers[] = {-7, 0, 6, 234};
```

Note that arrays are zero indexed. In other words, the first number is at position `0`, the second is at position `1`, and so forth. You can access the elements in an array at any given index by putting the index of the relevant value in a square bracket after the variable name. If you want to set the brightness of an LED connected to pin 9 to the third entry in an array, for example, you can do so like this:

```
analogWrite(9,numbers[2]);
```

Note that because numbering starts at zero, the index of `2` represents the third value in the array. If you want to change one of the values of the array, you can do so in a similar fashion:

```
numbers[2] = 10;
```

Next, you will use arrays (as shown in these examples) to create a structure that can hold the sequence of notes that you want to play on your speaker.

Making Note and Duration Arrays

To store the info about the song you want to play, you can use two arrays of the same length. The first contains the list of pitches, and the second contains the list of durations for which each note should play in milliseconds. You can then iterate through the indices of these arrays and play back your tune.

Using the meager musical skills that I've maintained from my music classes back in high school, I've assembled a short and catchy tune:

```
//Note Array
int notes[] = {
 NOTE_A4, NOTE_E3, NOTE_A4, 0,
 NOTE_A4, NOTE_E3, NOTE_A4, 0,
 NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
 NOTE_E4, NOTE_E3, NOTE_A4, 0
};

//The Duration of each note (in ms)
int times[] = {
 250, 250, 250, 250,
 250, 250, 250, 250,
 125, 125, 125, 125, 125, 125, 125, 125,
 250, 250, 250, 250
};
```

Note that both arrays are the same length: 20 items. Notice that some of the notes are specified as `0`. These are musical rests (unplayed beats). Each note pairs with a duration from the second array. For those familiar with music theory, note that I've made quarter notes 250ms and eighth notes 125ms. The song is in "four-four" time, in musical terms.

Try out this given note sequence, first; then try to create your own!

NOTE Listen to a recording of this tune, played by an Arduino: `www.exploringarduino.com/content/ch5`. You can also find this recording on the Wiley website shown at the beginning of this chapter.

Completing the Program

The last step is to actually add playback functionality to the sketch. This can be accomplished with a simple `for` loop that goes through each index in the array, and plays the given note for the given duration. Since you presumably don't want to listen to this over and over again, you can put the playback functionality in the `setup()` function so that it only happens once. You can restart playback by hitting the Reset button. Listing 5-1 shows the complete playback program.

Listing 5-1: Arduino Music Player—music.ino

```
//Plays a song on a speaker

#include "pitches.h" //Header file with pitch definitions

const int SPEAKER=9;  //Speaker Pin

//Note Array
int notes[] = {
 NOTE_A4, NOTE_E3, NOTE_A4, 0,
 NOTE_A4, NOTE_E3, NOTE_A4, 0,
 NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
 NOTE_E4, NOTE_E3, NOTE_A4, 0
};

//The Duration of each note (in ms)
int times[] = {
 250, 250, 250, 250,
 250, 250, 250, 250,
 125, 125, 125, 125, 125, 125, 125, 125,
 250, 250, 250, 250
};


void setup()
{
 //Play each note for the right duration
 for (int i = 0; i < 20; i++)
 {
  tone(SPEAKER, notes[i], times[i]);
  delay(times[i]);
 }
}

void loop()
{
 //Press the Reset button to play again.
}
```

If you want to make your own music, make sure that the arrays remain of an equal length and that you change the upper bound on the `for()` loop. Because the `tone()` function can run in the background, it's important to use the `delay()` function. By delaying the code for an amount of time equal to the duration of the note, you ensure that the Arduino doesn't play the next note until the previous not has finished playing for the time you specified.

Understanding the Limitations of the tone() Function

The `tone()` function does have a few limitations to be aware of. Like the servo library, `tone()` relies on a hardware timer that is also used by the board's pulse-width modulation (PWM) functionality. If you use `tone()`, PWM does not work right on pins 3 and 11 (on boards other than the Mega).

Also remember that the Arduino I/O pins are not digital-to-analog converters (DACs). Hence, they output only a square wave at the provided frequency, not a sine wave. Although this suffices for making tones with a speaker, you'll find it undesirable for playing back music. If you want to play back wave files, your options include using a music-playing shield (such as the adafruit Wave Shield or the SparkFun MP3 shield), implementing a DAC converter, or using the built-in DAC available on the Arduino Due using the Due-only Audio library.

The last limitation is that you can use the `tone()` function on only one pin at a time, so it isn't ideal for driving multiple speakers. If you want to drive multiple speakers at the same time from a standard Arduino, you have to use manual timer interrupt control, something you learn more about in Chapter 12, "Hardware and Timer Interrupts."

NOTE To read a tutorial on advanced multispeaker control with an Arduino, visit `www.jeremyblum.com/2010/09/05/driving-5-speakers-simultaneously-with-an-arduino/`. You can also find this tutorial on the Wiley website shown at the beginning of this chapter.

Building a Micro Piano

Playing back sequences of notes is great for adding audio feedback to projects you've already created. For example, consider replacing or augmenting a green confirmation LED with a confirmation sound. But, what if you want to dynamically control the sound? To wrap up this chapter, you build a simple pentatonic piano. The pentatonic scale consists of just five notes per octave rather than the usual seven. Interestingly, the notes of a pentatonic scale have minimal dissonance between pitches, meaning they always sound good together. So, it makes a lot of sense to use pentatonic notes to make a simple piano.

NOTE The SudoGlove, among others things, is a control glove that can synthesize music using the pentatonic scale. You can learn more about it at `www.sudoglove.com`.

To make your Arduino piano, you use this pentatonic scale: C, D, E, G, A. You can choose what octave to use based on your preference. I chose to use the fourth octave from the header file.

First, wire five buttons up to your Arduino. As with the buttons in Chapter 2, "Digital Inputs, Outputs, and Pulse-Width Modulation" you use 10kΩ pull-down resistors with the buttons. In this scenario, you do not need to debounce the buttons because the note will be played only while the desired button is held down. Wire the buttons as shown in Figure 5-6 and keep the speaker wired as you had it previously.

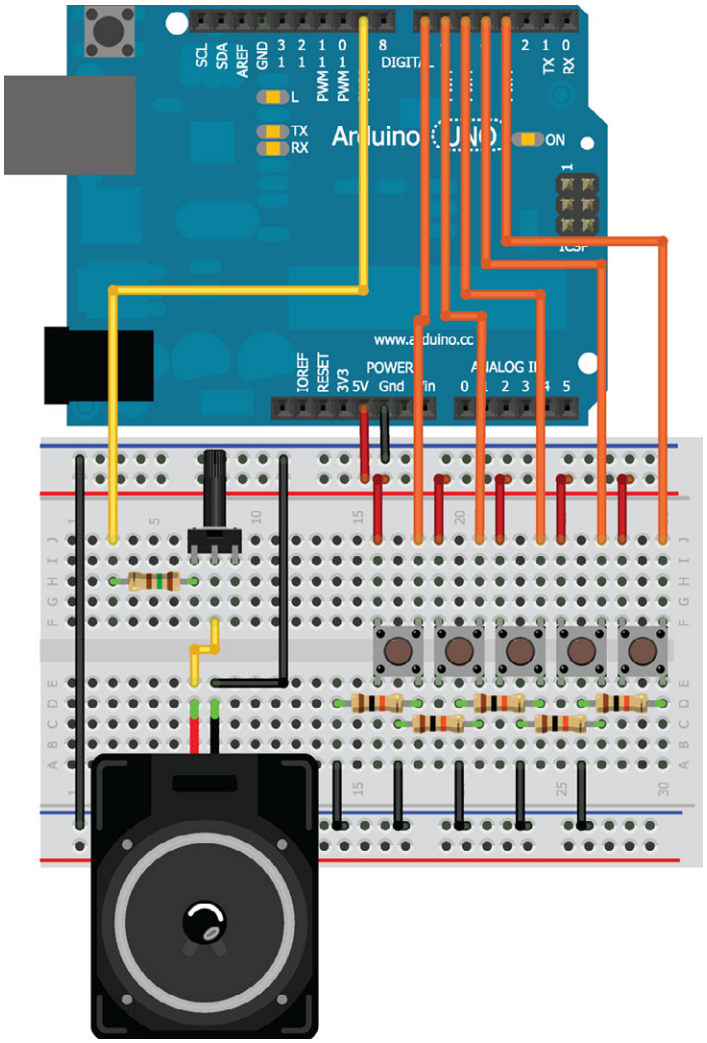


Image created with Fritzing.

Figure 5-6: Micro piano wiring diagram

The code for the piano is actually very simple. In each iteration through the loop, each button is checked. So long as a button is pressed, a note is played. Here, `tone()` is used without a duration because the note will play as long as the button is held. Instead, `noTone()` is called at the end of `loop()` to ensure that the speaker stops making noise when all the buttons have been released. Because only a few notes are needed, you can copy the values from the header file that you care about directly into the main program file. In a new sketch, load up the code shown in Listing 5-2 and upload it to your Arduino. Then, jam away on your piano!

Listing 5-2: Pentatonic Micro Piano—piano.ino

```
//Pentatonic Piano
//C D E G A

#define NOTE_C  262 //Hz
#define NOTE_D  294 //Hz
#define NOTE_E  330 //Hz
#define NOTE_G  392 //Hz
#define NOTE_A  440 //Hz

const int SPEAKER=9;  //Speaker on pin 9

const int BUTTON_C=7;  //Button pin
const int BUTTON_D=6;  //Button pin
const int BUTTON_E=5;  //Button pin
const int BUTTON_G=4;  //Button pin
const int BUTTON_A=3;  //Button pin


void setup()
{
 //No setup needed
 //Tone function sets outputs
}

void loop()
{
 while (digitalRead(BUTTON_C))
  tone(SPEAKER, NOTE_C);
 while(digitalRead(BUTTON_D))
  tone(SPEAKER, NOTE_D);
 while(digitalRead(BUTTON_E))
  tone(SPEAKER, NOTE_E);
 while(digitalRead(BUTTON_G))
  tone(SPEAKER, NOTE_G);
 while(digitalRead(BUTTON_A))
  tone(SPEAKER, NOTE_A);

 //Stop playing if all buttons have been released
 noTone(SPEAKER);
}
```

Each `while()` loop will continuously call the `tone()` function at the appropriate frequency for as long as the button is held down. The button can be read within the `while()` loop evaluation to avoid having to first save the reading to a temporary value. `digitalRead()` returns a Boolean "true" whenever a button input goes high; the value can be evaluated directly by the `while()` loop. To keep your code neater, you don't need to use brackets for the contents of a loop if the contents are only one line, as in this example. If you have multiple lines, you must use curly brackets as you have in previous examples.

NOTE To watch a demo video of the micro piano, visit `www.exploringarduino.com/content/ch5`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Summary

In this chapter you learned about the following:

- Speakers create a pressure wave that travels through the air and is perceived as sound by your ears.
- Changing electric current induces a magnetic field that can be used to create sound from a speaker.
- The `tone()` function can be used to generate sounds of arbitrary frequencies and durations.
- The Arduino programming language supports the use of arrays for iterating through sequences of data.
- Speaker volume can be adjusted using a potentiometer in series with a speaker.

CHAPTER 6

USB and Serial Communication

Parts You'll Need for This Chapter

Arduino Uno
Arduino Leonardo
USB cable (A to B for Uno)
USB cable (A to Micro B for Leonardo)
LED
RGB LED (common cathode)
150Ω resistor
220Ω resistor (×3)
10kΩ resistor (×2)
Pushbutton
Photoresistor
TMP36 temperature sensor
Two-axis joystick (SparkFun, Parallax, or adafruit suggested)
Jumper wires
Breadboard
Potentiometer

CODE AND DIGITAL CONTENT FOR THIS CHAPTER

Code downloads, video, and other digital content for this chapter can be found at `www.exploringarduino.com/content/ch6`.

In addition, all code can be found at `www.wiley.com/go/exploringarduino` on the Download Code tab. The code is in the chapter 06 download and individually named according to the names throughout the chapter.

Perhaps the most important part of any Arduino is its capability to be programmed directly via a USB serial port. This feature enables you to program the Arduino without any special hardware, such as an AVR ISP MKII. Ordinarily, microcontrollers rely on a dedicated piece of external hardware (such as the MKII) to serve as a *programmer* that connects between your computer and the microcontroller you are trying to program. In the case of the Arduino, this programmer is essentially built into the board, instead of being a piece of external hardware. What's more, this gives you a direct connection to the ATMega's integrated Universal Synchronous/Asynchronous Receiver and Transmitter (USART). Using this interface, you can send information between your host computer and the Arduino, or between the Arduino and other serial-enabled components (including other Arduinos).

This chapter covers just about everything you could want to know about connecting an Arduino to your computer via USB and transmitting data between the two. Different Arduinos have different serial capabilities, so this chapter covers each of them, and you build sample projects with each serial communication technology to get yourself acquainted with how to take advantage of them as best as possible. Note that, as a result of this, the parts list includes several types of Arduinos. Depending on which Arduino you are trying to learn about, you can pick and choose which sections to read, which examples to explore, and which parts from the parts list you actually need for your Arduino explorations.

Understanding the Arduino's Serial Communication Capabilities

As already alluded to in the introduction to this chapter, the different Arduino boards offer lots of different serial implementations, both in terms of how the hardware implements the USB-to-serial adapters and in terms of the software support for various features. First, in this section, you learn about the various serial communication hardware interfaces offered on different Arduino boards.

NOTE To learn all about serial communication, check out this tutorial: `www.jeremyblum.com/2011/02/07/arduino-tutorial-6-serial-communication-and-processing/`. You can also find this tutorial on the Wiley website shown at the beginning of this chapter.

To begin, you need to understand the differences between serial and USB. Depending on how old you are, you might not even remember serial (or technically, RS-232) ports, because they have been primarily replaced by USB. Figure 6-1 shows what a standard serial port looks like.

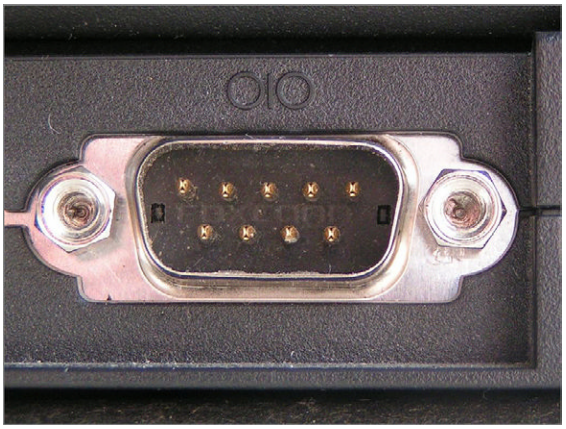


Figure 6-1: Serial port

The original Arduino boards came equipped with a serial port that you connected to your computer with a 9-pin serial cable. Nowadays, few computers still have these ports, although you can use adapters to make DB-9 (the type of 9-pin connector) serial ports from USB ports. Microcontrollers like the ATMega328P that you find on the Arduino Uno have one hardware serial port. It includes a transmit (TX) and receive (RX) pin that can be accessed on digital pins 0 and 1. As explained in the sidebar in Chapter 1, "Getting Up and Blinking with the Arduino," the Arduino is equipped with a bootloader that allows you to program it over this serial interface. To facilitate this, those pins are "multiplexed" (meaning that they are connected to more than one function); they connect, indirectly, to the transmit and receive lines of your USB cable. However, serial and USB are not directly compatible, so one of two methods is used to bridge

the two. Option one is to use a secondary integrated circuit (IC) to facilitate the conversion between the two (either on or off the Arduino board). This is the type of interface present on an Uno, where an intermediary IC facilitates USB-to-serial communication. Option two is to opt for a microcontroller that has a USB controller built in (such as the Arduino Leonardo's 32U4 MCU).

Arduino Boards with an Internal or External FTDI USB-to-Serial Converter

As just explained, many Arduino boards (and Arduino clones) use a secondary integrated circuit to facilitate the USB-to-serial conversion. The "FTDI" chip is a popular chip that has just one function: convert between serial and USB. When your computer connects to an FTDI chip, it shows up in your computer as a "Virtual Serial Port" that you can access as if it was a DB9 port wired right into your computer. Figure 6-2 shows the bottom of an Arduino Nano, which utilizes an integrated FTDI chip.

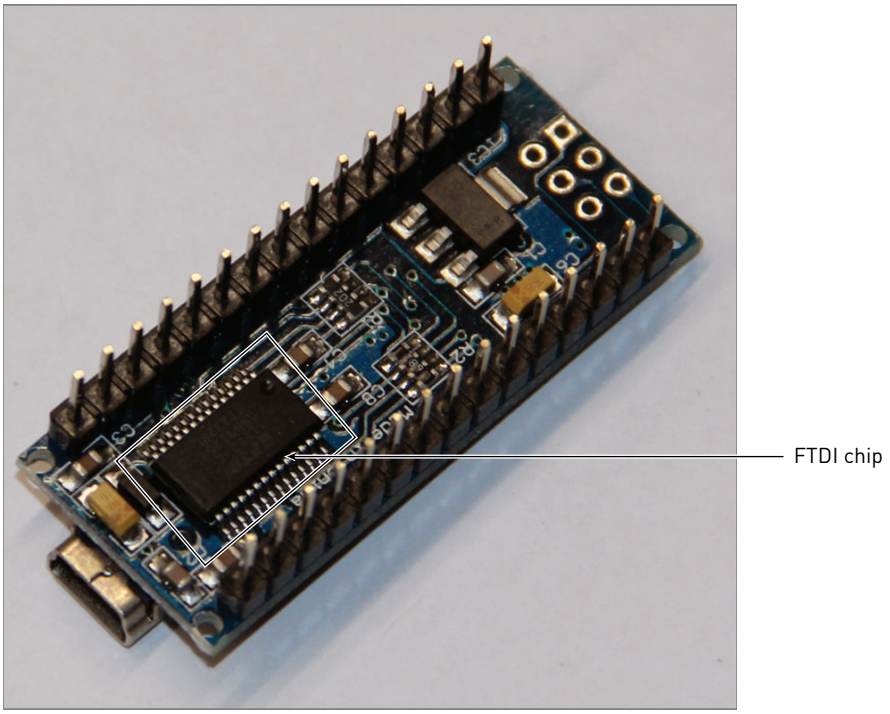


Figure 6-2: Arduino Nano with integrated FTDI chip shown

NOTE For your computer to communicate with a FTDI serial-to-USB adapter, you need to install drivers. You can find the most recent versions for Windows, OS X, and Linux at `www.ftdichip.com/Drivers/VCP.htm`. This is also linked from the Chapter 6 page on the Exploring Arduino website.

On some boards, usually to reduce board size, the FTDI chip is external to the main board, with a standardized 6-pin "FTDI connector" left for connecting to either an FTDI cable (A USB cable with an FTDI chip built in to the end of the cable) or a small FTDI breakout board. Figures 6-3 and 6-4 show these options.



Credit: adafruit Industries, www.adafruit.com

Figure 6-3: FTDI cable

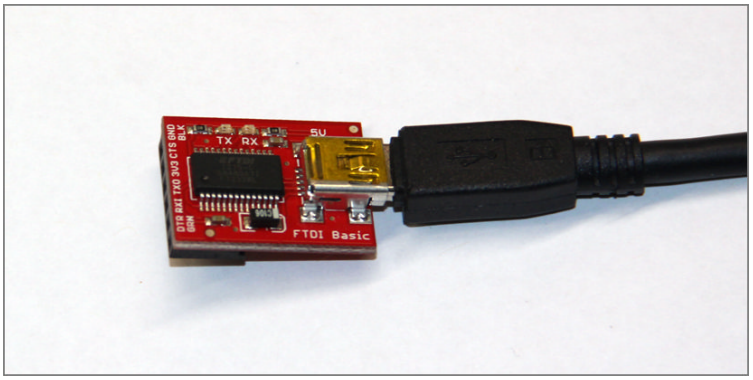


Figure 6-4: SparkFun FTDI adapter board

Using a board with a removable FTDI programmer is great if you are designing a project that will not need to be connected to a computer via USB to run. This will reduce cost if you are making several devices, and will reduce overall size of the finished product.

Following is a list of Arduino boards that use an onboard FTDI chip. Note, new Arduino boards no longer use an FTDI chip, so most of these have been discontinued. However, there are still many clones of these boards available for purchase, so they are listed here for completeness:

- Arduino Nano
- Arduino Extreme
- Arduino NG
- Arduino Diecimila
- Arduino Duemilanove
- Arduino Mega (original)

Following is a list of Arduino boards that use an external FTDI programmer:

- Arduino Pro
- Arduino Pro Mini
- LilyPad Arduino
- Arduino Fio
- Arduino Mini
- Arduino Ethernet

Arduino Boards with a Secondary USB-Capable ATMega MCU Emulating a Serial Converter

The Arduino Uno was the first board to introduce the use of an integrated circuit other than the FTDI chip to handle USB-to-serial conversion. Functionally, it works exactly the same way, with a few minor technical differences. Figure 6-5 shows the Uno's 8U2 serial converter (now a 16U2 on newer revisions).

Following is a brief list of the differences:

- First, in Windows, boards with this new USB-to-serial conversion solution require a custom driver to be installed. This driver comes bundled with the Arduino IDE when you download it. (Drivers are not needed for OS X or Linux.)

- Second, the use of this second microcontroller unit (MCU) for the conversion allowed a custom Arduino vendor ID and product ID to be reported to the host computer when the board is connected. When an FTDI-based board was connected to a computer, it just showed up as generic USB-serial device. When an Arduino using a non-FTDI converter IC (an ATMega 8U2 in the case of early Arduino Unos, now a 16U2) is connected, it is identified to the computer as an Arduino.

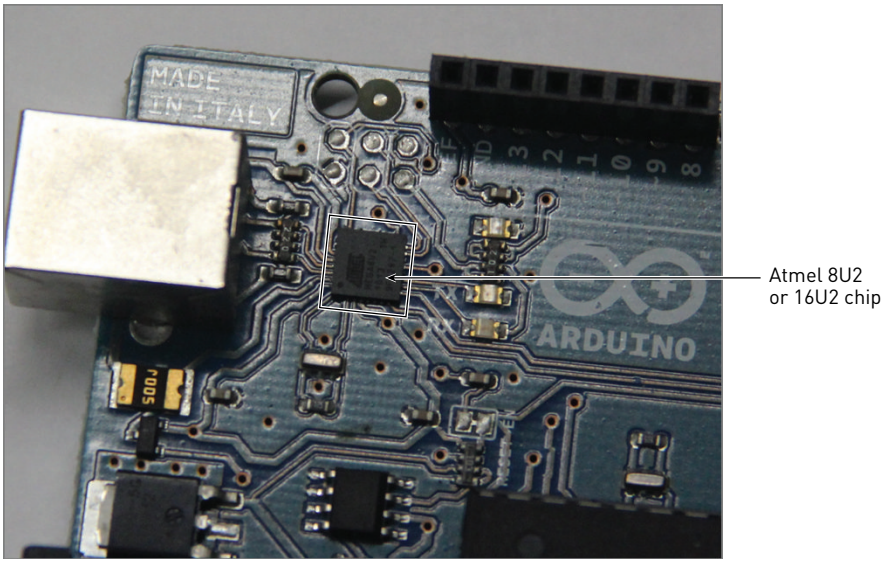


Figure 6-5: View of the Arduino Uno's 8U2 serial converter chip

- Lastly, because the secondary MCU is fully programmable (it's running a firmware stack called LUFA that emulates a USB-to-serial converter), you can change its firmware to make the Arduino show up as something different from a virtual serial port, such as a joystick, keyboard, or MIDI device. If you were to make this sort of change, the USB-to-serial LUFA firmware would not be loaded, and you would have to program the Arduino directly using the in-circuit serial programmer with a device like the AVR ISP MKII.

Following is a list of Arduino boards that use an onboard secondary MCU to handle USB-to-serial conversion:

- Arduino Uno
- Arduino Mega 2560
- Arduino Mega ADK (based on 2560)
- Arduino Due (can also be programmed directly)

Arduino Boards with a Single USB-Capable MCU

The Arduino Leonardo was the first board to have only one chip that acts both as the user-programmable MCU and as the USB interface. The Leonardo (and similar Arduino boards) employs the ATMega 32U4 microcontroller, a chip that has direct USB communication built in. This feature results in several new features and improvements.

First, board cost is reduced because fewer parts are required, and because one less factory programming step is needed to produce the boards. Second, the board can more easily be used to emulate USB devices other than a serial port (such as a keyboard, mouse, or joystick). Third, the single ordinary USART port on the ATMega does not have be multiplexed with the USB programmer, so communication with the host computer and a secondary serial device (such as a GPS unit) can happen simultaneously.

Following is a list of Arduino boards that use a single USB-capable MCU:

- Arduino Due (can also be programmed via secondary MCU)
- LilyPad Arduino USB
- Arduino Esplora
- Arduino Leonardo
- Arduino Micro

Arduino Boards with USB-Host Capabilities

Some Arduino boards can connect to USB devices as a host, enabling you to connect traditional USB devices (keyboards, mice, Android phones) to an Arduino. Naturally, there must be appropriate drivers to support the device you are connecting to. For example, you cannot just connect a webcam to an Arduino Due and expect to be able to snap photos with no additional work. The Due presently

supports a USB host class that enables you to plug a keyboard or mouse into the Due's on-the-go USB port to control it. The Arduino Mega ADK uses the Android Open Accessory Protocol (AOA) to facilitate communication between the Arduino and an Android device. This is primarily used for controlling Arduino I/O from an application running on the Android device.

Two Arduino boards that have USB-host capabilities are the Arduino Due and the Arduino Mega ADK (based on Mega 2560).

Listening to the Arduino

The most basic serial function that you can do with an Arduino is to print to the computer's serial terminal. You've already done this in several of the previous chapters. In this section, you explore the functionality in more depth, and later in the chapter you build some desktop apps that respond to the data you send instead of just printing it to the terminal. This process is the same for all Arduinos.

Using print Statements

To print data to the terminal, you only need to utilize three functions:

- `Serial.begin(baud_rate)`
- `Serial.print("Message")`
- `Serial.println("Message")`

where `baud_rate` and `"Message"` are variables that you specify.

As you've already learned, `Serial.begin()` must be called once at the start of the program in `setup()` to prepare the serial port for communication. After you've done this, you can freely use `Serial.print()` and `Serial.println()` functions to write data to the serial port. The only difference between the two is that `Serial.println()` adds a carriage return at the end of the line (so that the next thing printed will appear on the following line). To experiment with this functionality, wire up a simple circuit with a potentiometer connected to pin A0 on the Arduino, as shown in Figure 6-6.

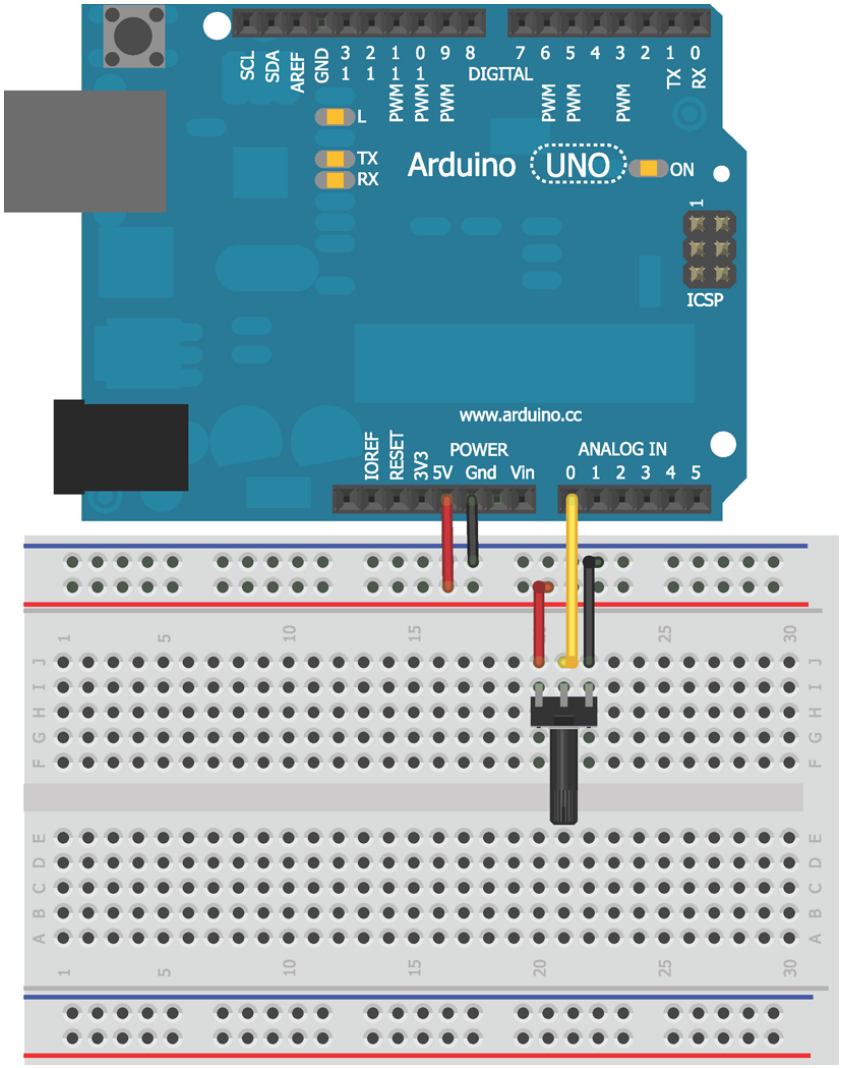


Figure 6-6: Potentiometer wiring diagram

After wiring your potentiometer, load on the simple program shown in Listing 6-1 that will read the value of the potentiometer and print it as both a raw value and a percentage value.

Listing 6-1: Potentiometer Serial Print Test Program—pot.ino

```
//Simple Serial Printing Test with a Potentiometer

const int POT=0;  //Pot on analog pin 0

void setup()
{
  Serial.begin(9600); //Start serial port with baud = 9600
}

void loop()
{
  int val = analogRead(POT);           //Read potentiometer
  int per = map(val, 0, 1023, 0, 100); //Convert to percentage
  Serial.print("Analog Reading: ");
  Serial.print(val);                   //Print raw analog value
  Serial.print("  Percentage: ");
  Serial.print(per);                   //Print percentage analog value
  Serial.println("%");                 //Print % sign and newline
  delay(1000);                         //Wait 1 second, then repeat
}
```

Using a combination of `Serial.print()` and `Serial.println()` statements, this code prints both the raw and percentage values once per second. Note that by our using `Serial.println()` only on the last line, each previous transmission stays on the same line.

Open the serial monitor from the Arduino IDE and ensure that your baud rate is set to 9600 to match the value set in the Arduino sketch. You should see the values printing out once per second as you turn the potentiometer.

Using Special Characters

You can also transmit a variety of "special characters" over serial, which allow you to change the formatting of the serial data you are printing. You indicate these special characters with a slash escape character (`\`) followed by a command character. There are a variety of these special characters, but the two of greatest interest are the tab and newline characters. To insert a tab character, add a `\t` to the string. To insert a newline character, add a `\n` to the string. This proves particularly useful if you want a newline to be inserted at the beginning of a string, instead of at the end as the `Serial.println()` function does. If, for some reason, you actually want to print `\n` or `\t` in the string, you can do so by printing `\\n` or `\\t`, respectively. Listing 6-2 is a modification of the previous code to use these special characters to show data in a tabular format.

Listing 6-2: Tabular Printing using Special Characters—pot_tabular.ino

```
//Tabular serial printing test with a potentiometer

const int POT=0;  //Pot on analog pin 0

void setup()
{
  Serial.begin(9600); //Start Serial Port with Baud = 9600
}

void loop()
{
  Serial.println("\nAnalog Pin\tRaw Value\tPercentage");
  Serial.println("-----------------------------------------");
  for (int i = 0; i < 10; i++)
  {
    int val = analogRead(POT);            //Read potentiometer
    int per = map(val, 0, 1023, 0, 100); //Convert to percentage

    Serial.print("A0\t\t");
    Serial.print(val);
    Serial.print("\t\t");
    Serial.print(per);            //Print percentage analog value
    Serial.println("%");          //Print % sign and newline
    delay(1000);                  //Wait 1 second, then repeat
  }
}
```

As you turn the potentiometer, the output from this program should look something like the results shown in Figure 6-7.

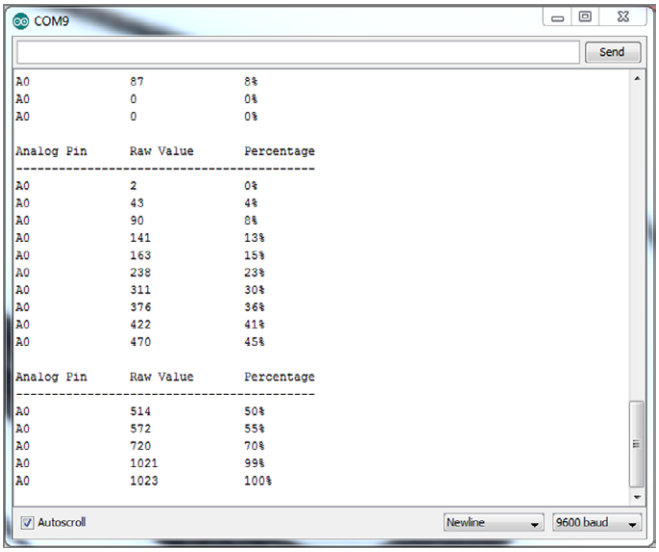


Figure 6-7: Screenshot of serial terminal with tabular data

Changing Data Type Representations

The `Serial.print()` and `Serial.println()` functions are fairly intelligent when it comes to printing out data in the format you are expecting. However, you have options for outputting data in various formats, including hexadecimal, octal, and binary. Decimal-coded ASCII is the default format. The `Serial.print()` and `Serial.println()` functions have an optional second argument that specifies the print format. Table 6-1 includes examples of how you would print the same data in various formats and how it would appear in your serial terminal.

Table 6-1: Serial Data Type Options

DATA TYPE	EXAMPLE CODE	SERIAL OUTPUT
Decimal	`Serial.println(23);`	23
Hexadecimal	`Serial.println(23, HEX);`	17
Octal	`Serial.println(23, OCT)`	27
Binary	`Serial.println(23, BIN)`	00010111

Talking to the Arduino

What good is a conversation with your Arduino if it's only going in one direction? Now that you understand how the Arduino sends data to your computer, let's spend some time discussing how to send commands from your computer to the Arduino. You've probably already noticed that the Arduino IDE serial monitor has a text entry field at the top, and a drop-down menu at the bottom. Figure 6-8 highlights both.

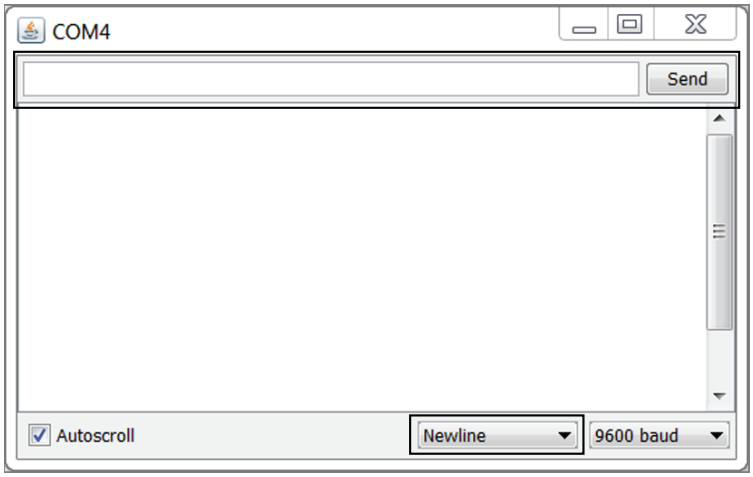


Figure 6-8: Screenshot of serial terminal highlighting text entry field and Line Ending Options drop-down menu

First, make sure that the drop-down is set to Newline. The drop-down menu determines what, if anything, is appended to end of your commands when you send them to the Arduino. The examples in the following sections assume that you have Newline selected, which just appends a `\n` to the end of anything that you send from the text entry field at the top of the serial monitor window.

Unlike with some other terminal programs, the Arduino IDE serial monitor sends your whole command string at one time (at the baud rate you specify) when you press the Enter key or the Send button. This is in contrast to other serial terminals like PuTTy (linked from this chapter's digital content page at `www.exploringarduino.com`) that send characters as you type them.

Reading Information from a Computer or Other Serial Device

You start by using the Arduino IDE serial monitor to send commands manually to the Arduino. Once that's working, you'll learn how to send multiple command values at once and how to build a simple graphical interface for sending commands.

It's important to recall that the Arduino's serial port has a buffer. In other words, you can send several bytes of data at once and the Arduino will queue them up and process them in order based on the content of your sketch. You do not need to worry about sending data faster than your loop time, but you do need to worry about sending so much data that it overflows the buffer and information is lost.

Telling the Arduino to Echo Incoming Data

The simplest thing you can do is to have the Arduino echo back everything that you send it. To accomplish this, the Arduino basically just needs to monitor its serial input buffer and print any character that it receives. To do this, you need to implement two new commands from the `Serial` object:

- `Serial.available()` returns the number of characters (or bytes) that are currently stored in the Arduino's incoming serial buffer. Whenever it's more than zero, you will read the characters and echo them back to the computer.
- `Serial.read()` reads and returns the next character that is available in the buffer.

Note that each call to `Serial.read()` will only return 1 byte, so you need to run it for as long as `Serial.available()` is returning a value greater than zero. Each time `Serial.read()` grabs a byte, that byte is removed from the buffer, as well, so the next byte is ready to be read. With this knowledge, you can now write and load the echo program in Listing 6-3 on to your Arduino.



Listing 6-3: Arduino Serial Echo Test—echo.ino

```
//Echo every character

char data; //Holds incoming character

void setup()
{
  Serial.begin(9600); //Serial Port at 9600 baud
}

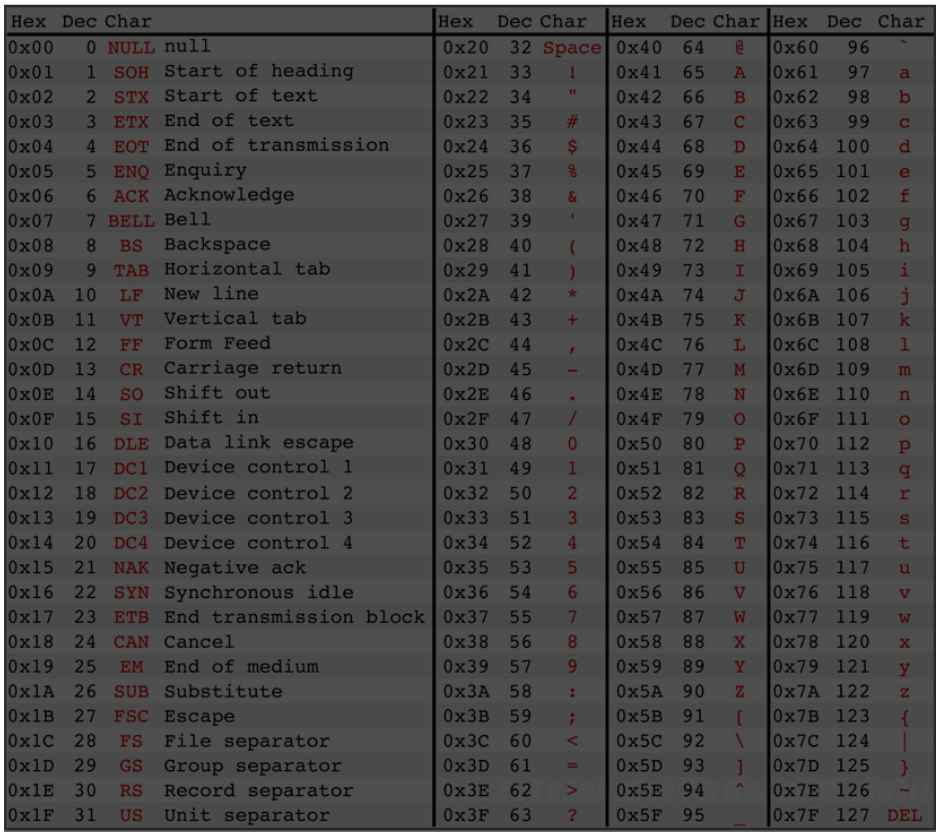
void loop()
{
  //Only print when data is received
  if (Serial.available() > 0)
  {
    data = Serial.read(); //Read byte of data
    Serial.print(data);   //Print byte of data
  }
}
```

Launch the serial monitor and type anything you want into the text entry field. As soon as you press Send, whatever you typed is echoed back and displayed in the serial monitor. You have already selected to append a "newline" to the end of each command, which will ensure that each response is on a new line. That is why `Serial.print()` is used instead of `Serial.println()` in the preceding sketch.

Understanding the Differences Between Chars and Ints

When you send an alphanumeric character via the serial monitor, you aren't actually passing a "5", or an "A". You're sending a byte that the computer interprets as a character. In the case of serial communication, the ASCII character set is used to represent all the letters, number, symbols, and special commands that you might want to send. The base ASCII character set, shown in Figure 6-9, is a 7-bit set and contains a total of 128 unique characters or commands.

When reading a value that you've sent from the computer, as you did in Listing 6-3, the data must be read as a `char` type. Even if you are only expecting to send numbers from the serial terminal, you need to read values as a character first, and then convert as necessary. For example, if you were to modify the code to declare `data` as type `int`, sending a value of `5` would return `53` to the serial monitor because the decimal representation of the character 5 is the number 53. You can confirm this by looking at the ASCII reference table in Figure 6-9.



Hex	Dec	Char		Hex	Dec	Char		Hex	Dec	Char		Hex	Dec	Char	
0x00	0	NULL	null	0x20	32	Space		0x40	64	@		0x60	96	`	
0x01	1	SOH	Start of heading	0x21	33	!		0x41	65	A		0x61	97	a	
0x02	2	STX	Start of text	0x22	34	"		0x42	66	B		0x62	98	b	
0x03	3	ETX	End of text	0x23	35	#		0x43	67	C		0x63	99	c	
0x04	4	EOT	End of transmission	0x24	36	$		0x44	68	D		0x64	100	d	
0x05	5	ENQ	Enquiry	0x25	37	%		0x45	69	E		0x65	101	e	
0x06	6	ACK	Acknowledge	0x26	38	&		0x46	70	F		0x66	102	f	
0x07	7	BELL	Bell	0x27	39	'		0x47	71	G		0x67	103	g	
0x08	8	BS	Backspace	0x28	40	(		0x48	72	H		0x68	104	h	
0x09	9	TAB	Horizontal tab	0x29	41	)		0x49	73	I		0x69	105	i	
0x0A	10	LF	New line	0x2A	42	*		0x4A	74	J		0x6A	106	j	
0x0B	11	VT	Vertical tab	0x2B	43	+		0x4B	75	K		0x6B	107	k	
0x0C	12	FF	Form Feed	0x2C	44	,		0x4C	76	L		0x6C	108	l	
0x0D	13	CR	Carriage return	0x2D	45	-		0x4D	77	M		0x6D	109	m	
0x0E	14	SO	Shift out	0x2E	46	.		0x4E	78	N		0x6E	110	n	
0x0F	15	SI	Shift in	0x2F	47	/		0x4F	79	O		0x6F	111	o	
0x10	16	DLE	Data link escape	0x30	48	0		0x50	80	P		0x70	112	p	
0x11	17	DC1	Device control 1	0x31	49	1		0x51	81	Q		0x71	113	q	
0x12	18	DC2	Device control 2	0x32	50	2		0x52	82	R		0x72	114	r	
0x13	19	DC3	Device control 3	0x33	51	3		0x53	83	S		0x73	115	s	
0x14	20	DC4	Device control 4	0x34	52	4		0x54	84	T		0x74	116	t	
0x15	21	NAK	Negative ack	0x35	53	5		0x55	85	U		0x75	117	u	
0x16	22	SYN	Synchronous idle	0x36	54	6		0x56	86	V		0x76	118	v	
0x17	23	ETB	End transmission block	0x37	55	7		0x57	87	W		0x77	119	w	
0x18	24	CAN	Cancel	0x38	56	8		0x58	88	X		0x78	120	x	
0x19	25	EM	End of medium	0x39	57	9		0x59	89	Y		0x79	121	y	
0x1A	26	SUB	Substitute	0x3A	58	:		0x5A	90	Z		0x7A	122	z	
0x1B	27	FSC	Escape	0x3B	59	;		0x5B	91	[		0x7B	123	{	
0x1C	28	FS	File separator	0x3C	60	<		0x5C	92	\		0x7C	124	|	
0x1D	29	GS	Group separator	0x3D	61	=		0x5D	93	]		0x7D	125	}	
0x1E	30	RS	Record separator	0x3E	62	>		0x5E	94	^		0x7E	126	~	
0x1F	31	US	Unit separator	0x3F	63	?		0x5F	95	_		0x7F	127	DEL	

Credit: Ben Borowiec, www.benborowiec.com.

Figure 6-9: ASCII table

However, you'll often want to send numeric values to the Arduino. So how do you do that? You can do so in a few ways. First, you can simply compare the characters directly. If you want to turn an LED on when you send a `1`, you can compare the character values like this: `if (Serial.read() == '1')`. Note that the single quotes around the `'1'` indicate that it should be treated like a character.

A second option is to convert each incoming byte to an integer by subtracting the zero-valued character, like this: `int val = Serial.read() - '0'`. However, this doesn't work very well if you intend to send numbers that are greater than 9, because they will be multiple digits. To deal with this, the Arduino IDE includes a handy function called `parseInt()` that attempts to extract integers from a serial data stream. The examples that follow elaborate on these techniques.

Sending Single Characters to Control an LED

Before your dive into parsing larger strings of multiple-digit numbers, start by writing a sketch that uses a simple character comparison to control an LED.

You'll send a `1` to turn an LED on, and a `0` to turn it off. Wire an LED up to pin 9 of your Arduino as shown in Figure 6-10.

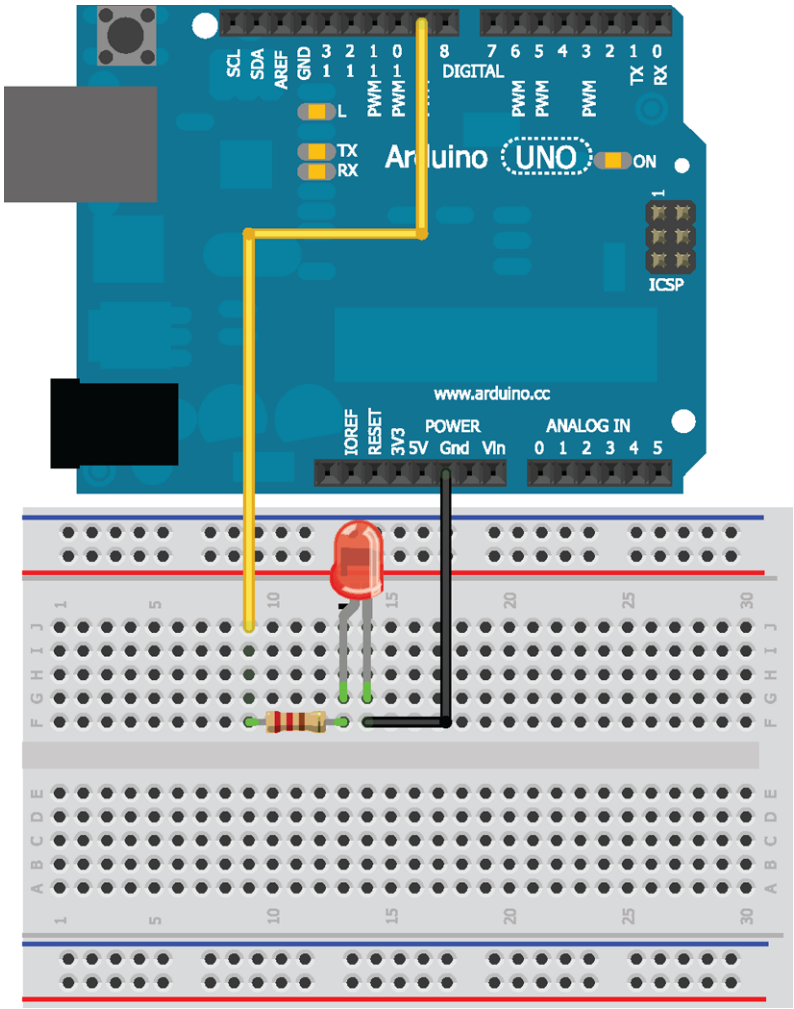


Figure 6-10: Single LED connected to Arduino on pin 9

As explained in the previous section, when only sending a single character, the easier thing to do is to do a simple character comparison in an `if` statement. Each time a character is added to the buffer, it is compared to a `'0'` or a `'1'`, and the appropriate action is taken. Load up the code in Listing 6-4 and experiment with sending a `0` or a `1` from the serial terminal.

Listing 6-4: Single LED Control using Characters—single_char_control.ino

```
//Single Character Control of an LED

const int LED=9;

char data; //Holds incoming character

void setup()
{
  Serial.begin(9600); //Serial Port at 9600 baud
  pinMode(LED, OUTPUT);
}

void loop()
{
  //Only act when data is available in the buffer
  if (Serial.available() > 0)
  {
    data = Serial.read(); //Read byte of data
    //Turn LED on
    if (data == '1')
    {
      digitalWrite(LED, HIGH);
      Serial.println("LED ON");
    }
    //Turn LED off
    else if (data == '0')
    {
      digitalWrite(LED, LOW);
      Serial.println("LED OFF");
    }
  }

}
```

Note that an `else if` statement is used instead of a simple `else` statement. Because your terminal is also set to send a newline character with each transmission, it's critical to clear these from the buffer. `Serial.read()` will read in the newline character, see that is not equivalent to a `'0'` or a `'1'`, and it will be overwritten the next time `Serial.read()` is called. If just an `else` statement were used, both `'0'` and `'\n'` would trigger turning the LED off. Even when sending a `'1'`, the LED would immediately turn off again when the `'\n'` was received!

Sending Lists of Values to Control an RGB LED

Sending a single command character is fine for controlling a single digital pin, but what if you want to accomplish some more complex control schemes? This section explores sending multiple comma-separate values to simultaneously command multiple devices. To facilitate testing this, wire up a common cathode RGB LED as shown in Figure 6-11.

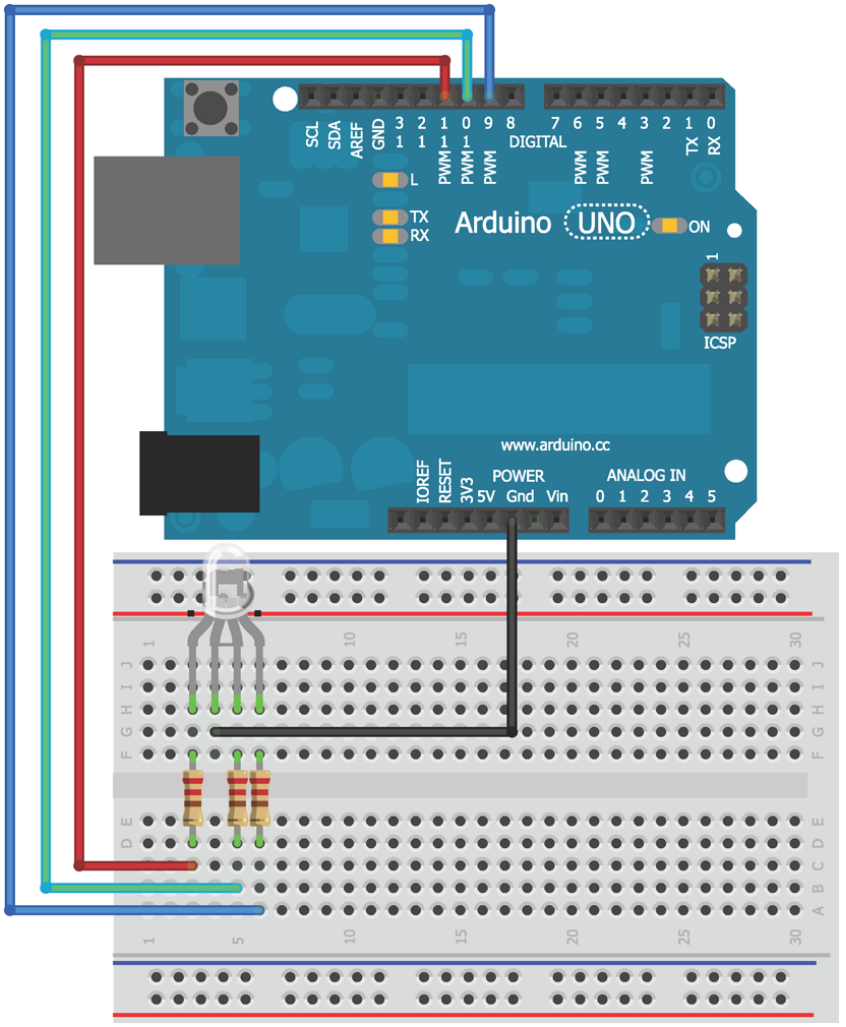


Figure 6-11: RGB LED connected to Arduino

To control this RGB LED, you send three separate 8-bit values (0–255) to set the brightness of each LED color. For example, to set all the colors to full brightness, you send `"255,255,255"`. This presents a few challenges:

- You need to differentiate between numbers and commas.
- You need to turn this sequence of characters into integers that you can pass to `analogWrite()` functions.
- You need to be able to handle the fact that values could be one, two, or three digits.

Thankfully, the Arduino IDE implements a very handy function for identifying and extracting integers: `Serial.parseInt()`. Each call to this function waits until a non-numeric value enters the serial buffer, and converts the previous digits into an integer. The first two values are read when the commas are detected, and the last value is read when the newline is detected.

To test this function for yourself, load the program shown in Listing 6-5 on to your Arduino.

Listing 6-5: RGB LED Control via Serial—list_control.ino

```
//Sending Multiple Variables at Once

//Define LED pins
const int RED   =11;
const int GREEN =10;
const int BLUE  =9;

//Variables for RGB levels
int rval = 0;
int gval = 0;
int bval = 0;

void setup()
{
  Serial.begin(9600); //Serial Port at 9600 baud

  //Set pins as outputs
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);
}

void loop()
{
  //Keep working as long as data is in the buffer
  while (Serial.available() > 0)
```

```
  {
    rval = Serial.parseInt();  //First valid integer
    gval = Serial.parseInt();  //Second valid integer
    bval = Serial.parseInt();  //Third valid integer

    if (Serial.read() == '\n') //Done transmitting
    {
      //set LED
      analogWrite(RED, rval);
      analogWrite(GREEN, gval);
      analogWrite(BLUE, bval);
    }
  }
}
```

The program keeps looking for the three integer values until a newline is detected. Once this happens, the values that were read are used to set the brightness of the LEDs. To use this, open the serial monitor and enter three values between 0 and 255 separated by a comma, like `"200,30,180"`. Try mixing all kinds of pretty colors!

Talking to a Desktop App

Eventually, you're bound to get bored of doing all your serial communication through the Arduino serial monitor. Conveniently, just about any desktop programming language you can think of has libraries that allow it to interface with the serial ports in your computer. You can use your favorite desktop programming language to write programs that send serial commands to your Arduino and that react to serial data being transmitted from the Arduino to the computer.

In this book, Processing is the desktop programming language of choice because it is very similar to the Arduino language that you have already become familiar with. In fact, the Arduino programming language is based on Processing! Other popular desktop languages (that have well-documented serial communication libraries) include Python, PHP, Visual Basic, C, and more. First, you'll learn how to read transmitted serial data in Processing, and then you'll learn how you can use Processing to create a simple graphical user interface (GUI) to send commands to your Arduino.

Talking to Processing

Processing has a fairly simple programming interface, and it's similar to the one you've already been using for the Arduino. In this section, you install Processing, and then write a simple graphical interface to generate a graphical

output based on serial data transmitted from your Arduino. Once that's working, you implement communication in the opposite direction to control your Arduino from a GUI on your computer.

Installing Processing

First things first, you need to install Processing on your machine. This is the same process that you followed in the first chapter to get the Arduino IDE installed. Visit `http://processing.org/download/` (or find the download link on the digital content page for this chapter on `www.exploringarduino.com`) and download the compressed package for your operating system. Simply unzip it to your preferred location and you are ready to go! Run the Processing application, and you should see an IDE that looks like the one shown in Figure 6-12.

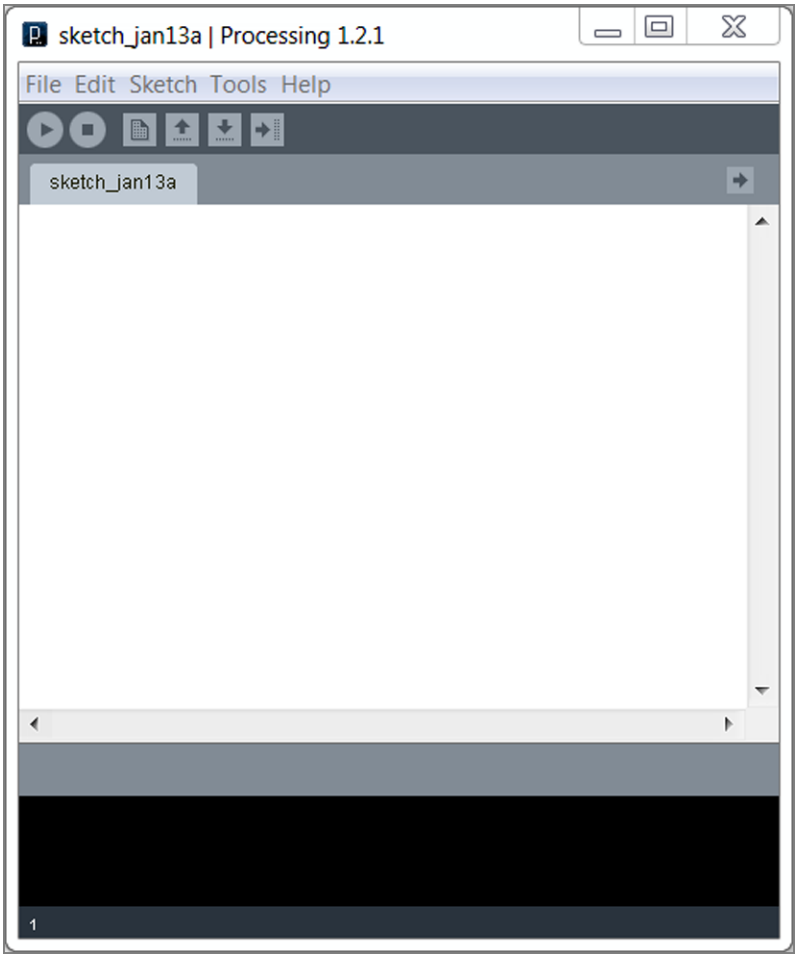


Figure 6-12: The Processing IDE. Does it look familiar?

Controlling a Processing Sketch from Your Arduino

For your first experiment with Processing, you use a potentiometer connected to your Arduino to control the color of a window on your computer. Wire up your Arduino with a potentiometer, referencing Figure 6-6 again. You already know the Arduino code necessary to send the analog values from the potentiometer to your computer. The fact that you are now feeding the serial data into Processing does not have any impact on the way you transmit it.

Reference the code in Listing 6-6 and load it on to your Arduino. It sends an updated value of the potentiometer to the computer's serial port every 50 milliseconds. The 50ms is important; if you were to send it as fast as possible, the Processing sketch wouldn't be able to handle it as quickly as you are sending it, and you would eventually overflow the serial input buffer on your computer.

Listing 6-6: Arduino Code to send Data to the Computer—pot_to_processing/arduino_read_pot

```
//Sending POT value to the computer

const int POT=0; //Pot on analog pin 0

int val; //For holding mapped pot value

void setup()
{
  Serial.begin(9600); //Start Serial
}

void loop()
{
  val = map(analogRead(POT), 0, 1023, 0, 255); //Read and map POT
  Serial.println(val);                         //Send value
  delay(50);                                   //Delay so we don't flood
                                               //the computer
}
```

Now comes the interesting part: writing a Processing sketch to do something interesting with this incoming data. The sketch in Listing 6-7 reads the data in the serial buffer and adjusts the brightness of a color on the screen of your computer based on the value it receives. First, copy the code from Listing 6-7 into a new Processing sketch. You need to change just one important part. The Processing sketch needs to know which serial port to expect data to arrive on. This is the same port that you've been programming the Arduino from. In the

following listing, replace `"COM3"` with your serial port number. Remember that on Linux and Mac it will look like `/dev/ttyUSB0`, for example. You can copy the exact name from within the Arduino IDE if you are unsure.

```
port = new Serial(this, "COM3", 9600); //setup serial
```

Listing 6-7: Processing Code to Read Data and Change Color on the Screen—pot_to_processing/processing_display_color

```
//Processing Sketch to Read Value and Change Color on the Screen

//Import and initialize serial port library
import processing.serial.*;
Serial port;

float brightness = 0; //For holding value from pot

void setup()
{
  size(500,500);                          //Window size
  port = new Serial(this, "COM3", 9600); //Set up serial
  port.bufferUntil('\n');                 //Set up port to read until
                                          //newline
}

void draw()
{
  background(0,0,brightness); //Updates the window
}

void serialEvent (Serial port)
{
  brightness = float(port.readStringUntil('\n')); //Gets val
}
```

After you've loaded the code into your Processing IDE and set the serial port properly, make sure that the Arduino serial monitor isn't open. Only one program on your computer can have access to the serial port at a time. Click the Run button in the Processing IDE (the button in the top left of the window with a triangle); when you do so, a small window will pop up (see Figure 6-13). As you turn the potentiometer, you should see the color of the window change from black to blue.

Now that you've seen it working, let's walk through the code to gain a better understanding of how the Processing sketch is working. Unlike in Arduino, the serial library is not imported automatically. By calling `import processing.serial.*;` and `Serial port;` you are importing the serial library and making a serial object called `port`.

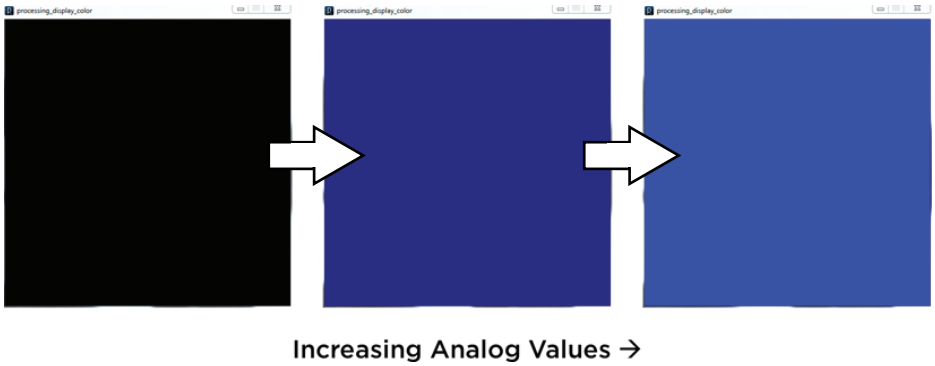


Figure 6-13: Example windows from Processing sketch

Like the Arduino, Processing has a `setup()` function that runs once at the beginning of the sketch. In this sketch, it sets up the serial port and creates a window of size 500×500 pixels with the command `size(500,500)`. The command `port = new Serial(this, "COM3", 9600)` tells Processing everything it needs to know about creating the serial port. The instance (referred to as "port") will run in this sketch and communicate on COM3 (or whatever your serial port is) at 9600 baud. The Arduino and the program on your computer must agree on the speed at which they communicate; otherwise, you'll get garbage characters. `port.bufferUntil('\n')` tells Processing to buffer the serial input and not do anything with the information until it sees a newline character.

Instead of `loop()`, Processing defines other special functions. This program uses `draw()` and `serialEvent()`. The `draw()` function is similar to Arduino's `loop()`; it runs continuously and updates the display. The `background()` function sets the color of the window by setting red, green, and blue values (the three arguments of the function). In this case, the value from the potentiometer is controlling the blue intensity, and red and green are set to `0`. You can change what color your pot is adjusting simply by swapping which argument `brightness` is filling in. RGB color values are 8-bit values ranging from `0` to `255`, which is why the potentiometer is mapped to those values before being transmitted.

`serialEvent()` is called whenever the `bufferUntil()` condition that you specified in the `setup()` is met. Whenever a newline character is received, the `serialEvent()` function is triggered. The incoming serial information is read as a string with `port.readStringUntil('\n')`. You can think of a string as an array of text. To use the string as a number, you must convert it to a floating-point number with `float()`. This sets the brightness variable, changing the background color of the application window.

To stop the application and close the serial port, click the Stop button in the Processing IDE; it's the square located next to the Run button.

SUDOGLOVE PROCESSING DEBUGGER

The SudoGlove is a control glove that drives RC cars and controls other hardware. I developed a Processing debugging display that graphically shows the values of various sensors. You can learn more about it here: `www.sudoglove.com`.

Download the source code for the Processing display here: `www.jeremyblum.com/2011/03/25/processing-based-sudoglove-visual-debugger/`. You can also find this source code on the Wiley website shown at the beginning of this chapter.

Sending Data from Processing to Your Arduino

The obvious next step is to do the opposite. Wire up an RGB LED to your Arduino as shown in Figure 6-11 and load on the same program from earlier that you used to receive a string of three comma-separated values for setting the red, green, and blue intensities (Listing 6-5). Now, instead of sending a string of three values from the serial monitor, you select a color using a color picker.

Load and run the code in Listing 6-8 in Processing, remembering to adjust the serial port number accordingly as you did with the previous sketch. Processing sketches automatically load collateral files from a folder called "data" in the sketch folder. The hsv.jpg file is included in the code download for this chapter. Download it and place it in a folder named "data" in the same directory as your sketch. Processing defaults to saving sketches in your Documents folder. The structure will look similar to the one shown in Figure 6-14.

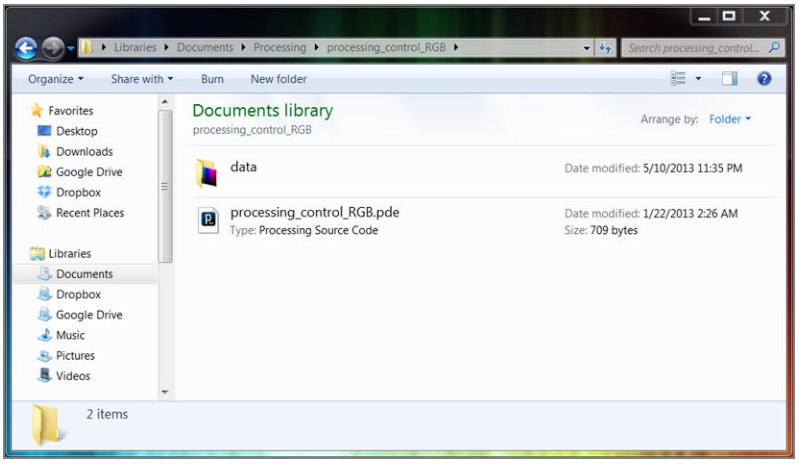


Figure 6-14: Folder structure

The image in the data folder will serve as the color selector.

Listing 6-8: Processing Sketch to Set Arduino RGB Colors— processing_control_RGB/processing_control_RGB

```
import processing.serial.*; //Import serial library
PImage img;                 //Image object
Serial port;                //Serial port object

void setup()
{
  size(640,256);                          //Size of HSV image
  img = loadImage("hsv.jpg");             //Load in background image
  port = new Serial(this, "COM9", 9600); //Open serial port
}

void draw()
{
  background(0);   //Black background
  image(img,0,0);  //Overlay image
}

void mousePressed()
{
  color c = get(mouseX, mouseY); //Get the RGB color where mouse was
pressed
  String colors = int(red(c))+","+int(green(c))+","+int(blue(c))+"\n"; //
extract
values from color
  print(colors);      //Print colors for debugging
  port.write(colors); //Send values to Arduino
}
```

When you execute the program, you should see a screen like the one shown in Figure 6-15 pop up. Click different colors and the RGB values will be transmitted to the Arduino to control the RGB LED's color. Note that the serial console also displays the commands being sent to assist you in any debugging.

After you've finished staring at all the pretty colors, look back at the code and consider how it's working. As before, the serial library is imported and a serial object called `port` is created. A `PImage` object call `img` is also created. This will hold the background image. In the `setup()`, the serial port is initialized, the display window is set to the size of the image, and image is imported into the image object by calling `img = loadImage("hsv.jpg").`

In the `draw()` function, the image is loaded in the window with `image(img,0,0).` `img` is the image you want to draw in the window, and `0, 0` are coordinates where the image will start to be drawn. `0,0` is the top left of the application window.

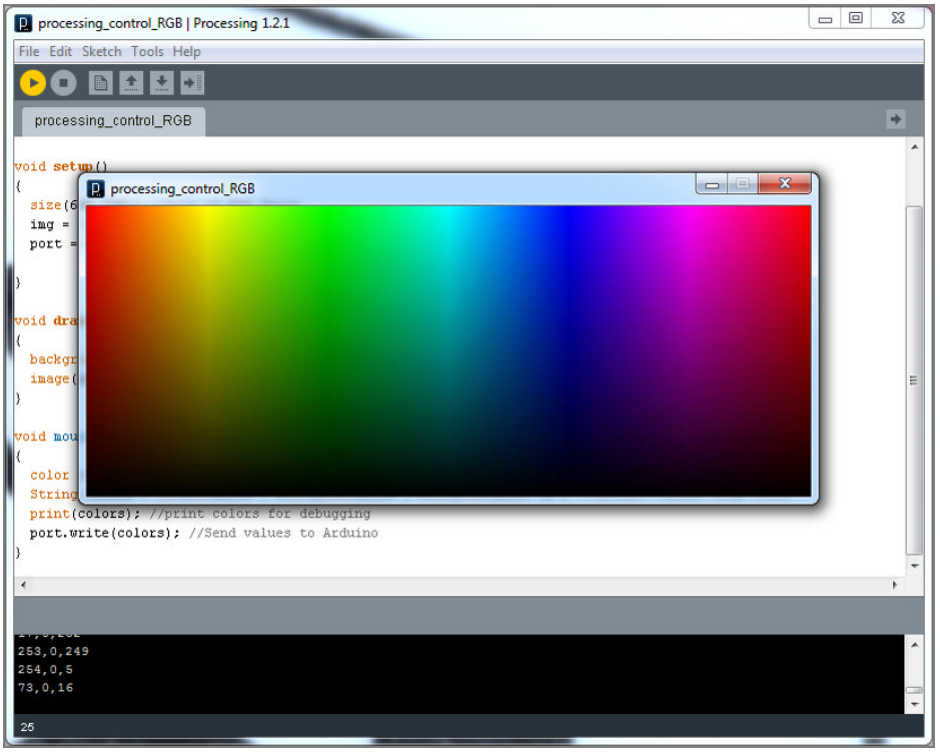


Figure 6-15: Processing color selection screen

Every time the mouse button is pressed, the `mousePressed()` function is called. The color of the pixel where you clicked is saved to a `color` object named `c`. The `get()` method tells the application where to get the color from (in this case, the location of the mouse's X and Y position in the sketch). The sketch converts the object `c` into a string that can be sent to the Arduino by converting to integers representing red, green, and blue. These values are also printed to the Processing console so that you can see what is being sent.

Ensure that the Arduino is connected and programmed with the code from Listing 6-5. Run the processing sketch (with the correct serial port specified) and click around the color map to adjust the color of the LED connected to your Arduino.

Learning Special Tricks with the Arduino Leonardo (and Other 32U4-Based Arduinos)

The Leonardo, in addition to other Arduinos that implement MCUs that connect directly to USB, has the unique ability to emulate nonserial devices such as a keyboard or mouse. In this section you learn about using a Leonardo to

emulate these devices. You need to be extremely careful to implement these functions in a way that does not make reprogramming difficult. For example, if you write a sketch that emulates a mouse and continuously moves your pointer around the screen, you might have trouble clicking on the Upload button in the Arduino IDE! In this section, you learn a few tricks that you can use to avoid these circumstances.

TIP If you get stuck with a board that's too hard to program due to its keyboard or mouse input, hold down the Reset button and release it while pressing the Upload button in the Arduino IDE to reprogram it.

When you first connect a Leonardo to a Windows computer, you need to install drivers, just as you did with the Arduino Uno in the first chapter. Follow the same directions at `http://arduino.cc/en/Guide/ArduinoLeonardoMicro#toc8` for Leonardo-specific instructions. (These instructions are also linked from the digital content page for this chapter from `www.exploringarduino.com`.)

Emulating a Keyboard

Using the Leonardo's unique capability to emulate USB devices, you can easily turn your Arduino into a keyboard. Emulating a keyboard allows you to easily send key-combination commands to your computer or type data directly into a file that is open on your computer.

Typing Data into the Computer

The Leonardo can emulate a USB keyboard, sending keystrokes and key combinations. This section explores how to use both concepts. First, you write a simple program that records data from a few analog sensors into a comma-separated-value (.csv) format that you can later open up with Excel or Google spreadsheets to generate a graph of the values.

Start by opening the text editor of your choice and saving a blank document with a .csv extension. To do this, you can generally choose the file type in the Save dialog, select "All Files," and manually type the file name with the extension, such as "data.csv." The demo video also shows how to create a .csv file.

Next, create a simple circuit like the one shown in Figure 6-16. It will monitor both light and temperature levels using analog sensors that you have already seen in Chapter 3, "Reading Analog Sensors." In addition to the sensors, the circuit includes a button for turning the logging functionality on and off, and an LED that will indicate whether it is currently logging data.

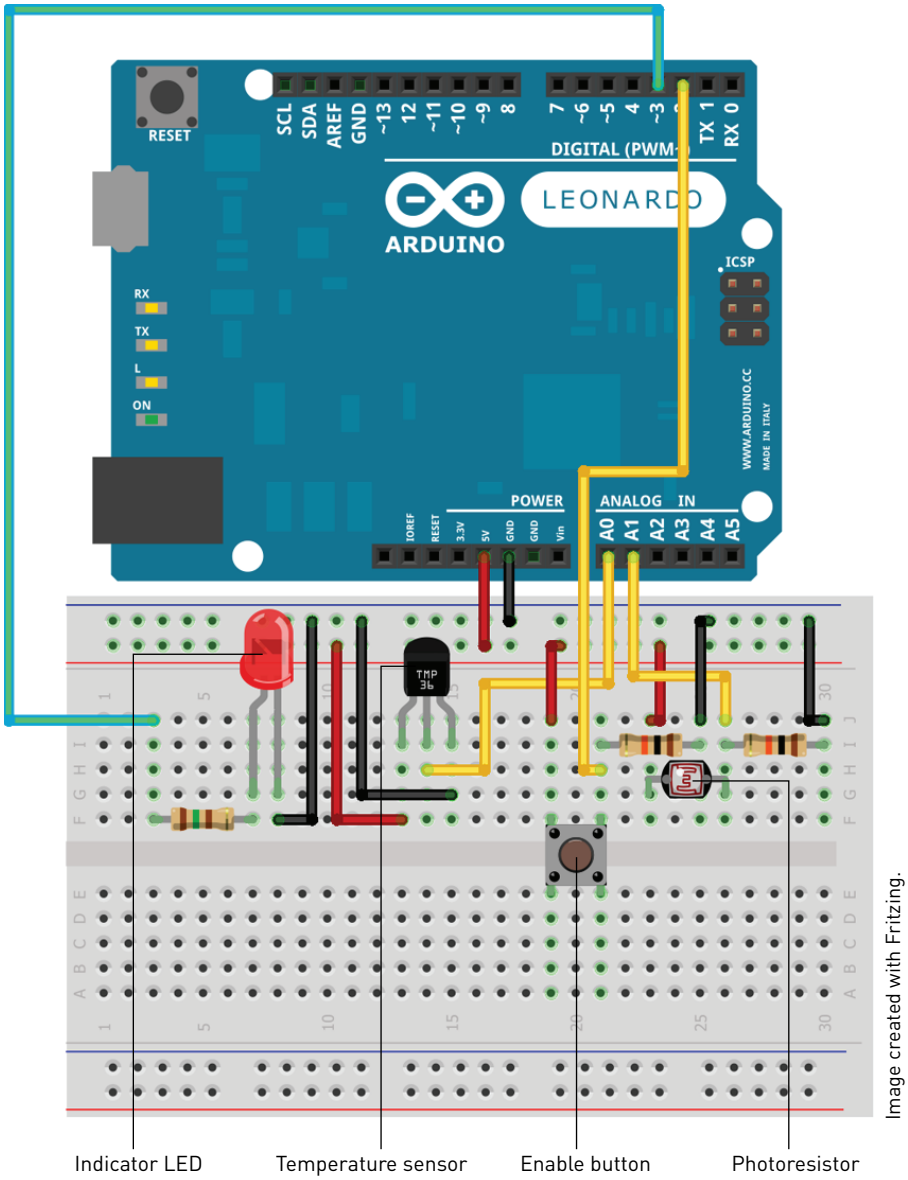


Image created with Fritzing.

Figure 6-16: Temperature and light sensor circuit

Using the same debouncing function that you implemented in Chapter 2, "Digital Inputs, Outputs, and Pulse-Width Modulation," you use the pushbutton to toggle the logging mode on and off. While in logging mode, the Arduino polls the sensors and "types" those values into your computer in a comma-separated format once every second. An indicator LED remains illuminated while you are logging data. Because you want the Arduino to be constantly polling the state

of the button, you cannot use a `delay()` function to wait 1000ms between each update. Instead, you use the `millis()` function, which returns the number of milliseconds since the Arduino was last reset. You can make the Arduino send data every time the `millis()` function returns a multiple of 1000ms, effectively creating a nonblocking 1-second delay between transmissions. To do this, you can use the modulo operator (`%`). Modulo returns the remainder of a division. If, for example, you executed `1000%1000`, you would find that the result is 0 because 1000/1000=1, with a remainder of 0. `1500%1000`, on the other hand, would return 500 because 1500/1000 is equal to 1, with a remainder of 500. If you take the modulus of `millis()` with `1000`, the result is zero every time `millis()` reaches a value that is a multiple of 1000. By checking this with an `if()` statement, you can execute code once every second.

Examine the code in Listing 6-9 and load it onto your Arduino Leonardo. Ensure that you've selected "Arduino Leonardo" from the Tools > Board menu in the Arduino IDE.

Listing 6-9: Temperature and Light Data Logger—csv_logger.ino

```
//Light and Temp Logger

const int TEMP   =0;      //Temp sensor on analog pin 0
const int LIGHT  =1;      //Light sensor on analog pin 1
const int LED    =3;      //Red LED on pin 13
const int BUTTON =2;      //The button is connected to pin 2

boolean lastButton = LOW;    //Last button state
boolean currentButton = LOW; //Current button state
boolean running = false;     //Not running by default
int counter = 1;             //An index for logged data entries

void setup()
{
  pinMode (LED, OUTPUT);   //Set blue LED as output
  Keyboard.begin();        //Start keyboard emulation
}

void loop()
{
  currentButton = debounce(lastButton);           //Read debounced state

  if (lastButton == LOW && currentButton == HIGH) //If it was pressed…
    running = !running;                           //Toggle running state

  lastButton = currentButton;                     //Reset button value

  if (running)                              //If logger is running
```

```
  {
    digitalWrite(LED, HIGH);              //Turn the LED on
    if (millis() % 1000 == 0)             //If time is multiple
                                          //of 1000ms
    {
      int temperature = analogRead(TEMP); //Read the temperature
      int brightness = analogRead(LIGHT); //Read the light level
      Keyboard.print(counter);            //Print the index number
      Keyboard.print(",");                //Print a comma
      Keyboard.print(temperature);        //Print the temperature
      Keyboard.print(",");                //Print a comma
      Keyboard.println(brightness);       //Print brightness, newline
      counter++;                          //Increment the counter
    }
  }
  else
  {
    digitalWrite(LED, LOW);  //If logger not running, turn LED off
  }
}

/*
* Debouncing Function
* Pass it the previous button state,
* and get back the current debounced button state.
*/
boolean debounce(boolean last)
{
  boolean current = digitalRead(BUTTON);       //Read the button state
  if (last != current)                         //If it's different…
  {
    delay(5);                                  //Wait 5ms
    current = digitalRead(BUTTON);             //Read it again
  }
  return current;                              //Return the current
                                               //value
}
```

Before you test the data logger, let's highlight some of the new functionality that has been implemented in this sketch. Similarly to how you initialized the serial communication, the keyboard communication is initialized by putting `Keyboard.begin()` in the `setup()`.

Each time through `loop()`, the Arduino checks the state of the button and runs the debouncing function that you are already familiar with. When the button is pressed, the value of the *running* variable is inverted. This is accomplished by setting it to its opposite with the `!` operator.

While the Arduino is in *running* mode, the logging step is executed only every 1000ms using the logic described previously. The keyboard functions work very similarly to the serial functions. `Keyboard.print()` "types" the given string into

your computer. After reading the two analog sensors, the Arduino sends the values to your computer as keystrokes. When you use `Keyboard.println()`, the Arduino emulates pressing the Enter or Return key on your keyboard after sending the given string. An incrementing counter and both analog values are entered in a comma-separated format.

Follow the demo video from this chapter's web page to see this sketch in action. Make sure that your cursor is actively positioned in a text document, and then press the button to start logging. You should see the document begin to populate with data. Hold your hand over the light sensor to change the value or squeeze the temperature sensor to see the value increase. When you have finished, press the button again to stop logging. After you save your file, you can import it into the spreadsheet application of your choice and graph it over time. This is shown in the demo video.

NOTE To watch a demo video of the live temperature and light logger, visit `www.exploringarduino.com/content/ch6`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Commanding Your Computer to Do Your Bidding

In addition to typing like a keyboard, you can also use the Leonardo to emulate key combinations. On Windows computers, pressing the Windows+L keys locks the computer screen (On Linux, you can use Control+Alt+L). Using that knowledge paired with a light sensor, you can have your computer lock automatically when you turn the lights off. OS X uses the Control+Shift+Eject, or Control+Shift+Power keys to lock the machine, which can't be emulated by the Leonardo because it cannot send an Eject or Power simulated button press. In this example, you learn how to lock a Windows computer. You can continue to use the same circuit shown in Figure 6-16, though only the light sensor will be used in this example.

Run the previous sketch at a few different light levels and see how the light sensor reading changes. Using this information, you should pick a threshold value below which you will want your computer to lock. (In my room, I found that with the lights off the value was about `300`, and it was about `700` with the lights on. So, I chose a threshold value of `500`.) When the light sensor value drops below that value, the `lock` command will be sent to the computer. You might want to adjust this value for your environment.

Load the sketch in Listing 6-10 on to your Arduino. Just make sure you have your threshold set to a reasonable value first, by testing what light levels in your room correspond to various analog levels. If you pick a poorly calibrated value, it might lock your computer as soon as you upload it!

Listing 6-10: Light-Based Computer Lock—lock_computer.ino

```
//Locks your computer when you turn off the lights

const int LIGHT     =1;   //Light sensor on analog pin 1
const int THRESHOLD =500;  //Brightness must drop below this level
                           //to lock computer

void setup()
{
  Keyboard.begin();
}

void loop()
{
  int brightness = analogRead(LIGHT);    //Read the light level

  if (brightness < THRESHOLD)
  {
    Keyboard.press(KEY_LEFT_GUI);
    Keyboard.press('l');
    delay(100);
    Keyboard.releaseAll();
  }
}
```

After loading the program, try flipping the lights off. Your computer should lock immediately. The following video demo shows this in action. This sketch implements two new keyboard functions: `Keyboard.press()` and `Keyboard.releaseAll()`. Running `Keyboard.press()` is equivalent to starting to hold a key down. So, if you want to hold the Windows key and the L key down at the same time, you run `Keyboard.press()` on each. Then, you delay for a short period of time and run the `Keyboard.releaseAll()` function to let go of, or release, the keys. Special keys are defined on the Arduino website: `http://arduino.cc/en/Reference/KeyboardModifiers`. (This definition table is also linked from the content page for this chapter at `www.exploringarduino.com/content/ch6`.)

NOTE To watch a demo video of the light-activated computer lock, visit `www.exploringarduino.com/content/ch6`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Emulating a Mouse

Using a two-axis joystick and some pushbuttons, you can use an Arduino Leonardo to make your own mouse! The joystick will control the mouse location, and the buttons will control the left, middle, and right buttons of the mouse.

Just like with the keyboard functionality, the Arduino language has some great functions built in that make it easy to control mouse functionality.

First things first, get your circuit set up with a joystick and some buttons as shown in Figure 6-17. Don't forget that your buttons need to have pull-down resistors! The joystick will connect to analog pins 0 and 1. (Joysticks are actually just two potentiometers hooked up to a knob.) When you move the joystick all the way in the x direction, it maxes out the x potentiometer, and the same goes for the y direction.

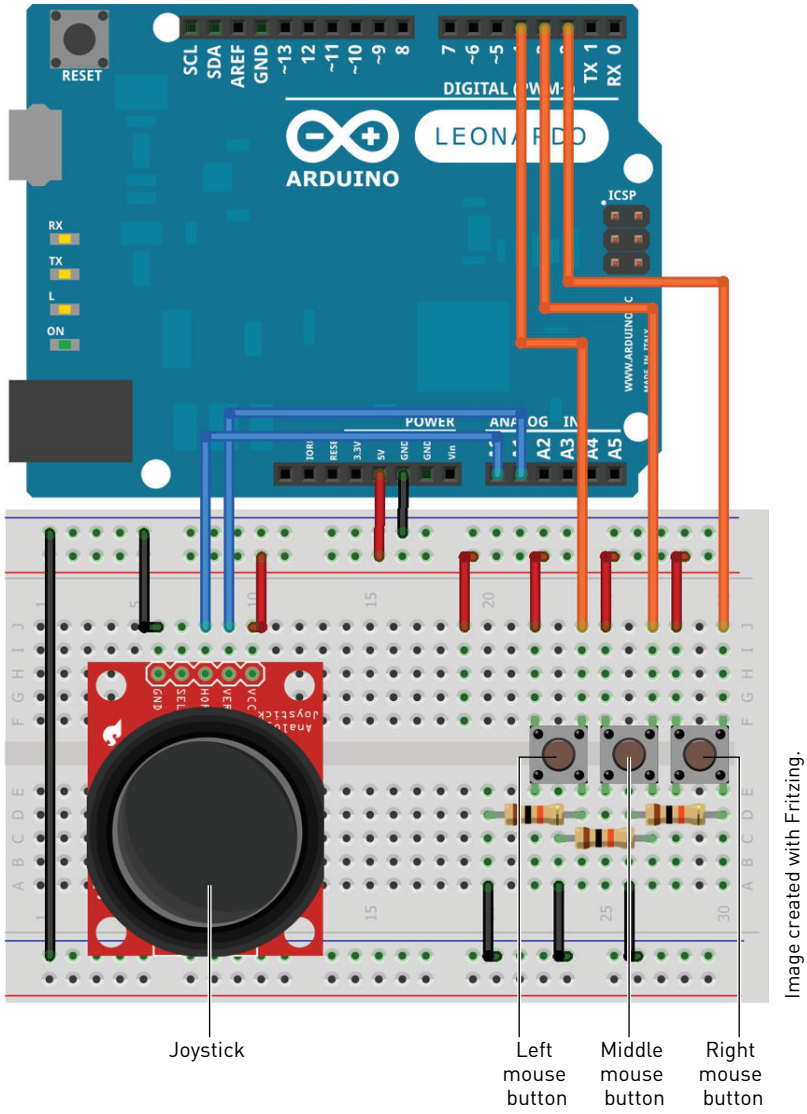


Figure 6-17: Joystick Leonardo mouse circuit

The diagram shows a SparkFun joystick, but any will do. (In the video described after the listing, I used a Parallax joystick.) Depending on the orientation of the joystick, you might need to adjust the bounds of the map function or swap the x/y in the code below.

After you've wired the circuit, it's time to load some code onto the Leonardo. Load up the code in Listing 6-11 and play with the joystick and buttons; the pointer on your screen should respond accordingly.

Listing 6-11: Mouse Control Code for the Leonardo—mouse.ino

```
// Make a Mouse!

const int LEFT_BUTTON   =4;  //Input pin for the left button
const int MIDDLE_BUTTON =3;  //Input pin for the middle button
const int RIGHT_BUTTON  =2;  //Input pin for the right button
const int X_AXIS        =0;  //Joystick x-axis analog pin
const int Y_AXIS        =1;  //Joystick y-axis analog pin

void setup()
{
  Mouse.begin();
}

void loop()
{
  int xVal = readJoystick(X_AXIS);         //Get x-axis movement
  int yVal = readJoystick(Y_AXIS);         //Get y-axis movement

  Mouse.move(xVal, yVal, 0);               //Move the mouse

  readButton(LEFT_BUTTON, MOUSE_LEFT);     //Control left button
  readButton(MIDDLE_BUTTON, MOUSE_MIDDLE); //Control middle button
  readButton(RIGHT_BUTTON, MOUSE_RIGHT);   //Control right button

  delay(5);                                //This controls responsiveness
}

//Reads joystick value, scales it, and adds dead range in middle
int readJoystick(int axis)
{
  int val = analogRead(axis);              //Read analog value
  val = map(val, 0, 1023, -10, 10);        //Map the reading

  if (val <= 2 && val >= -2)               //Create dead zone to stop mouse
drift
    return 0;

  else                                     //Return scaled value
    return val;
```

```
}

//Read a button and issue a mouse command
void readButton(int pin, char mouseCommand)
{
  //If button is depressed, click if it hasn't already been clicked
  if (digitalRead(pin) == HIGH)
  {
    if (!Mouse.isPressed(mouseCommand))
    {
      Mouse.press(mouseCommand);
    }
  }
  //Release the mouse if it has been clicked.
  else
  {
    if (Mouse.isPressed(mouseCommand))
    {
      Mouse.release(mouseCommand);
    }
  }
}
```

This is definitely one of the more complicated sketches that have been covered so far, so it's worth stepping through it to both understand the newly introduced functions and the program flow used to make the joystick mouse.

Each of the button and joystick pins are defined at the top of the sketch, and the mouse library is started in the setup. Each time through the loop, the joystick values are read and mapped to movement values for the mouse. The mouse buttons are also monitored and the button presses are transmitted if necessary.

A `readJoystick()` function was created to read the joystick values and map them. Each joystick axis has a range of 1024 values when read into the analog-to-digital converter (ADC). However, mouse motions are relative. In other words, passing a value of 0 to `Mouse.move()` for each axis will result in no movement on that axis. Passing a positive value for the x-axis will move the mouse to the right, and a negative value will move it to the left. The larger the value, the more the mouse will move. Hence, in the `readJoystick()` function, a value of 0 to 1023 is mapped to a value of `-10` to `10`. A small buffer value around `0` is added where the mouse will not move. This is because even while the joystick is in the middle position, the actual value may fluctuate around `512`. By setting the desired distance back to 0 after being mapped within a certain range, you guarantee that the mouse will not move on its own while the joystick is not being actuated. Once the values are ascertained, `Mouse.move()` is given the x and y values to move the mouse. A third argument for `Mouse.move()` determines the movement of the scroll wheel.

To detect mouse clicks, the `readButton()` function was created so that it can be repeated for each of the three buttons to detect. The function detects the current state of the mouse with the `Mouse.isPressed()` command and controls the mouse accordingly using the `Mouse.press()` and `Mouse.release()` functions.

NOTE To watch a demo video of the joystick mouse controlling a computer pointer, check out `www.exploringarduino.com/content/ch6`. You can also find this video on the Wiley website shown at the beginning of this chapter.

Summary

In this chapter you learned about the following:

- Arduinos connect to your computer via a USB-to-serial converter.
- Different Arduinos facilitate a USB-to-serial conversion using either dedicated ICs or built-in USB functionality.
- Your Arduino can print data to your computer via your USB serial connection.
- You can use special serial characters to format your serial printing with newlines and tabs.
- All serial data is transmitted as character that can be converted to integers in a variety of ways.
- You can send comma-separated integer lists and use integrated functions to parse them into commands for your sketch.
- You can send data from your Arduino to a Processing desktop application.
- You can receive data from a Processing application on your desktop to control peripherals connected to your Arduino.
- An Arduino Leonardo can be used to emulate a keyboard or mouse.